

Representation of a Binary Tree

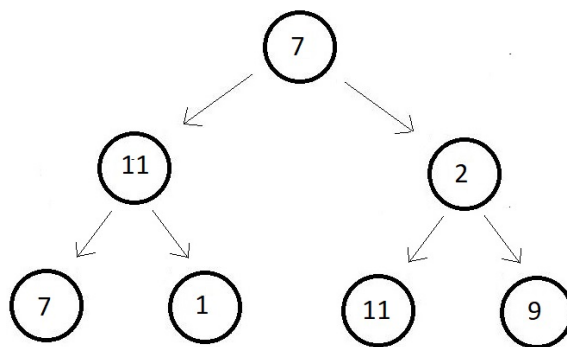
In the previous lecture, we saw the various types of binary trees we have. We looked at their examples, and I am confident that all of you understood everything very clearly. In today's lesson, we'll consider different techniques for representing binary trees in programming, and see which one best suits our needs.

So, the first way to represent a binary tree is by using arrays. We call this 'array representation'. And this method is not very recommended for representing binary trees. You will very soon know why.

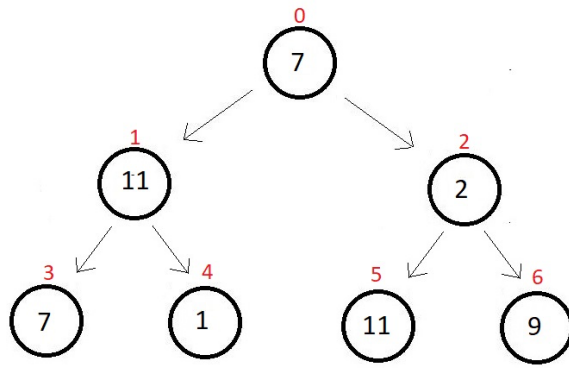
Array representation of Binary trees:

Arrays are linear data structures and for arrays to function, their size must be specified before elements are inserted into them. And this counts as the biggest demerit of representing binary trees using arrays. Suppose you declare an array of size 100, and after storing 100 nodes in it, you cannot even insert a single element further, regardless of all the spaces left in the memory. Another way you'd say is to copy the whole thing again to a new array of bigger size but that is not considered a good practice.

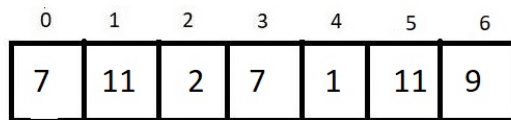
Anyways, we will use an array to represent a binary tree. Suppose we have a binary tree with 7 nodes.



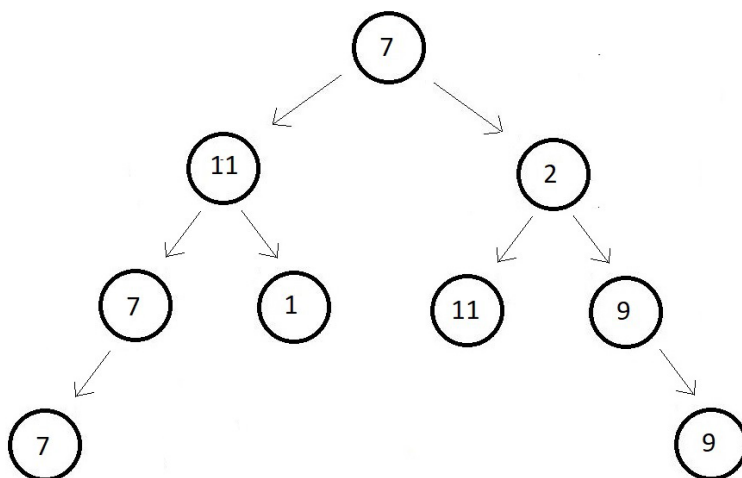
And there are actually a number of ways to represent these nodes via an array. I'll use the most convenient one where we traverse each level starting from the root node and from left to right and mark them with the indices these nodes would belong to.



And now we can simply make an array of length 7 and store these elements at their corresponding indices.

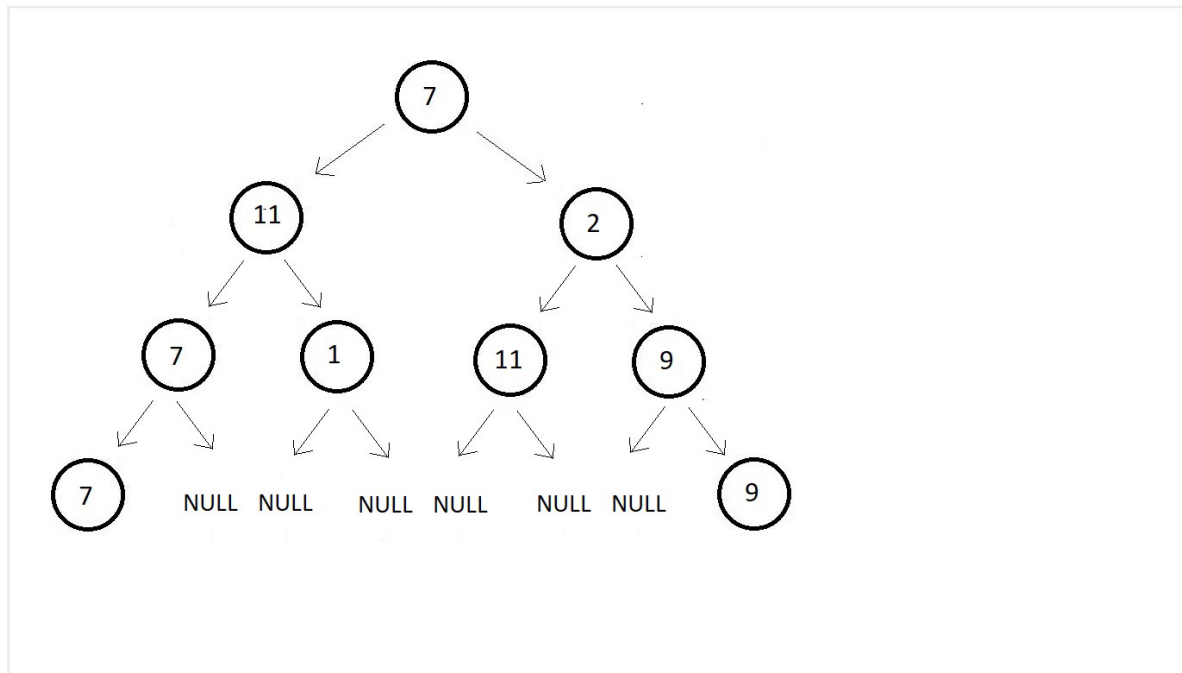


And you might be wondering about the cases where the binary is just not perfect. What if the last level has distributed leaves? Then let me tell you, there is a way out for that as well. Let's consider one case here. A binary tree with 9 nodes, and the last two nodes on the extremities of the last level.

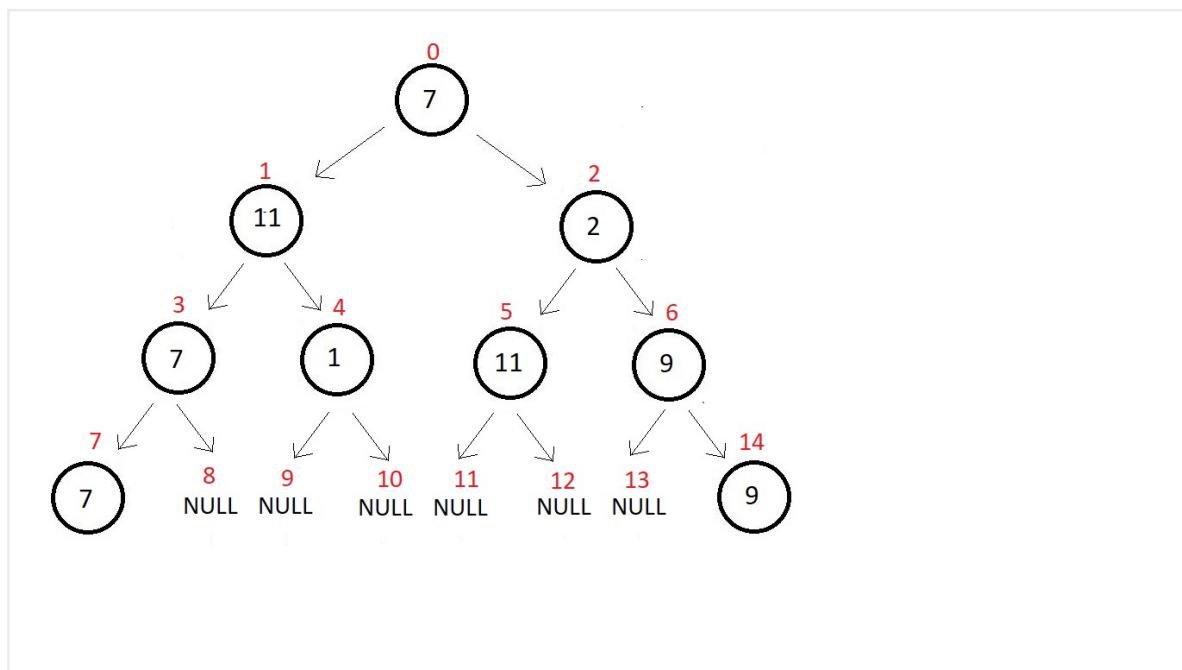


Here, while traversing we get stuck at the 8th index. We don't know if

declaring the last node as the 8th index element makes it a general representation of the tree or not. So, we simply make the tree perfect ourselves. We first assume the remaining vacant places to be NULL.



And now we can easily mark their indices from 0 to 14.



And the array representation of the tree looks something like this. It is an array of length 15.

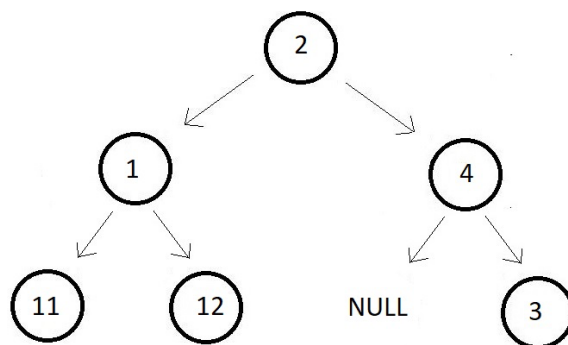
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
7	11	2	7	1	11	9	7							9

But was this even an efficient approach? Like Binary Trees are made only for efficient traversal and insertion and deletion and using an array for that really makes the process troublesome. Each of these operations becomes quite costly to accomplish. And that size constraint was already for making things worse. So overall, we would say that the array representation of a binary is not a very good choice. And what are the other options? We have another method to represent binary trees called the linked representation of binary trees. Don't confuse this with linked lists you have studied. And the reason why I am saying that is because linked lists are lists that are linear data structures.

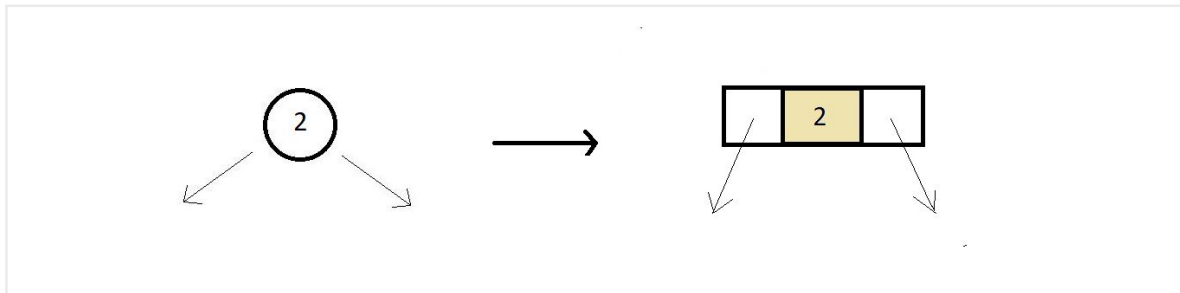
Linked Representation of Binary Trees:

This method of representing binary trees using linked nodes is considered the most efficient method of representation. For this, we use doubly-linked lists. I just hope you recall what doubly-linked lists are. We studied that here in the same playlist [Doubly Linked Lists Explained With Code in C Language](#).

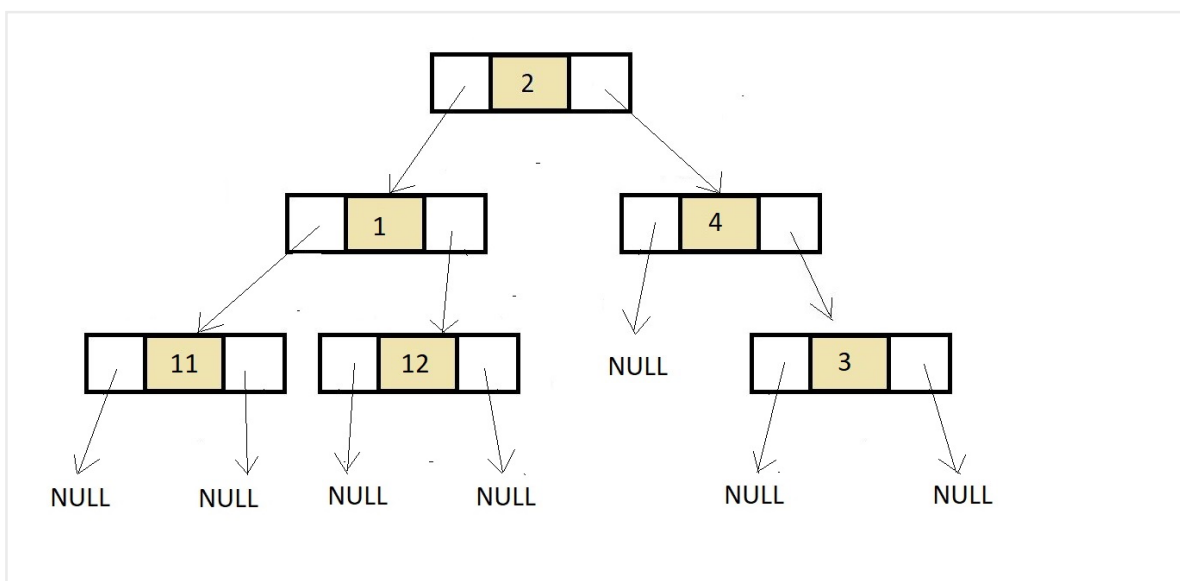
Using links makes the understanding of a binary tree very easy. It actually makes us visualize the tree even. Suppose we have a binary tree of 3 levels.



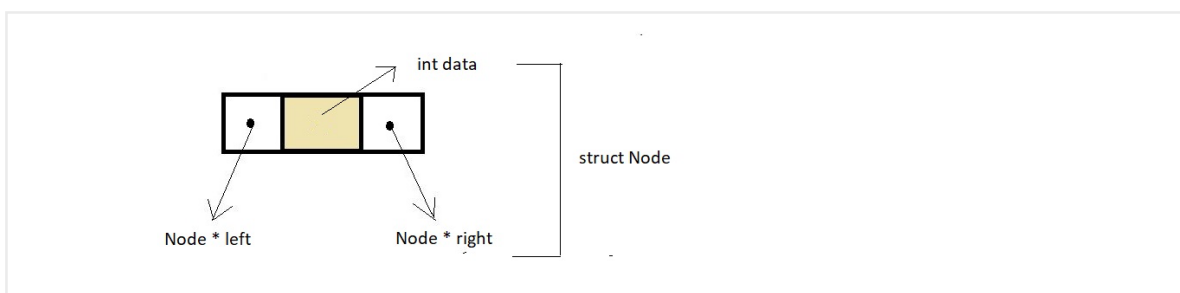
Now if you remember a doubly linked list helped us traversing both to the left and the right. And using that we would create a similar node here, pointing both to the left and the right child node. Follow the below representation of a node here in the linked representation of a binary tree.



You can see how closely this representation resembles a real tree node, unlike the array representation where all the nodes succumbed to a 2D structure. And now we can very easily transform the whole tree into its linked representation which is just how we imagined it would have looked in real life.



So, this was the representation of the binary tree we saw above using linked representation. And what are these nodes? These are structures having three structure members, first a data element to store the data of the node, and then two structure pointers to hold the address of the child nodes, one for the left, and the other for the right.



And let me show you that struct Node definition part in C language:

```
struct node{
    int data;
    struct node* left;
```

```
struct node* right;
};
Copy
```

Code Snippet 1: Creating the struct Node

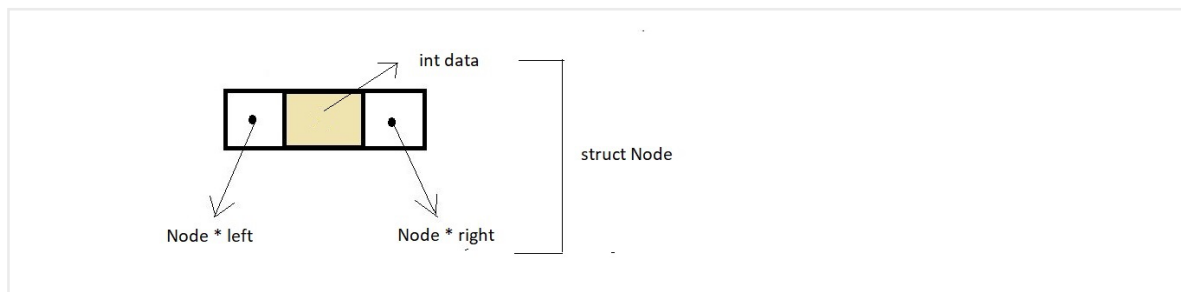
And that was the representation of a binary tree using a linked method. We still have not completely seen the programming part. I hope you could understand and most importantly visualize how the linked representation outperforms the array representation in terms of both implementation and complexity. We will see the programming of this linked representation in the C language in the next lecture.

Linked Representation Of Binary Tree in C

In the last lecture, we saw different representations of a binary tree. We saw its array representation. We saw its linked representation. In contrast to the linked representation of binary trees, we found its array representation to be futile at the end because of its size constraint and the complexity of its implementation.

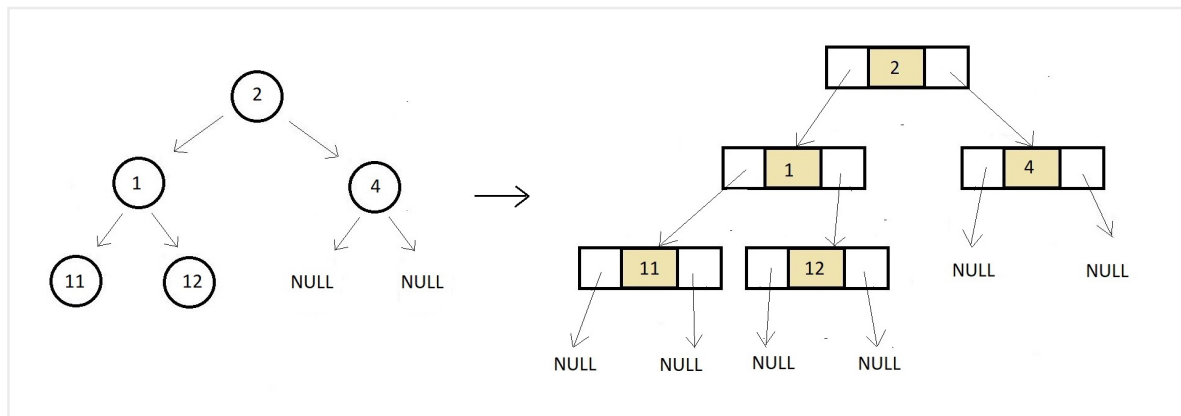
And since the array representation is not that tough to implement I believe, I'll continue with the linked representation, and a quick revision on that should work.

1. We use the concept of doubly-linked lists to represent a node.



2. The left pointer of this node points to the left child and the right pointer points to the right child, and if there is no left or right child, we represent that using a `NULL`.

See how a tree looks in real life, and how efficiently the linked representation of a tree helps us visualize the same.



We also saw the declaration of the structure Node in C. Today we'll see all that in detail. We'll see how these nodes get linked, and the creation of a binary tree via that. I have attached the source code below. Follow it as we proceed.

Understanding the code snippet below:

1. The first thing would be to create the struct Node we discussed yesterday. So, create a struct Node. Inside the structure, we have three embedded elements, first is an integer variable *data* to store the data of the node, second is a struct Node pointer called *left* to store the address of the left child node, and third is again a struct Node pointer called *right* to store the address of the right child node.

```
struct node{
    int data;
    struct node* left;
    struct node* right;
};
```

Copy

Code Snippet 1: Creating the struct Node

2. Our next job would be to create the root node. In main, create struct Node pointer *p*, and assign to it the memory in heap using malloc. Don't forget to include the header file <malloc.h> or <stdlib.h> And then using the pointer operator $\rightarrow$, assign some data to its *data* element and NULL to both the left and the right element of the struct Node *p*.

3. And now we can proceed to creating the further nodes. We have different ways to do that. First one is to copy the whole thing we did for the root node twice for both the children and name them p1 and p2. This will create three separate nodes *p*, *p1* and *p2*. Then just link them together by changing the left element of *p* from NULL to the left node's pointer *p1* and the right element of *p* from NULL to the right node's pointer *p2*.

4. But this is somewhere redundant and not considered a good practice as we are copying the whole thing again and again. So, we would create a dedicated function for creating a node.

Creating the createNode function:

5. Create a struct Node* function *createNode*, and pass the data for the node as the only parameter. Create a struct Node pointer n. Reserve a memory in heap for n using malloc. Basically, do the same thing we did above. Point the left and the right element of the struct n to NULL, and fill the data in the data element. And return the node pointer n you created.

This would simply create a node, and you can now very easily link them as per your wish via main to the other nodes.

```
struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the
heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node
}
```

Copy

Code Snippet 2: Creating the function createNode

Here is the whole source code:

```
#include<stdio.h>
#include<malloc.h>

struct node{
    int data;
    struct node* left;
    struct node* right;
};

struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the
heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node
}

int main(){
    /*
    // Constructing the root node
    struct node *p;
    p = (struct node *) malloc(sizeof(struct node));
```

```

p->data = 2;
p->left = NULL;
p->right = NULL;

// Constructing the second node
struct node *p1;
p1 = (struct node *) malloc(sizeof(struct node));
p1->data = 1;
p1->left = NULL;
p1->right = NULL;

// Constructing the third node
struct node *p2;
p2 = (struct node *) malloc(sizeof(struct node));
p2->data = 4;
p2->left = NULL;
p2->right = NULL;
*/

// Constructing the root node - Using Function (Recommended)
struct node *p = createNode(2);
struct node *p1 = createNode(1);
struct node *p2 = createNode(4);

// Linking the root node with left and right children
p->left = p1;
p->right = p2;
return 0;
}

```

Copy

Code Snippet 3: Implementing binary trees in C

Now we can see if our program doesn't revert any error while we try linking 3 nodes p, p1 and p2.

```

struct node *p = createNode(2);
struct node *p1 = createNode(1);
struct node *p2 = createNode(4);

```

```

p->left = p1;
p->right = p2;

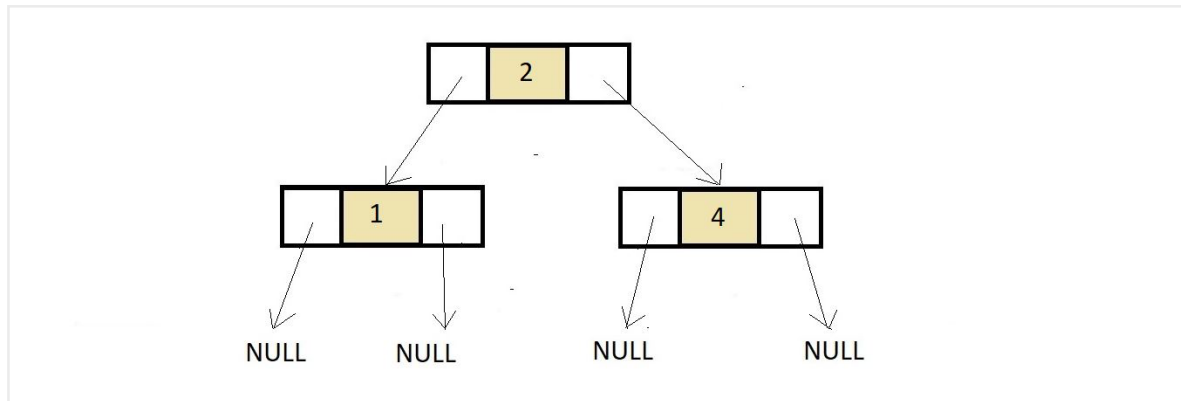
```

Copy

Code Snippet 4: Using createNode to create nodes.

No, it didn't show any error, and this is how we created three nodes and linked two of them to the root node. You can imagine how it would have

looked.



We can create even more of these nodes, and link them accordingly. We will start further with the operations on these very soon. Stay connected.

Traversal in Binary Tree (InOrder, PostOrder and PreOrder Traversals)

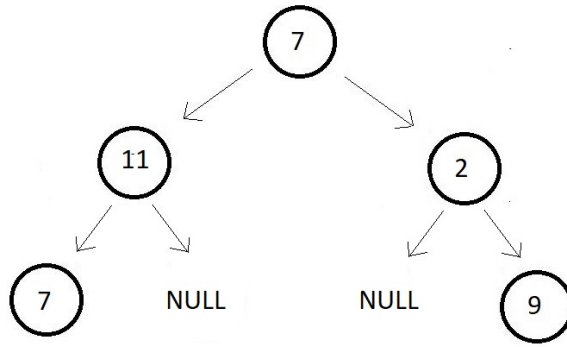
In the last tutorial, we learned to implement the linked representation of binary trees in C. We saw how easy it was for us to visualize the elements decorated as a real tree when implemented using linked representation. Today, we'll learn how to traverse in binary trees.

Traversal in a binary tree:

If you remember the previous lectures where we saw the traversal in linked lists, traversal meant visiting every node in the list at least once. Traversal has several applications, one might want to visit each node and print their values, another might want to add each of those elements. It is, therefore, crucial to study the techniques of traversal.

But things were easy when we used to traverse through the lists, or the arrays, since they were linear structures, and the directions you choose to traverse were limited, either front to rear or from rear to front. But today we'll accomplish the complexity of traversing through a non-linear data structure, binary tree.

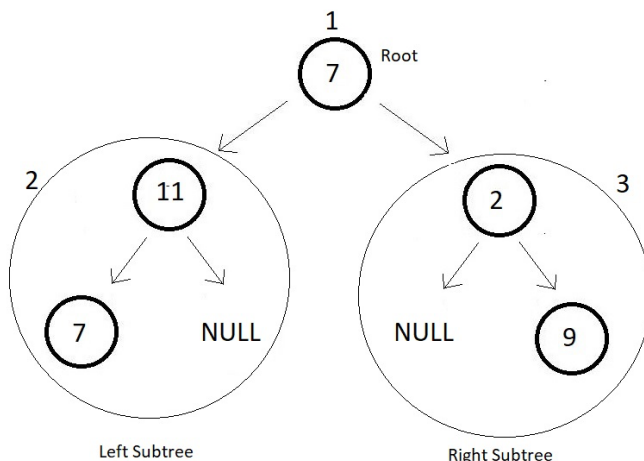
So basically, we have three different ways to traverse in a binary tree. They are InOrder, PostOrder, and PreOrder Traversals. Let's take a sample tree, and walk through it one by one, using each traversal technique for better understanding.



Let's start with the first one.

PreOrder Traversal in a Binary Tree:

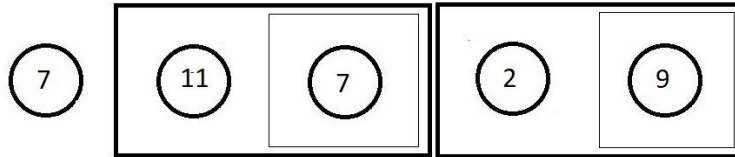
So in this traversal technique, the first node you start with is the root node. And thereafter you visit the left subtree and then the right subtree. Taking the above example, I'll mark your order of traversal as below. You first visit section 1, then 2, and then 3.



Now, this was to give you a general idea of the traverse in a binary tree using PreOrder Traversal. Each time you get a tree, you first visit its root node, and then move to its left subtree, and then to the right.

So, here you first visit the root node element 7 and then move to the left subtree. The left subtree in itself is a tree with root node 11. So, you visit that and move further to the left subtree of this subtree. There you see a single element 7, you visit that, and then move to the right subtree which is a NULL. So, you're finished with the left subtree of the main tree. Now, you move to the right subtree which has element 2 as its node. And then a NULL to its left and element 9 to its right.

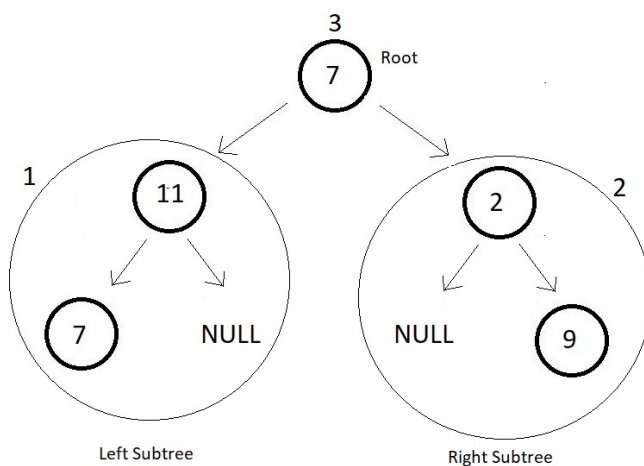
So basically, you recursively visit each subtree in the same order. And your final traversal order is:



Here, each block represents a different subtree.

PostOrder Traversal in a Binary Tree:

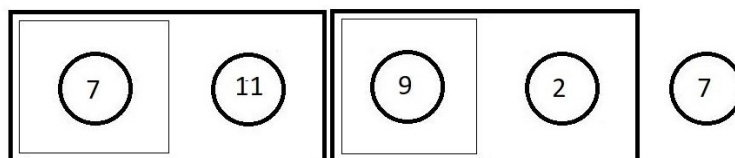
In this traversal technique, things are quite opposite to the PreOrder traversal. Here, you first visit the left subtree, and then the right subtree. So, the last node you'll visit is the root node. Taking the same above example, I'll mark your order of traversal as below. You first visit section 1, then 2, and then 3.



This was again a general idea of you traverse in a binary tree using PostOrder Traversal. Each time you get a tree, you first visit its left subtree, and then its right subtree, and then move to its root node.

I expect you to write the flow of the traversal in PostOrder yourself, and let me know if you could. We would anyway see them in detail.

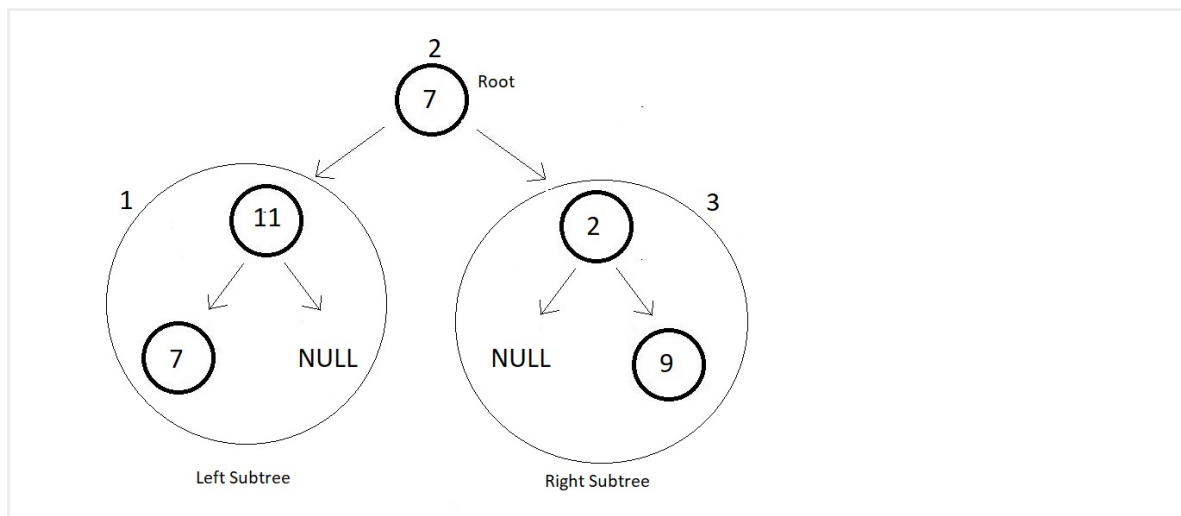
Hint: Your final traversal order should be.



Here, each block represents a different subtree.

InOrder Traversal in a Binary Tree:

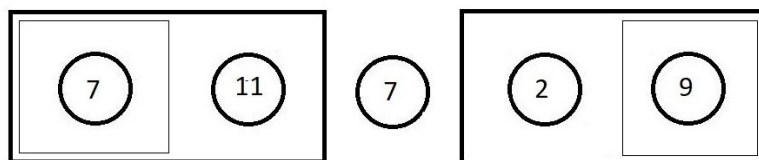
In this traversal technique, we simply start with the left subtree, that is you first visit the left subtree, and then go to the root node and then you'll visit the right subtree. Taking the same above example, I'll mark your order of traversal as below. You first visit section 1, then 2, and then 3.



This was a general idea of you traverse in a binary tree using InOrder Traversal. Each time you get a tree, you first visit its left subtree, and then its root node, and then finally its right subtree.

I expect you to write the flow of the traversal in InOrder yourself, and let me know if you could.

Hint: Your final traversal order should be:



Here, each block represents a different subtree.

So basically, the names PreOrder, PostOrder, and InOrder were given with respect to the position we keep our root node at while traversing in the Binary Tree. Since we first visit the root node in the first technique, that's why the name PreOrder. And we visit the root node at last in the PostOrder and in between in InOrder. Each of these techniques has one thing in common: the left subtree is traversed before the right subtree.

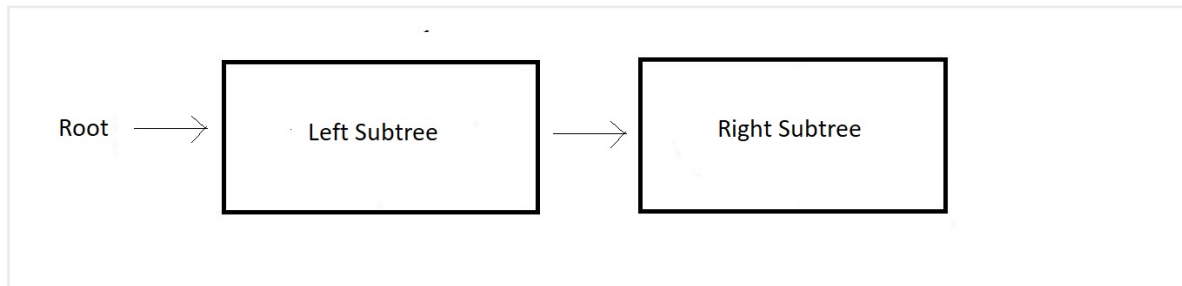
And these were the traversal techniques we had. We'll cover the programming part for each of these three in our coming lectures and in great detail. You shouldn't miss out on any of these. And do not forget the two techniques I left for you to explore.

Preorder Traversal in a Binary Tree (With C Code)

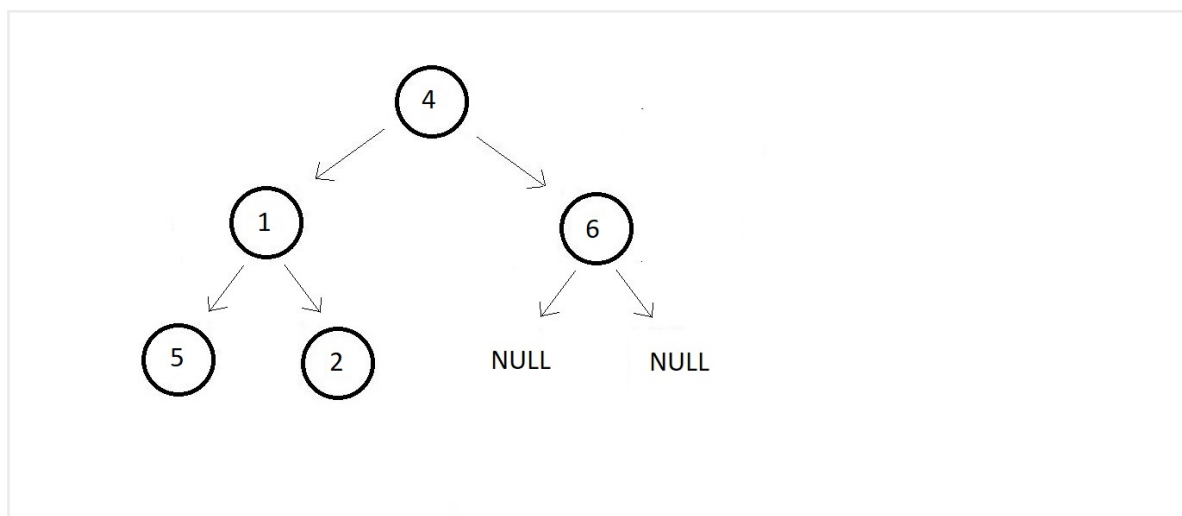
In the last lecture, we saw why we traverse through trees for different purposes. We primarily discussed, in brief, all the three types of traversal techniques. I discussed the flow of the PreOrder Traversal there itself and asked you to cover the last two yourselves. We will surely cover them in

detail starting today with the first one, PostOrder Traversal, and see its programming implementation in C.

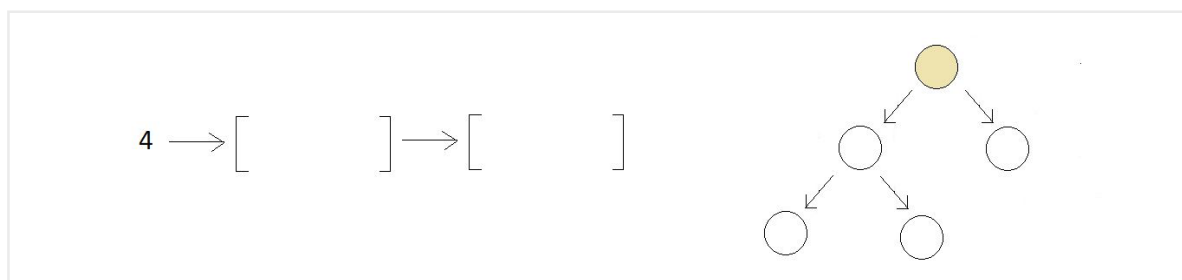
We basically have a basic idea of how PreOrder traversal works. Today we will see that in great detail. So, here you first start with the root node of the main tree and then get the hold of the left subtree. Now consider this left subtree as a new tree, and apply PreOrder on this. Recursively doing this with all the further subtrees, you will visit each node of this tree. Once you finish with the left subtree, you go to the right subtree and consider this as another tree and repeat the whole thing again.



Let's first take an example binary tree, and apply PreOrder Traversal on the same.

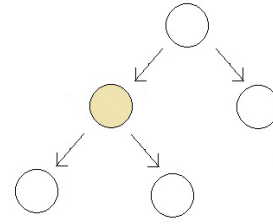


In the first step, you visit the root node and mark the presence of the left and the right subtree as separate individual trees.



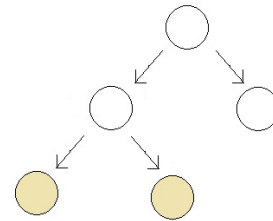
After you visit the root node, you move to the left subtree considering it as a different tree, and start with its root node.

4 $\rightarrow$ [1 $\rightarrow$ [] []] $\rightarrow$ []



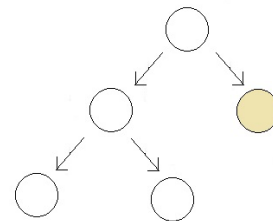
And then you proceed further with the left and right subtrees of this new tree we considered. And since both the left and right subtrees of this tree have just a single element in them, you finish visiting them, and return back to our original tree.

4 $\rightarrow$ [1 $\rightarrow$ [5] [2]] $\rightarrow$ []



And finally, we visit the right subtree, and since it contains no left or right subtree further, we finish our preorder traversal here itself.

4 $\rightarrow$ [1 $\rightarrow$ [5] [2]] $\rightarrow$ [6]



And our final order of preorder traversal is: 4 $\rightarrow$ 1 $\rightarrow$ 5 $\rightarrow$ 2 $\rightarrow$ 6.

Now we are ready to implement its programming having studied the flow in detail. I have attached the source code below. Follow it as we proceed.

Understanding the code snippet below:

1. First of all, we wouldn't start from scratch creating the struct Node and the createNode function and everything. So just copy the whole thing we did in our previous programming lecture and paste them here. This would save us a lot of time.
2. Create all the five nodes, using the createNode function, and link them using the arrow operator, and altering their left and right pointer elements. This creates our tree. The next thing would be to create the preOrder function.

```
// Constructing the root node - Using Function (Recommended)
struct node *p = createNode(4);
struct node *p1 = createNode(1);
```

```

struct node *p2 = createNode(6);
struct node *p3 = createNode(5);
struct node *p4 = createNode(2);
// Finally The tree looks like this:
//    4
//   /\
//  1  6
// /\
// 5  2

// Linking the root node with left and right children
p->left = p1;
p->right = p2;
p1->left = p3;
p1->right = p4;

```

Copy

Code Snippet 1: Creating the Binary tree

Creating the preOrder function:

3. Create a void function preOrder and pass the pointer to the root node of the tree you want to traverse as the only parameter. Inside the function, check if the pointer is not NULL, otherwise we wouldn't do anything. So, if it is not NULL, print the data element of the root struct node by using the arrow operator.

4. After you finish visiting the root node, simply call the same function recursively on the left and the right subtrees and you're done. Applying recursion does your job in its own subtle ways. It considers the left subtree as an individual tree and applies preorder on it, and the same goes for the right subtree.

```

void preOrder(struct node* root){
    if(root!=NULL){
        printf("%d ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

```

Copy

Code Snippet 2: Creating the preOrder function

Here is the whole source code:

```

#include<stdio.h>
#include<malloc.h>

struct node{
    int data;
    struct node* left;
    struct node* right;
}

```

```

};

struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the
heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node
}

void preOrder(struct node* root){
    if(root!=NULL){
        printf("%d ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

int main(){

    // Constructing the root node - Using Function (Recommended)
    struct node *p = createNode(4);
    struct node *p1 = createNode(1);
    struct node *p2 = createNode(6);
    struct node *p3 = createNode(5);
    struct node *p4 = createNode(2);
    // Finally The tree looks like this:
    //      4
    //     /\
    //    1 6
    //   /\
    //  5  2

    // Linking the root node with left and right children
    p->left = p1;
    p->right = p2;
    p1->left = p3;
    p1->right = p4;

    preOrder(p);
    return 0;
}

```

Copy

Code Snippet 3: Implementing the preOrder function

Now simply call the preOrder function passing the pointer to the root node as its parameter and see if it actually visits each node.

```
preOrder(p);
```

Copy

Code Snippet 4: Using the preOrder function

And the output we received was:

4 1 5 2 6

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

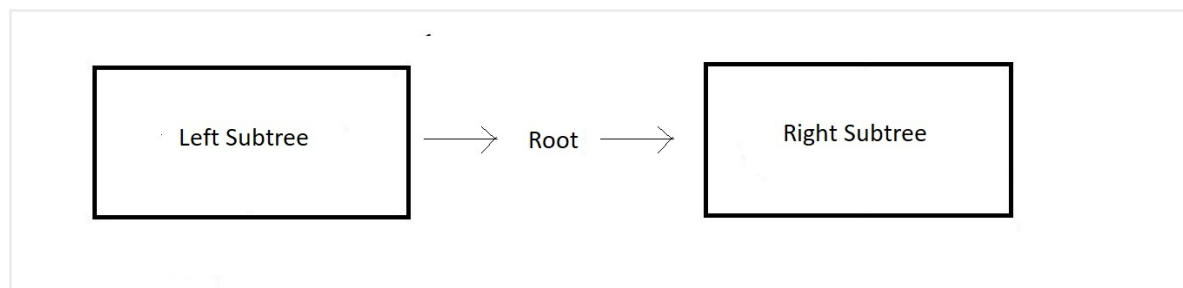
Figure 1: Output of the above code

So, it worked. It did visit each node, and it accurately followed the flow we discussed above. You can check the function yourself by creating your own binary tree. I don't think you need any more assistance there. And here we finish learning our first method. Let's make our way up to the next one.

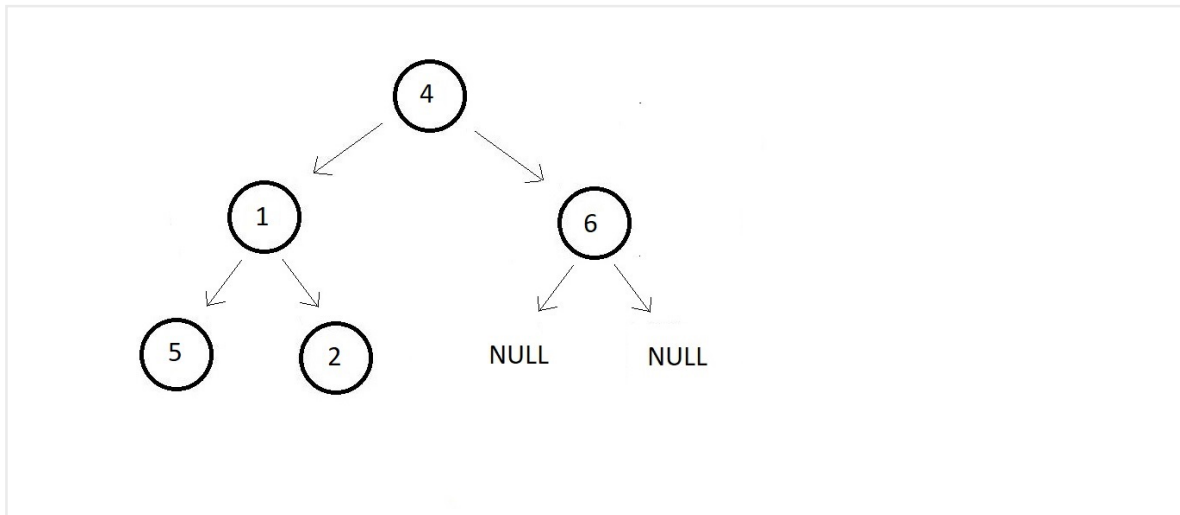
InOrder Traversal in a Binary Tree (With C Code)

In the last lecture, we discussed in detail our second traversal technique, the PostOrder Traversal. We saw its programming implementation in C. If you have not seen the last two lectures already, I suggest checking them out before continuing. Since today we are going to see our last traversal technique, it is a must to see them. Today, we'll see the InOrder Traversal in detail, and also cover its programming part.

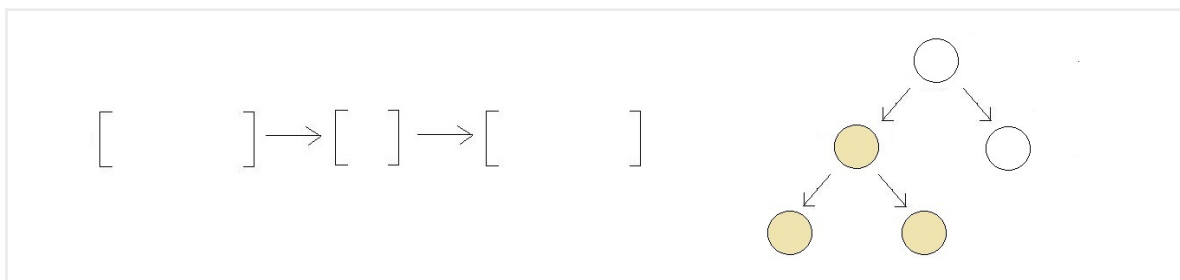
We did study the basic idea of how InOrder traversal works. I had even asked you all previously to explore the flow of this technique yourselves. Today we will see that in detail. So, here you first start with the left subtree and consider that as another tree in itself. You then apply InOrder on this subtree. You then come back to the root node. And then swiftly move to the right subtree and repeat the whole thing again considering this as another individual tree. And this would finish your job of visiting each node. Since the root is in between here, hence the name InOrder.



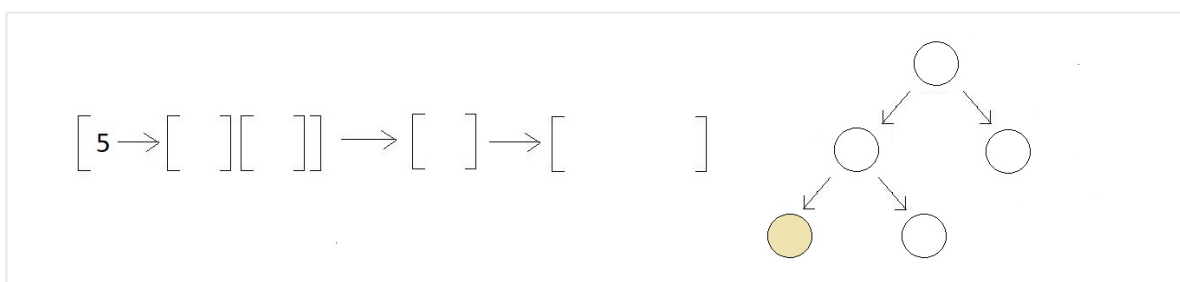
Our flow would remain unchanged, as we follow the same approach as the last two lectures. First, we will take an example binary tree, and apply InOrder Traversal on the same.



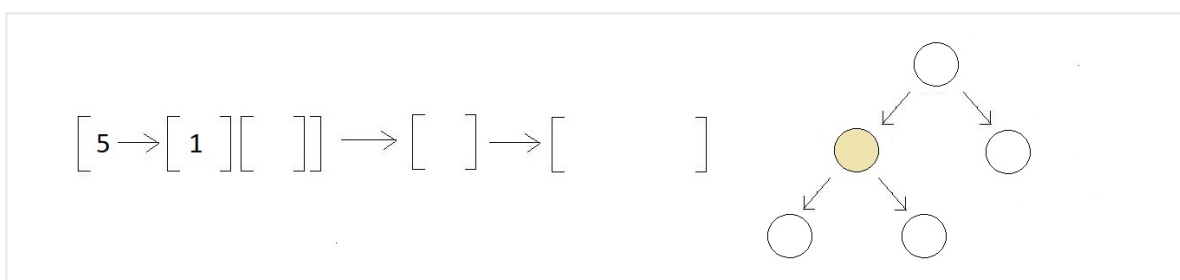
In the first step, you consider the left subtree as a completely different one and apply InOrder separately to it. But before you start with the left subtree, make sure to mark the presence of the root node and the right subtree as following:



Now, you go through the left subtree, and further visit the left subtree of this new tree first.

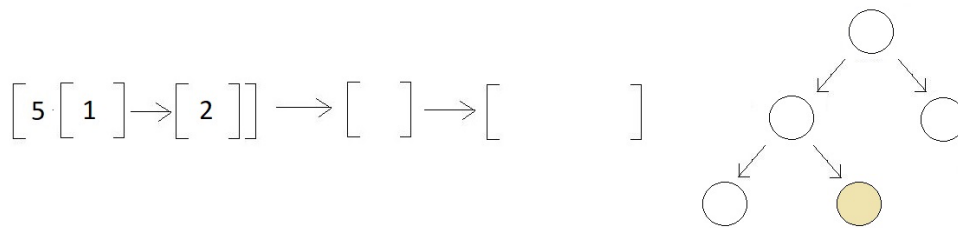


And then you proceed with the root of this new tree we considered.

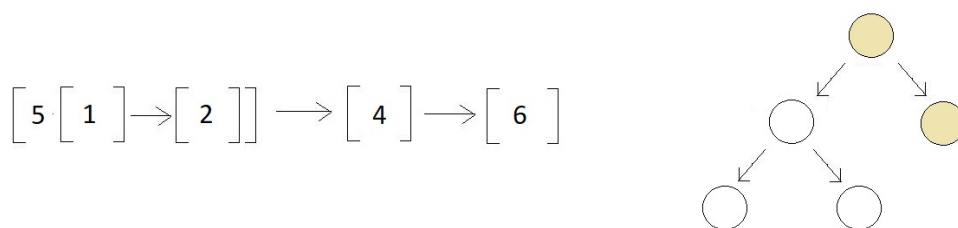


And after that, you visit the right subtree of this tree which has just a single

element.



With that being visited you move back to the original tree. And here, we visit the root node first and then the right subtree, and since it contains no left or right subtree further, we finish visiting our right subtree. There we mark the end of our InOrder traversal.



And our final order of inorder traversal is: $5 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 6$.

Having completed the flow of the InOrder Traversal, we are now ready to implement its programming. I have attached the source code below. Follow it as we proceed.

Understanding the code snippet below:

1. You know what to do first, right? We avoid doing repetitions, so we wouldn't start from scratch creating the whole struct Node and the createNode thing again rather we would just copy the whole thing we did in the PostOrder Traversal programming part and paste them here. This would get us even the codes for PreOrder and PostOrder.
2. You should now be able to create Binary Tree yourselves by now. So, just create the same binary tree we observed above using the createNode function. And if you are still not sure about creating a binary tree, follow the previous lectures. Let's now move to create the InOrder function.

Creating the inOrder function:

3. Create a void function inOrder and pass the pointer to the root node of the tree you want to traverse as the only parameter. Inside the function, check if the pointer is not NULL, as we are doing every time, since this is the base case for the recursion to stop. If it is NULL, we wouldn't do anything but if it isn't we would call the same function recursively on the left subtree using the left element of the root struct.
4. After we finish visiting the left subtree, we print the data element of

the root node indicating it as visited.

5. Having visited both the left subtree and the root, we now move to the right subtree and call it recursively. This completes our flow. And we are done visiting all the nodes.

```
void inOrder(struct node* root){
    if(root!=NULL){
        inOrder(root->left);
        printf("%d ", root->data);
        inOrder(root->right);
    }
}
```

Copy

Code Snippet 1: Creating the InOrder function

Here is the whole source code:

```
#include<stdio.h>
#include<malloc.h>

struct node{
    int data;
    struct node* left;
    struct node* right;
};

struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the
heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node
}

void preOrder(struct node* root){
    if(root!=NULL){
        printf("%d ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

void postOrder(struct node* root){
    if(root!=NULL){
        postOrder(root->left);
        postOrder(root->right);
        printf("%d ", root->data);
    }
}
```

```

    }
}

void inOrder(struct node* root){
    if(root!=NULL){
        inOrder(root->left);
        printf("%d ", root->data);
        inOrder(root->right);
    }
}

int main(){

    // Constructing the root node - Using Function (Recommended)
    struct node *p = createNode(4);
    struct node *p1 = createNode(1);
    struct node *p2 = createNode(6);
    struct node *p3 = createNode(5);
    struct node *p4 = createNode(2);
    // Finally The tree looks like this:
    //   4
    //  /\
    // 1  6
    // /\
    // 5  2

    // Linking the root node with left and right children
    p->left = p1;
    p->right = p2;
    p1->left = p3;
    p1->right = p4;

    // preOrder(p);
    // printf("\n");
    // postOrder(p);
    // printf("\n");
    inOrder(p);
    return 0;
}

```

Copy

Code Snippet 2: Implementing the inOrder function

Now simply call all the traversal functions, preOrder, postOrder, and inOrder passing the pointer to the root node as their parameter and see how they work.

```
preOrder(p);
printf("\n");
postOrder(p);
printf("\n");
inOrder(p);
```

Copy

Code Snippet 3: Using the preOrder, the postOrder, and the inOrder functions

And the output we received was:

4 1 5 2 6

5 2 1 6 4

5 1 2 4 6

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 1: Output of the above code

And it worked accurately to our knowledge. You should feel and observe the differences in all the traversal techniques. Things matched with our manual traversal above. You must verify the functions yourself by creating your own binary tree and by applying all the techniques. And here we finish all our traversal methods.

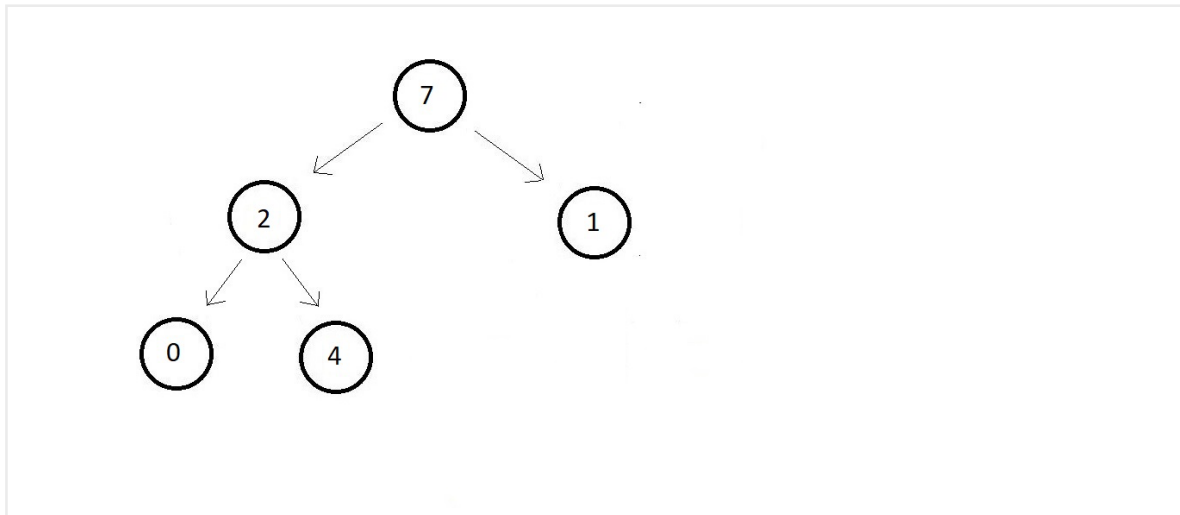
The binary tree we took as an example could be traversed manually very easily. I wonder, however, whether you can handle a large binary tree, or if you should use the same approach in an assessment with a time constraint. Obviously, no. There are ways faster than this. And I'll make sure you learn that in the next lecture.

***Best* Trick To Find PreOrder, InOrder & PostOrder Traversal**

In the previous tutorial, we covered our last traversal technique, the InOrder Traversal. We dealt with its programming in C. Earlier to that, we saw the other two traversal techniques. In the end, I did tell you that we will pursue a faster method to traverse through a Binary Tree. So today we'll see a faster, in fact, the best trick to find the PreOrder, the InOrder, and the PostOrder Traversal in binary trees.

This method shall help you in case you are in a hurry, or you have an assessment the next day. If it's just about marks, you can count entirely on the lecture and get good grades, I promise you. Throughout this lecture, it is assumed you have jumped right into this lecture, and haven't gone to the previous lectures of the series.

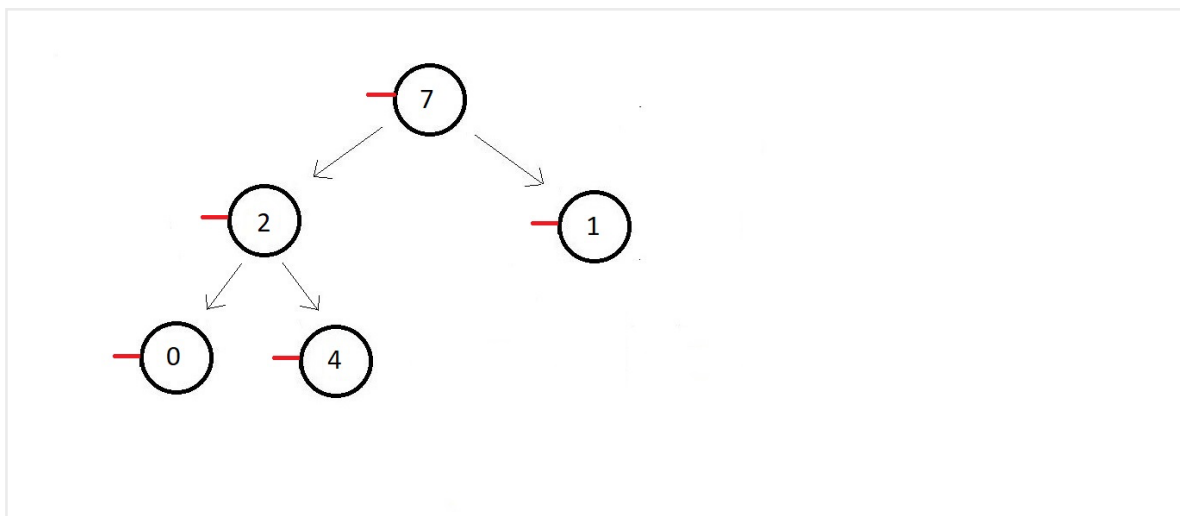
I would like to provide an example of a Binary Tree. We will then traverse through it using different techniques.



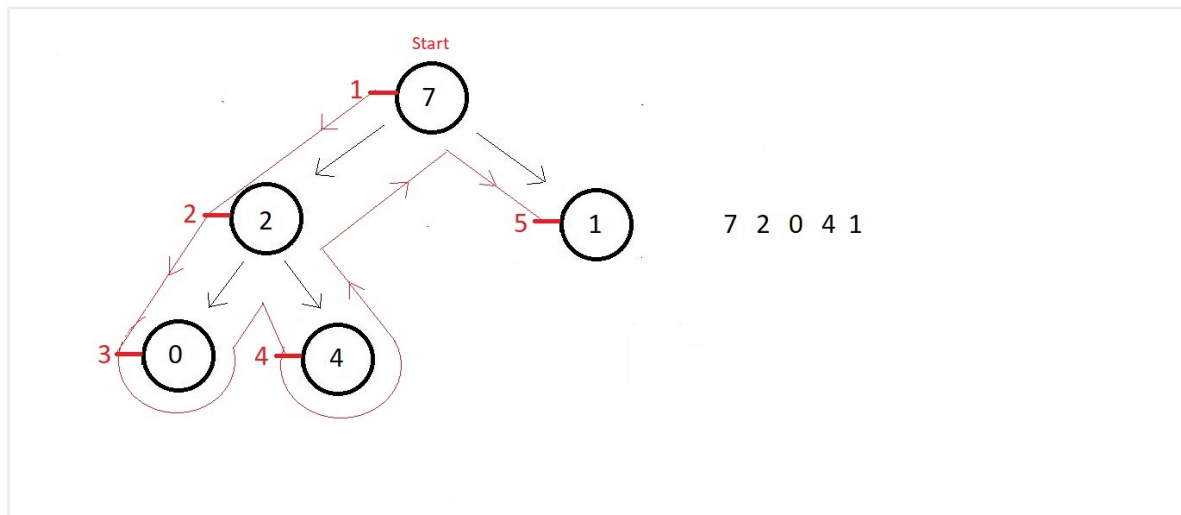
First, we'll see traversal using the PreOrder approach.

PreOrder Traversal:

In this method, we start with the node, and then move to the left subtree and then to the right subtree. But the trick says, extend an edge to the left of all the nodes. Follow the figure below to understand what is being said.



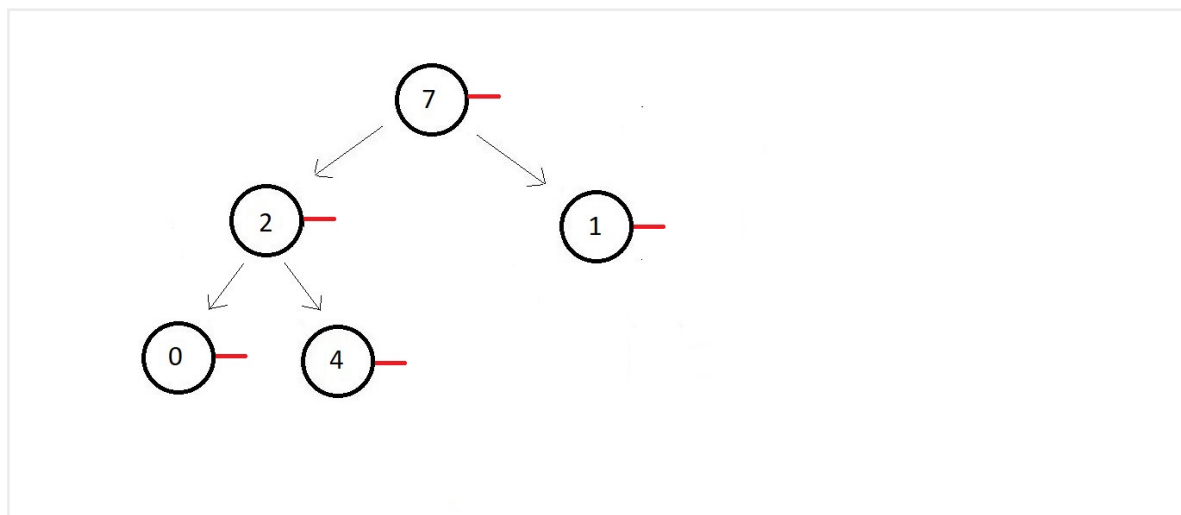
And now, start driving from the root to the left of the root node, and whenever you intersect a red edge while driving, print its value. Refer to the illustration below. Arrows have been made to direct you to the path we have followed.



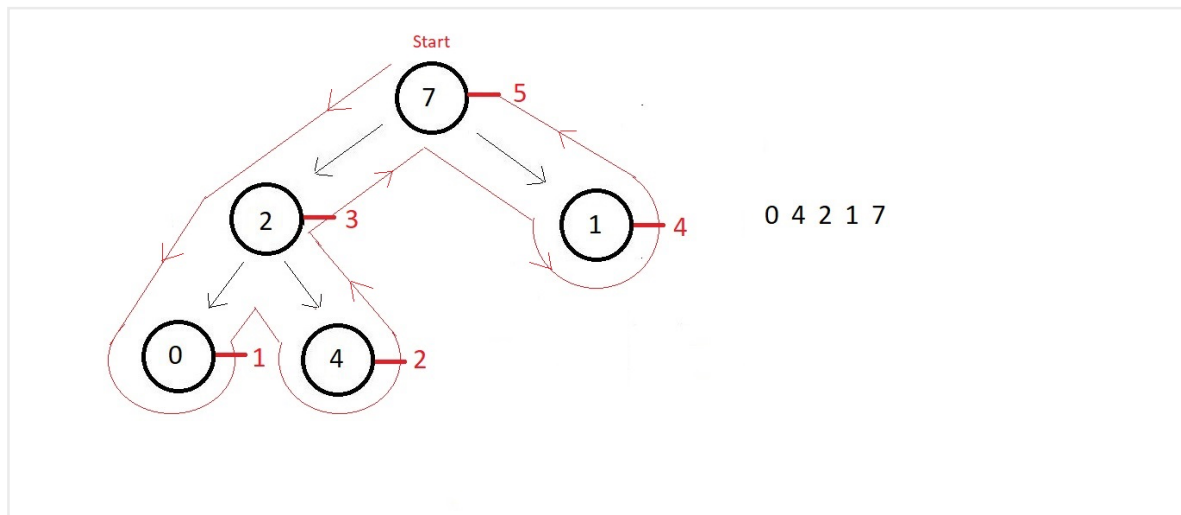
And this is how you were quickly able to write the order of nodes you visited first in the PreOrder Traversal. It was $7 \rightarrow 2 \rightarrow 0 \rightarrow 4 \rightarrow 1$. Let's see how easy PostOrder Traversal is.

PostOrder Traversal:

In this method, we start with the left subtree first, and then move to the right subtree, and then finally to the root node. But the trick illustrates, extending an edge to the right of all the nodes. Earlier it was left, now it is right. Follow the figure below to understand.



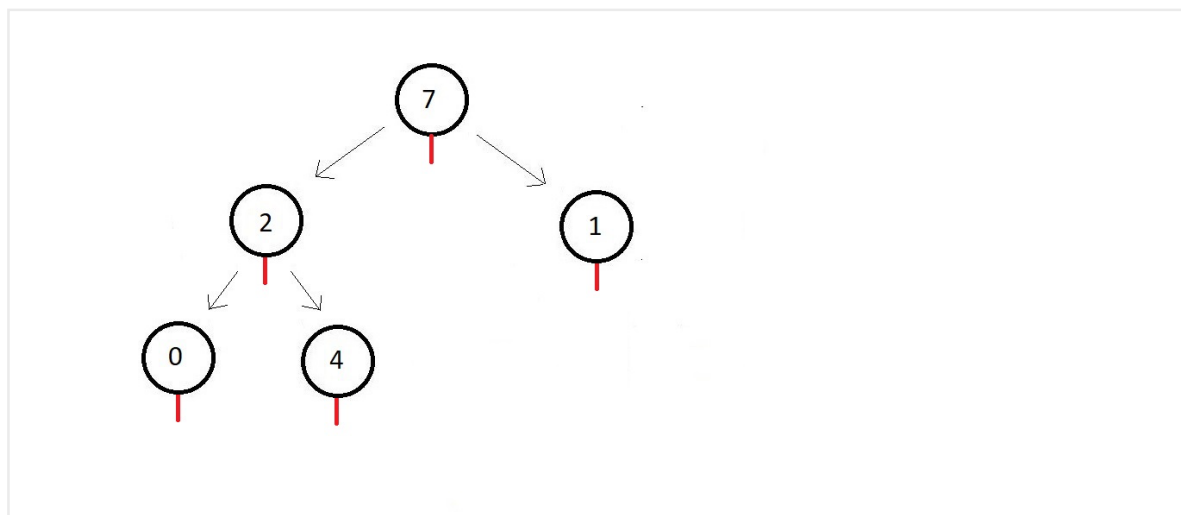
And now, drive the same way we did earlier. Start driving from the root to the left of the root node and whenever you intersect a red edge while driving, print its value. Refer to the illustration below. Arrows have been drawn to direct you to the path we have taken.



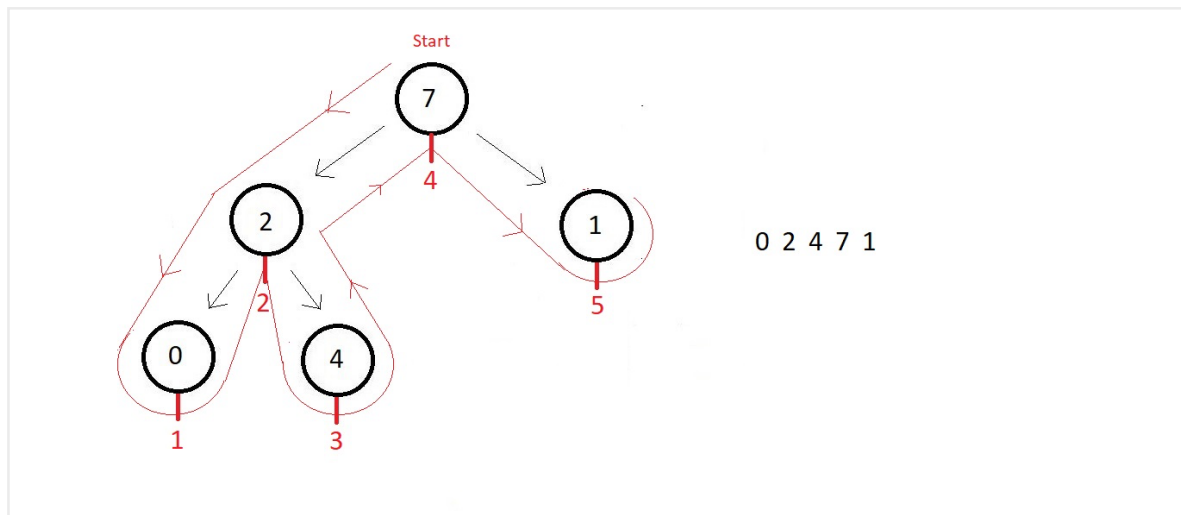
And this is how you were quickly able to write the order of nodes you visited first in the PostOrder Traversal. It was $0 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 7$. Let's see lastly how easy InOrder Traversal is.

InOrder Traversal:

In this method, we start with the left subtree first, and then move to the root node and then finally to the right subtree. But the trick says, extend an edge to the bottom of all the nodes. We went to the left, to the right, and now to the bottom. Follow the figure below to understand.



Then, drive as we have done all along. Start driving from the root to the left of the root node and whenever you intersect a red edge while driving, print its value. Refer to the illustration below. Arrows have been drawn to direct you to the path we have taken.



And this is how you were quickly able to write the order of nodes you visited first in the InOrder Traversal. It was $0 \rightarrow 2 \rightarrow 4 \rightarrow 7 \rightarrow 1$.

You can do one thing now. Create the binary tree in the above example and use the programs we created in earlier lectures for each of these traversals to verify if this trick actually works. And I can assure you the authenticity of this trick for all the larger binary trees you will get in your assessments. Use this blindly. But make sure you cover up the concepts sooner or later. Because that is what helps you in the end.

Here we mark the end of our traversal techniques. Next on the list is a much-anticipated topic, Binary Search Trees (BST). I hope you'll join me there.

Binary Search Trees: Introduction & Properties

In the last lecture, we finished learning about all the traversal methods in binary trees. To make it easier for you, we even found a way to keep track of the order of nodes you visit in every traversal method. Today, we'll start learning about what a binary search tree is and what features they possess. Earlier we concluded a few points about binary trees. Binary trees are normal trees with each parent having either two or less than two children, that is the degree of the tree should be two or less than two.

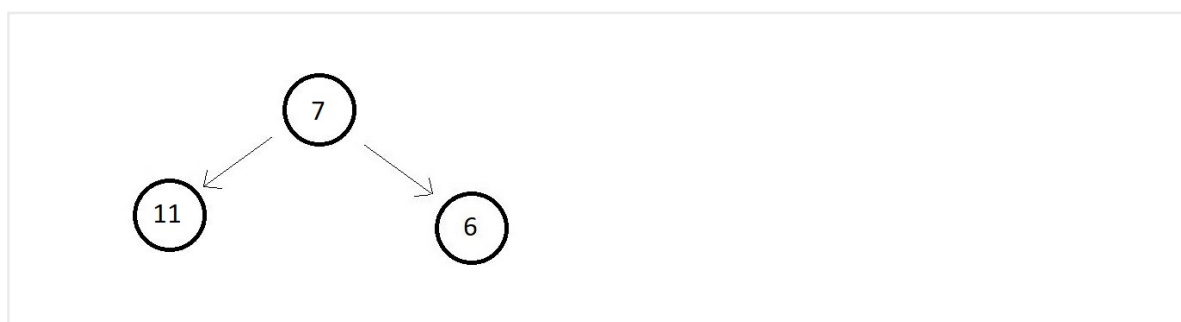


Figure 1

Tell me if the above tree is a binary tree or not. Of Course, it is. Since all the nodes have degrees either two or zero. But is this binary tree a binary search tree? Wait. A distinction between binary trees and binary search trees is what I am seeking to establish. Following are the properties of a

binary search tree:

Binary Search Trees:

1. It is a type of binary tree.
2. All nodes of the left subtree are lesser than the node itself.
3. All nodes of the right subtree are greater than the node itself.
4. Left and Right subtrees are also binary trees.
5. There are no duplicate nodes.

Having discussed all the properties, you must now tell me if the above binary tree was a binary search tree or not. The answer should be no. Since the left subtree of the root node has a single element that is greater than the root node violating the 2nd property, it is not a binary search tree. Let me take one more example.



Figure 2

Tell me if this is a binary search tree. Still no. Because the left subtree is good but the right subtree of the root node is lesser than the root node itself violating the 3rd property. Actually, I have always found you all to be smart enough to answer these petty questions. Let's consider a big one and see if it's a binary search tree or not.

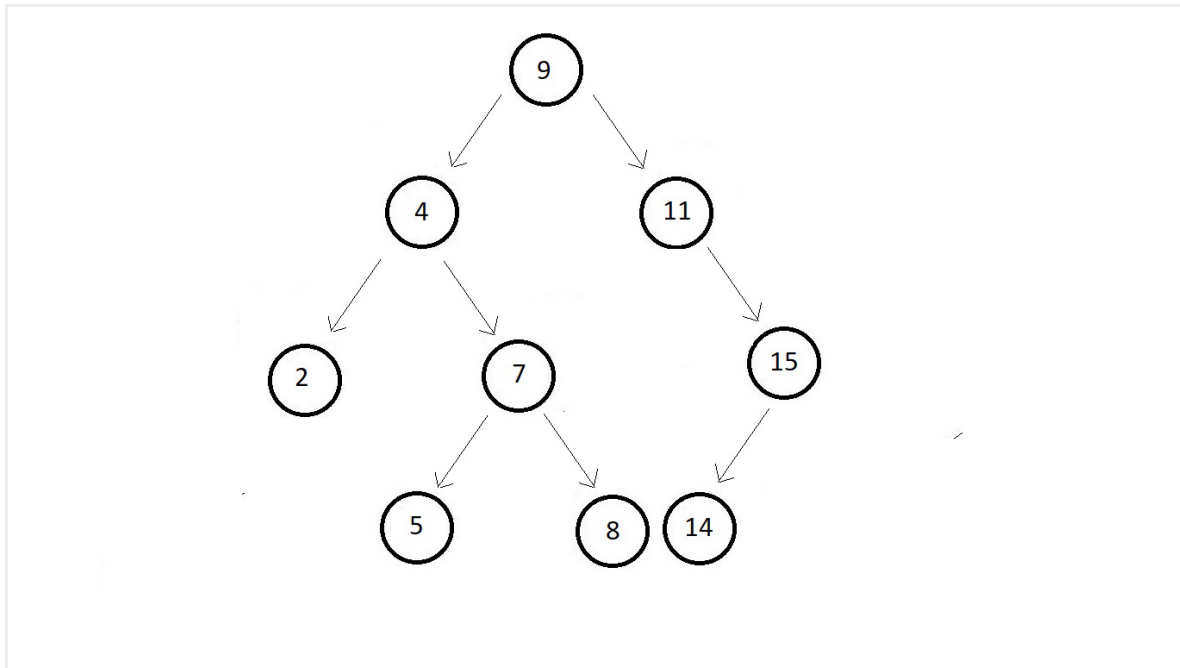


Figure 3

Take your time and analyze the different properties of a binary search tree and tell if this is a binary search tree or not.

Solution: YES.

Why?

The very first thing to observe here is the properties of a Binary Search Tree. You would check if all the properties are satisfied for each of the nodes of the tree. So, you first start with the root node which is element 9 and see if all the nodes on the left subtree {4, 2, 5, 7} are smaller than 9 and all the nodes of the right subtree {11, 15, 14} are greater than 9. And since they are, we'll proceed with the next node. Doing this for all the nodes, we'll conclude that this is a Binary Search Tree. Had there been even one violation for any of the nodes, we would have said, no.

Lastly, there is one more amazing property that I've been keeping all along to amaze you. It says **the InOrder traversal of a binary search tree gives an ascending sorted array**. So, this is one of the easiest ways to check if a tree is a binary search tree. Let's write the InOrder Traversal of the tree in figure 3.

We'll use the same technique we learned in the last lecture to quickly find the InOrder traversal. We'll extend an edge to the bottom of each node.

We still haven't seen how to make a binary search tree which shall be our next job. We'll also make programs for creating a binary search tree, and for checking if a tree is a binary search tree or not. This was all about the basics of binary search trees. We introduced this to you and discussed its most important properties.

Checking if a binary tree is a binary search tree or not!

In the last lecture, we got to know about one more variety of binary tree, called binary search trees. We discussed all the properties of a binary search tree. We drew a fine line between binary trees and binary search trees. Today, we'll delve into their differences and distinguish a binary search tree from a normal binary tree. And check if a binary tree is a binary search tree or not.

But before that let's briefly revise the properties of a binary search tree once.

Properties of a binary search tree:

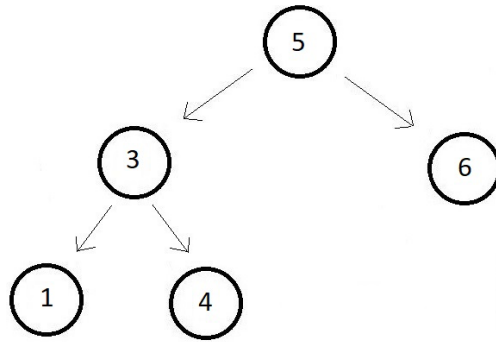
1. All nodes of the left subtree are lesser than the node itself.
2. All nodes of the right subtree are greater than the node itself.
3. Left and Right subtrees are also binary trees.
4. There are no duplicate nodes.
5. The InOrder traversal of a binary search tree gives an ascending sorted array.

And the last one is of utmost importance to us. And that is what we use the most to check if a binary tree is a binary search tree or not. So, basically, we'll check if a binary tree is a binary search tree or not by making an InOrder Traversal in the tree and analyzing its order. I have attached the source code below. Follow it as we proceed.

Understanding the code snippet below:

1. Since we'll create a binary search tree at first which is nothing but a binary tree only and we'll use the InOrder traversal function as well, we'll simply copy everything we did in our last programming tutorial where we learned to create the function inOrder. And we did this because we avoided doing repetitions in the course and starting from scratch, creating the whole struct Node and the createNode thing again and all the functions would have made the lecture redundant. And this has also saved us a lot of time.
2. I just hope that you've followed all the way from the start of the Binary Search lecture and not just jumped right here. Make sure you check them all before proceeding.

You should now be able to create Binary Tree yourselves. Create the binary search tree I've illustrated below using the createNode function. And if you are still not sure about creating a binary search tree, follow the previous lectures.



3. Let's now just create a function to check if the InOrder traversal of the binary tree is in ascending order or not.

Creating the function isBST:

4. Create an integer function *isBST* and pass the pointer to the root node of the tree you want to check as the only parameter. Inside the function, check if the pointer is not NULL, as we have been checking the whole time, and this is also considered as the base case for the recursion to stop. If it is NULL, we would simply return 1 since an empty tree is always a binary search tree. Else, this is a complex yet understandable part. You should follow what I am saying.
5. Create a static struct Node pointer *prev* initialised with NULL. This maintains the pointer to the parent node. And since the root node doesn't have any parent, we have initialized it with NULL and made it static.
6. Now, see if the left subtree is a Binary Search Tree or not, by calling it recursively. If it is not a BST, return 0 here itself. Else, see if the *prev* is not NULL otherwise this is the root node of the whole tree and we won't check this condition. If the *prev* node is not NULL and the current node, which is the root node of the current subtree, is smaller than or equal to the *prev* node, then we would return 0. Since this violates the increasing orderliness.
7. If it still passes all the if conditions we have structured above, we will store the current node in the *prev* and move it recursively to the right subtree. And this is nothing but a modified version of the InOrder Traversal.

```

int isBST(struct node* root){
    static struct node *prev = NULL;
    if(root!=NULL){
        if(!isBST(root->left)){
            return 0;
        }
        if(prev!=NULL && root->data <= prev->data){

```

```

        return 0;
    }
    prev = root;
    return isBST(root->right);
}
else{
    return 1;
}
}
Copy

```

Code Snippet 1: Creating the isBST function

I suggest ignoring the recursion and do not trace how the function isBST works if the function isBST is at all confusing. Just note that we have done nothing but the InOrder traversal of the tree, and we have checked if the previous value is smaller than the current value, that's it.

Note: Static variables are used when we don't want our value for that variable to change every time that function is called.

Here is the whole source code:

```

#include<stdio.h>
#include<malloc.h>

struct node{
    int data;
    struct node* left;
    struct node* right;
};

struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node
}

void preOrder(struct node* root){
    if(root!=NULL){
        printf("%d ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

```

```

void postOrder(struct node* root){
    if(root!=NULL){
        postOrder(root->left);
        postOrder(root->right);
        printf("%d ", root->data);
    }
}

```

```

void inOrder(struct node* root){
    if(root!=NULL){
        inOrder(root->left);
        printf("%d ", root->data);
        inOrder(root->right);
    }
}

```

```

int isBST(struct node* root){
    static struct node *prev = NULL;
    if(root!=NULL){
        if(!isBST(root->left)){
            return 0;
        }
        if(prev!=NULL && root->data <= prev->data){
            return 0;
        }
        prev = root;
        return isBST(root->right);
    }
    else{
        return 1;
    }
}

```

```

int main(){

```

// Constructing the root node - Using Function (Recommended)

```

struct node *p = createNode(5);
struct node *p1 = createNode(3);
struct node *p2 = createNode(6);
struct node *p3 = createNode(1);
struct node *p4 = createNode(4);

```

// Finally The tree looks like this:

```

//   5
//  /\
// 3 6
// /\

```

```
// 1 4

// Linking the root node with left and right children
p->left = p1;
p->right = p2;
p1->left = p3;
p1->right = p4;

// preOrder(p);
// printf("\n");
// postOrder(p);
// printf("\n");
inOrder(p);
printf("\n");
// printf("%d", isBST(p));
if(isBST(p)){
    printf("This is a bst" );
}
else{
    printf("This is not a bst");
}
return 0;
}
```

Copy

Code Snippet 2: Implementing the isBST function

Let's now check if our function actually works, and it reverts whether our binary tree is a BST or not. We would also like to print the InOrder traversal of the tree.

```
inOrder(p);
printf("\n");
if(isBST(p)){
    printf("This is a bst" );
}
else{
    printf("This is not a bst");
}
}
```

Copy

Code Snippet 3: Using the isBST function

And the output we received was:

```
1 3 4 5 6
```

```
This is a bst
```

```
PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>
```

Copy

Figure 1: Output of the above code

And yes, it says, our function is a Binary Search Tree and that was what we expected. The InOrder traversal is also in ascending order. So, things are

pretty accurate and consistent.

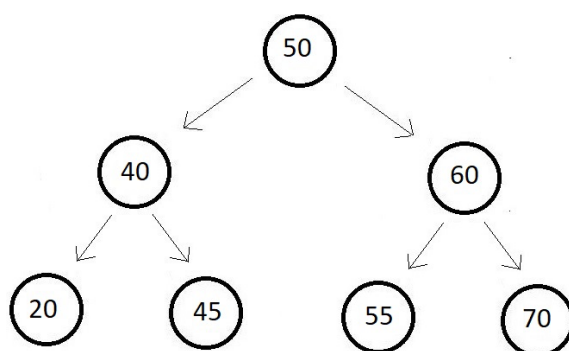
So, this was all we had to do to automate the checking of a binary tree for a binary search tree. Go through the functions again, and keep in mind the things I've told you not to worry about. Focus on the algorithm, and the modifications we made in the *InOrder* function. And that would be all. It is now time to move on to our next topic which entails our first operation on Binary Search Trees.

Searching in a Binary Search Trees (Search Operation)

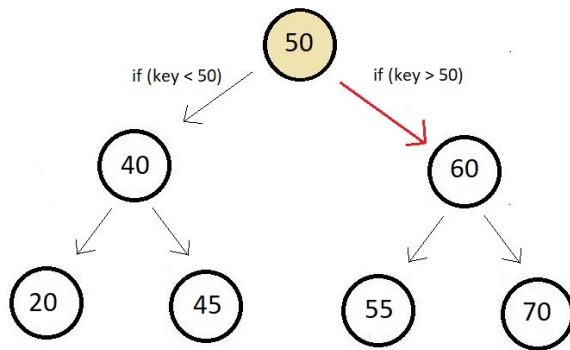
In the last tutorial, we saw the implementation of the *isBST* function in C. We thoroughly discussed the programming. We saw the modifications we had to make in the *inOrder* function to check if the InOrder traversal is in ascending order or not, and in fact, to check if a binary tree is a binary search tree or not. And today, we are going to look after our first binary search tree operation, the **search operation**. We'll see if any given key lies in the binary search tree or not.

Now you might be wondering what exactly is the significance of learning about binary search trees since binary trees were equally good. But let me tell you that one of the major applications of using a binary search tree, is to be able to search some key in the tree in **logn** time complexity in the best case where *n* is the number of nodes. Each time you compare a node with your key, you divide the search space to its half. But let's not proceed without an example.

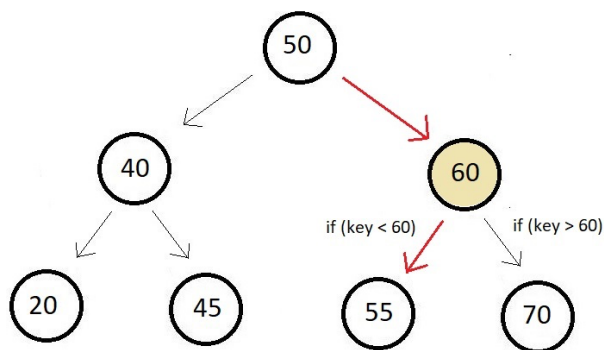
For our purpose of understanding, we will first examine a sample binary search tree. Suppose we have a Binary Search Tree illustrated below.



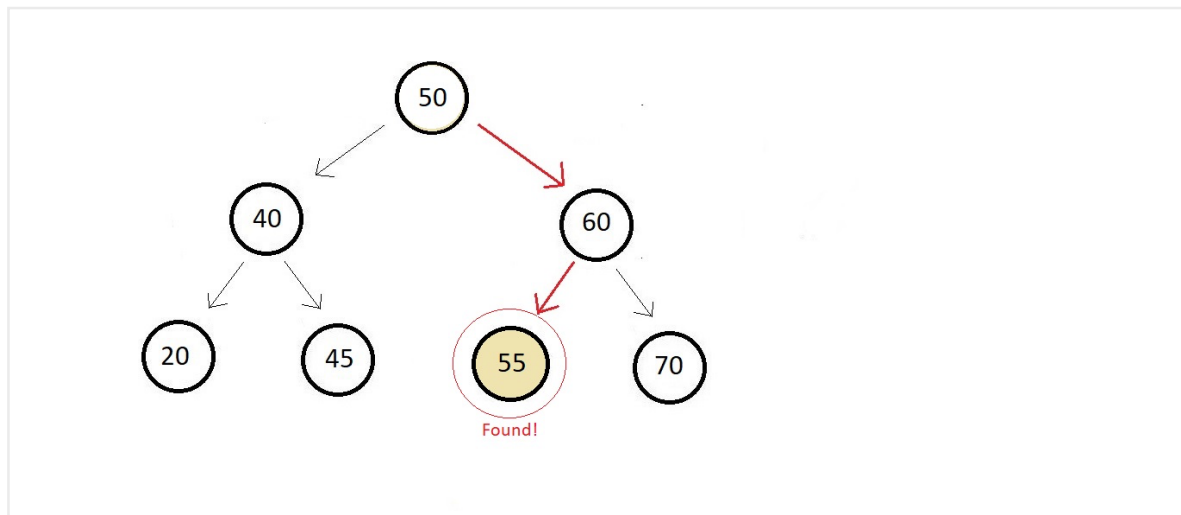
And let's say the key we want to search in this binary search tree is 55. Let's start our search. So, we'll first compare our key with the root node itself, which is 50.



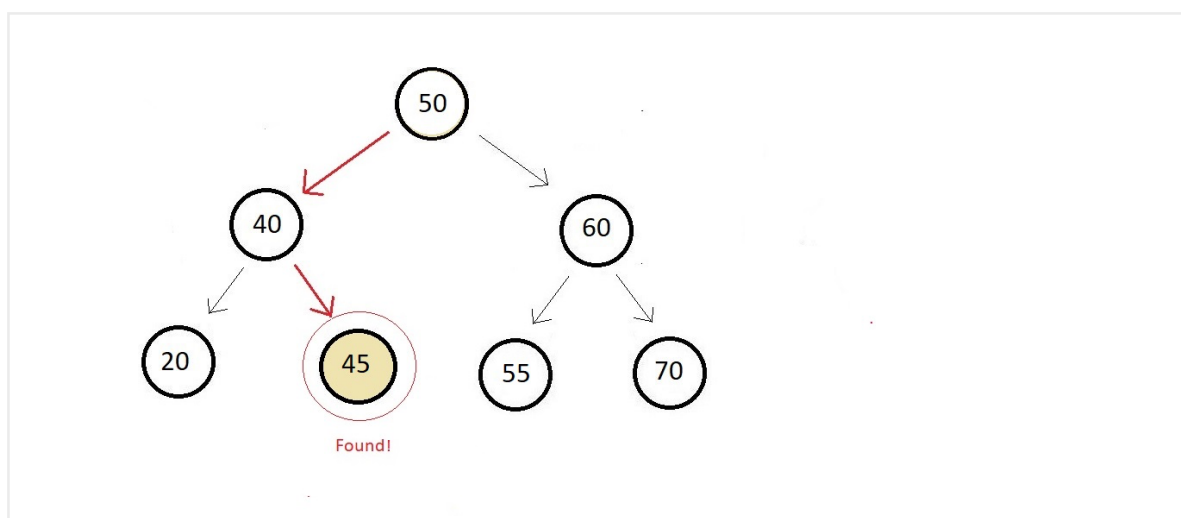
But since 50 is less than 55, which side should we proceed with? Left, or Right? Of course, right. Since all the elements in the right subtree of a node are greater than that node, we'll move to the right. The first element we check our key with is 60.



Now, since our key is smaller than 60, we'll move to the left of the current node. The Left subtree of 60 contains only one element and since that is equal to our key, we revert the positive result that yes, **the key was found**. Had this leaf node been not equal to the key, and since there are no subtrees further, we would have stopped here itself with negative results, saying the key was not found.



Let's see for one more key which is 45. Now, I'll directly illustrate the path we followed and whether it was found or not.



And yes, key 45 was found too. The path we followed is colored red. We went to 50 and found it smaller so moved to its left. Then we found 40, and since our key was greater than that, we moved to its right, and the leaf node we found was equal to 45 only.

Let's analyze the time complexity of the searching algorithm we have discussed above.

Time Complexity of the Search Operation in a Binary Search Tree:

If you remember, we had studied an algorithm called the Binary Search. We could use that algorithm only if the array we were searching for some key in was sorted. And that algorithm had the time complexity of **$O(\log n)$** where n was the length of the array. Because we were always dividing our search space into half on the basis of whether our key was smaller or greater than the *mid*. And as you might have guessed, searching in a binary search tree is very much similar to that.

Searching in a binary search tree holds **$O(\log n)$** time complexity in the best case where n is the number of nodes making it incredibly easier to search

an element in a binary search tree, and even operations like inserting get relatively easier.

Let's calculate exactly what happens. If you could see the above examples, the algorithm took the number of comparisons equal to the height of the binary search tree, because at each comparison we stepped down the depth by 1. So, the time complexity $T \propto h$, that is, our time complexity is proportional to the height of the tree. Therefore, the time complexity becomes **$O(h)$** .

Now, if you remember, the height of a tree ranges from $\log n$ to n , that is

$$(\log n) \leq h \leq n$$

So, the best-case time complexity is $O(\log n)$ and the worst-case time complexity is $O(n)$.

Our next aim would be to automate the searching process. The way we compared the nodes and decided to move to the left or to the right needs to be programmed. So today I'll just brief you all about the way we program this search operation and will implement them in C in the next lecture. We will also write pseudocode for better understanding.

Pseudocode for searching in a Binary Search Tree:

1. There will be a struct node pointer function *search* which will take the pointer to the root node and the key you want to search in the tree. And before you do anything, just check if the root node is not NULL. If it is, return NULL here itself. Otherwise, proceed further.
2. Now, check if the node you are at is the one you were looking for. If it is, return that node. And that would be it. But if that is not the one, just see if that key is greater than or less than that node. If it is less, then return recursively to the left subtree, otherwise to the right subtree. And that is all.

```
Node * search(node* root, key){
    if(root==NULL){
        return NULL;
    }
    if(key==root->data){
        return root;
    }
    else if(key<root->data){
        return search(root->left, key);
    }
    else{
        return search(root->right, key);
    }
}
```

Figure 1: Pseudocode for the search function

We'll see the programming segment in the next lecture in detail. We will make use of our IDEs. And program our own and run them to actually search for a key in a binary search tree.

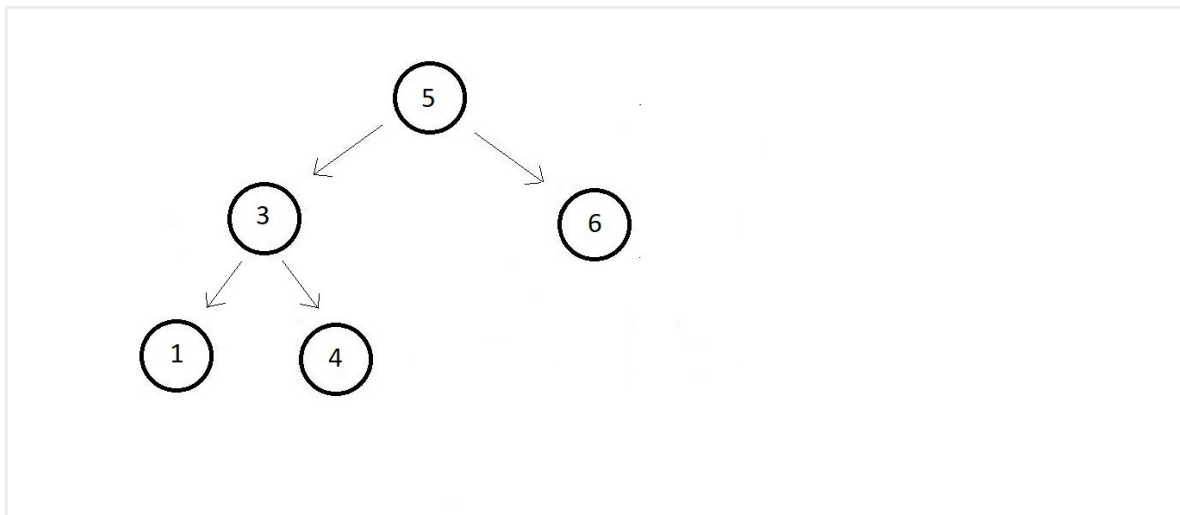
C Code For Searching in a BST

In the last lecture, we saw one important application of binary search trees. It is its search operation that allows us to search any key in the binary search tree in $\log n$ best case time complexity. We saw how searching is done by reducing the search space after every comparison, and how searching time is proportional to the height of the tree. Today, we'll see the programming of this search function in C and automate the searching process.

We did see the pseudocode of the search function in the last class. Follow it while we understand the procedure. I have attached the source code below. Follow it as we proceed.

Understanding the code snippet below:

1. First of all, we'll anyway have to create a binary search tree to be able to use it to search some keys. And since we created everything the last time, we'll just copy everything we have done till the last programming lecture. It covered everything from creating a node to building a tree. It also features all the traversal methods and the isBST function. We copied everything so as to keep everything in one place, and to avoid repeating things in the course, and this has also saved us a lot of time.
2. Let's create a binary search tree I've illustrated below using the createNode function. Creating a binary search tree should be an issue anymore. And if you are still not sure about creating a binary search tree, follow the previous lectures.



3. We'll do each of the other operations we usually do. But today our focus is on the search operation. Let's now just create a function to search a key in the binary search tree we created.

Creating the function search:

4. Create a struct Node pointer function *search* and pass into it the pointer to the root and the key you want to search as two of its

parameters.

5. First of all, check if the root is NULL. If the root is NULL, we haven't found our key, and we'll simply return NULL.
6. Now, check if the node we are currently at has the data element equal to the key we were searching for. If it is the case, we have found the key, and we'll simply return the pointer to the current root.
7. But if we still haven't found the key, it is probably in the left subtree of the root, or in the right subtree of the root. To judge which side to proceed with, we'll check if the current root is less than or greater than the data element of the root. If it is less than the root, the key probability lies on the left subtree, and hence we would reduce our search space and recursively start searching in the left subtree. Otherwise, we would start searching in the right subtree. Anyways, we are reducing our search space after each comparison.

```
struct node * search(struct node* root, int key){
    if(root==NULL){
        return NULL;
    }
    if(key==root->data){
        return root;
    }
    else if(key<root->data){
        return search(root->left, key);
    }
    else{
        return search(root->right, key);
    }
}
```

Copy

Code Snippet 1: Creating the search function

Recursion in the above function is pretty simple this time, unlike the isBST function we did early. Here, we simply decide every time which subtree to pursue next. And recursively go with that.

Here is the whole source code:

```
#include<stdio.h>
#include<malloc.h>

struct node{
    int data;
    struct node* left;
    struct node* right;
};
```

```

struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the
heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node
}

```

```

void preOrder(struct node* root){
    if(root!=NULL){
        printf("%d ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

```

```

void postOrder(struct node* root){
    if(root!=NULL){
        postOrder(root->left);
        postOrder(root->right);
        printf("%d ", root->data);
    }
}

```

```

void inOrder(struct node* root){
    if(root!=NULL){
        inOrder(root->left);
        printf("%d ", root->data);
        inOrder(root->right);
    }
}

```

```

int isBST(struct node* root){
    static struct node *prev = NULL;
    if(root!=NULL){
        if(!isBST(root->left)){
            return 0;
        }
        if(prev!=NULL && root->data <= prev->data){
            return 0;
        }
        prev = root;
        return isBST(root->right);
    }
}

```

```

    else{
        return 1;
    }
}

struct node * search(struct node* root, int key){
    if(root==NULL){
        return NULL;
    }
    if(key==root->data){
        return root;
    }
    else if(key<root->data){
        return search(root->left, key);
    }
    else{
        return search(root->right, key);
    }
}

```

```

int main(){

```

```

    // Constructing the root node - Using Function (Recommended)

```

```

    struct node *p = createNode(5);
    struct node *p1 = createNode(3);
    struct node *p2 = createNode(6);
    struct node *p3 = createNode(1);
    struct node *p4 = createNode(4);
    // Finally The tree looks like this:
    //   5
    //  /\
    // 3 6
    // /\
    // 1 4

```

```

    // Linking the root node with left and right children

```

```

    p->left = p1;
    p->right = p2;
    p1->left = p3;
    p1->right = p4;

```

```

    struct node* n = search(p, 10);
    if(n!=NULL){
        printf("Found: %d", n->data);
    }
    else{
        printf("Element not found");
    }
}

```

```

    }
    return 0;
}

```

Copy

Code Snippet 2: Implementing the search function

Now, make sure you store the value returned by the search function in a struct node pointer, say n. We'll see if our function actually works, and it reverts the pointer to the node it found, or a NULL if it didn't find the key.

Let's search 10, in the above function.

```

struct node* n = search(p, 10);
if(n!=NULL){
    printf("Found: %d", n->data);
}
else{
    printf("Element not found");
}

```

Copy

Code Snippet 3: Using the search function

And the output we received was:

Element not found

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 1: Output of the above code

And since 10 was not there in the binary search tree we created, the function *search* returned a NULL. Let's make it work for something which is there. Let's use the function to find 3 in the above BST.

```

struct node* n = search(p, 3);
if(n!=NULL){
    printf("Found: %d", n->data);
}
else{
    printf("Element not found");
}

```

Copy

Code Snippet 4: Using the search function again

And the output we received was:

Found: 3

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 2: Output of the above code

And yes, it says, our function *search* found the key 3 in the above binary search tree. It was simple to digest I believe. We had already learned the concepts in the last lecture. Searching shouldn't be a deal, since binary search trees were structured so as to make searching look easy. :)

So, this was all we had to do to automate the searching in a binary search tree. And that would be all. Create your own binary search trees, and test the *search* function. Please let me know if it has any flaws. It is now time to move on to our next operation which is something similar to what we did today but with a different approach.

Iterative Search in a Binary Search Tree

In the last lecture, we learned how to program the *search* function to be able to search keys in a binary search tree. The function returns the pointer to the node if it finds the key, otherwise returns NULL. We saw how easy binary search trees make the process of searching and offers a best-case runtime complexity of $\log n$.

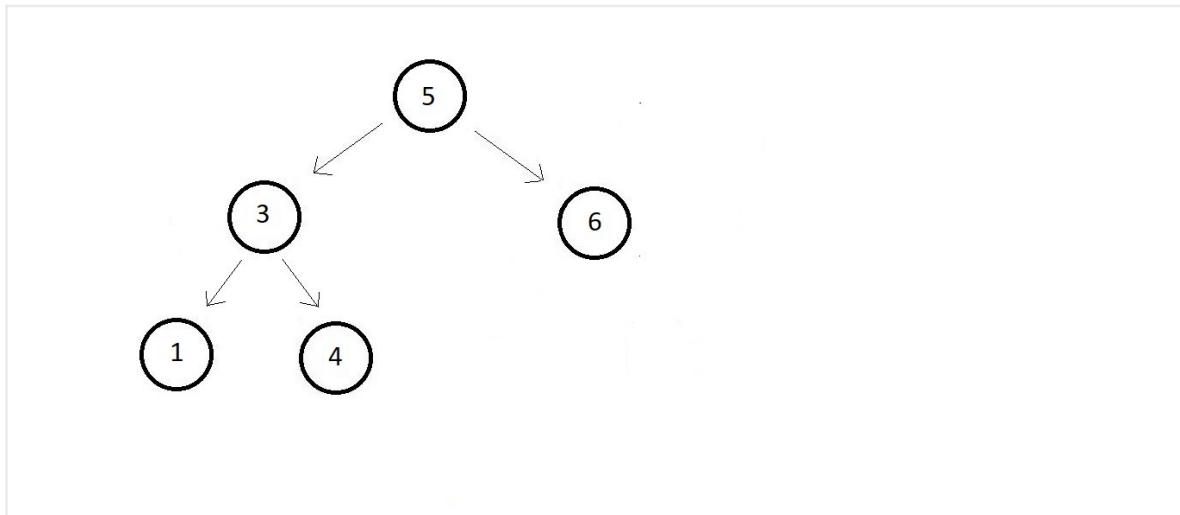
The *search* function we studied was a **recursive** function, calling the function recursively on the left or the right subtree. But today, we'll see the concept of searching **iteratively** in a binary search tree and its programming in C.

In the recursive approach, you start with the root and compare your key with the root node's data, and if it's smaller than the key, you select the whole left subtree and start searching in it thinking of it as another smaller tree. But in an iterative approach, we have the tree intact the whole time. We just make a pointer run from the root to the place we expect our key to be. Let's directly see the code for iterative search.

I have attached the source code below. Follow it as we proceed.

Understanding the code snippet below:

1. There is nothing much to explain in the code. Although for the people coming right here, we have just copied everything we had done till the last programming lecture which dealt with the search function. It had covered everything from creating a node, to building a tree. It also featured all the traversal methods, and all the other functions. We did this to save ourselves some time.
2. Let's create a binary search tree I've illustrated below using the *createNode* function. And without doing any further ado, we would move to creating the *searchIter*



Creating the function *searchIter*:

3. Create a struct Node pointer function *searchIter* and pass into it the pointer to the root and the key you want to search as two of its parameters.
4. First of all, check if the root's data itself is the key we were looking for. If the root data is equal to the key, we have found the key, and we'll return the pointer to the root.
5. If we couldn't find the key on the root, we'll further see if our key is greater than or smaller than the root data. If it is smaller than the root data, we will make the root node pointer now point to the left child of the root, using the arrow operator. And if the key is greater than the root data, we'll make the root node pointer point to the right child of the root
6. And we'll use a while loop to run steps 4 and 5 iteratively until our root becomes NULL, or we find the key and return from the function.
7. And in the end, if we couldn't find the key after iterating through the tree, or the root we received in the starting itself was a NULL, we would return NULL.

```

struct node * searchIter(struct node* root, int key){
    while(root!=NULL){
        if(key == root->data){
            return root;
        }
        else if(key<root->data){
            root = root->left;
        }
        else{
            root = root->right;
        }
    }
    return NULL;
}

```

Copy

Code Snippet 1: Creating the searchIter function

Iteration is more intuitive because you are able to see what's exactly happening and how things end. Even so, recursion is considered a powerful tool.

Here is the whole source code:

```
#include<stdio.h>
#include<malloc.h>

struct node{
    int data;
    struct node* left;
    struct node* right;
};

struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the
heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node
}

void preOrder(struct node* root){
    if(root!=NULL){
        printf("%d ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

void postOrder(struct node* root){
    if(root!=NULL){
        postOrder(root->left);
        postOrder(root->right);
        printf("%d ", root->data);
    }
}

void inOrder(struct node* root){
    if(root!=NULL){
        inOrder(root->left);
        printf("%d ", root->data);
    }
}
```

```

        inOrder(root->right);
    }
}

```

```

int isBST(struct node* root){
    static struct node *prev = NULL;
    if(root!=NULL){
        if(!isBST(root->left)){
            return 0;
        }
        if(prev!=NULL && root->data <= prev->data){
            return 0;
        }
        prev = root;
        return isBST(root->right);
    }
    else{
        return 1;
    }
}

```

```

struct node * searchIter(struct node* root, int key){
    while(root!=NULL){
        if(key == root->data){
            return root;
        }
        else if(key<root->data){
            root = root->left;
        }
        else{
            root = root->right;
        }
    }
    return NULL;
}

```

```

int main(){

```

```

    // Constructing the root node - Using Function (Recommended)
    struct node *p = createNode(5);
    struct node *p1 = createNode(3);
    struct node *p2 = createNode(6);
    struct node *p3 = createNode(1);
    struct node *p4 = createNode(4);
    // Finally The tree looks like this:
    //    5
    //   /\

```

```

// 3 6
// /\
// 1 4

// Linking the root node with left and right children
p->left = p1;
p->right = p2;
p1->left = p3;
p1->right = p4;

struct node* n = searchIter(p, 6);
if(n!=NULL){
printf("Found: %d", n->data);
}
else{
printf("Element not found");
}
return 0;
}
Copy

```

Code Snippet 2: Implementing the searchIter function

Now, make sure you store the value returned by the searchIter function in a struct node pointer, say n. We'll see if our function actually works, and it reverts the pointer to the node it found, or a NULL if it didn't find the key.

Let's search 10, in the above function. We did all this time as well.

```

struct node* n = searchIter(p, 10);
if(n!=NULL){
printf("Found: %d", n->data);
}
else{
printf("Element not found");
}
}
Copy

```

Code Snippet 3: Using the searchIter function

And the output we received was:

Element not found

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 1: Output of the above code

And since 10 was not there in the binary search tree we created, the function *searchIter* returned a NULL. Let's see if it works for something else. Let's use the function to find 6 in the above BST.

```

struct node* n = searchIter(p, 6);
if(n!=NULL){

```

```
printf("Found: %d", n->data);  
}  
else{  
    printf("Element not found");  
}
```

Copy

Code Snippet 4: Using the searchIter function again

And the output we received was:

Found: 6

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 2: Output of the above code

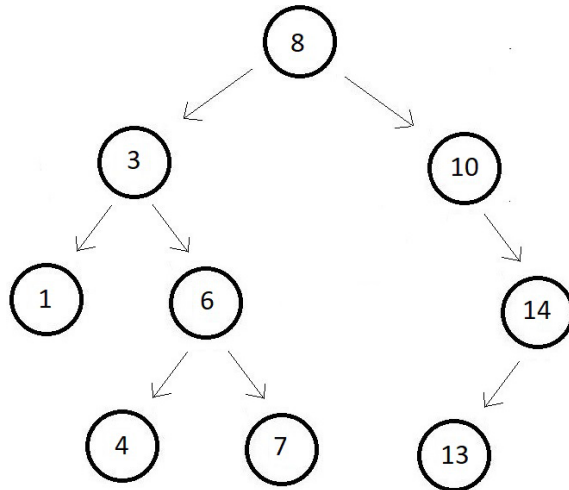
Ah yes. The function is working all good. Although we changed our approach from recursive to iterative, things remained pretty much the same. We still stop our searching after reaching the NULL. So, more or less the base case and the flow remain the same. And it did feel more intuitive. So, this was all we had in searching. We saw both approaches. I encourage you to experiment with both methods. Having well-versed in both methods, it is now time to move on to our next operation that is the insertion in a binary search tree. Furthermore, we will see the deletion of nodes in a binary search tree. Things are pretty interesting on the other side. Stay connected.

Insertion in a Binary Search Tree

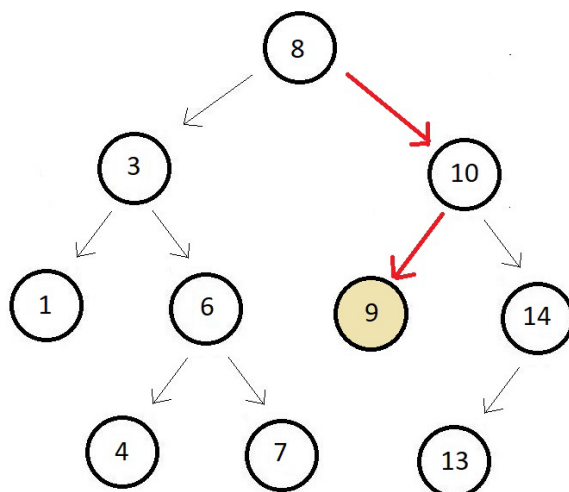
In the last lecture, we learned how to program the search function which will search for a key the user gives in the binary search tree, and revert its position if it is present in the tree, else return NULL. Today we'll learn our second operation in binary search trees which is the insertion in a binary search tree. Inserting in a binary search tree is really not a big deal, but yet very important.

Your half the job is already done if you have diligently followed the search operation. I am saying this because in the algorithm we will follow to insert an element, there are a lot of things in common to the algorithm we learn to search an element. We know a few facts about binary search trees. And I'll only mention the ones we will focus on, while we learn the insertion operation.

1. There are no duplicates in a binary search tree. So, if you could search the element you are being asked to insert, you would return that the number already exists.
2. Now, you would follow what we did in the search operation. Here is an example binary search tree, and the element we want to insert is 9.



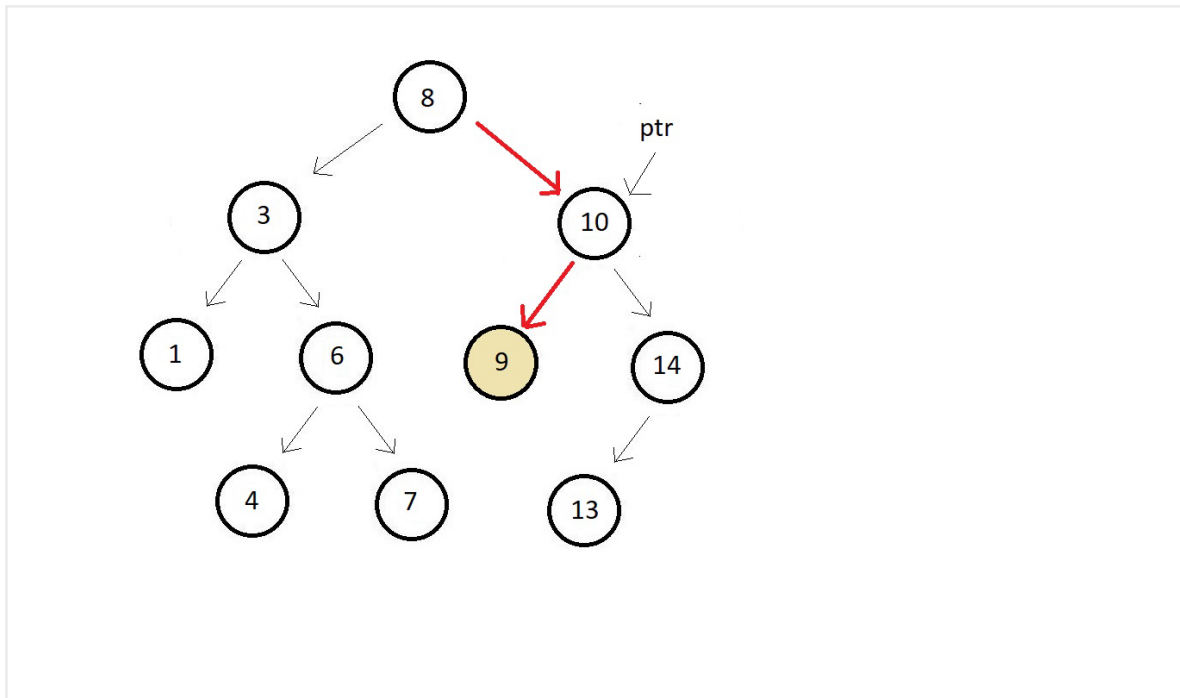
Now, you would simply start from the root node, and see if the element you want to insert is greater than or less than. And since 9 is greater than 8, we move to the right of the root. And then the root is the element 10, and since this time 9 is less than 10, we move to the left of it. And since there are no elements to its left, we simply insert element 9 there.



This was one simple case, but things become more complex when you have to insert your element at some internal position and not at the leaf. Now, before you insert a node, the first thing you would do is to create that node and allocate memory to it in heap using malloc. Then you would

initialize the node with the data given, and both the right and the left member of the node should be marked NULL.

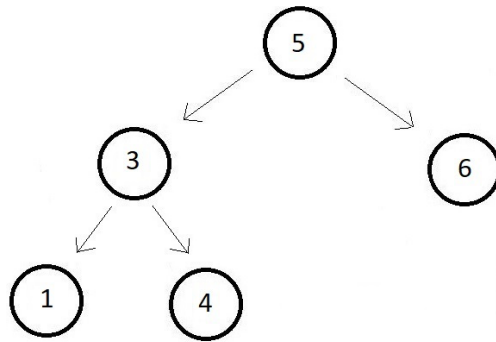
And another important thing to see here is the pointer you would follow the correct position with. In the above example, to be able to insert at that position, the pointer must be at node 10.



And then you check whether going to the left side is good, or the right. Here you came to the left, but had it been right, we would have updated our pointer *ptr* further and maintained a second pointer to the previous root. We'll see all this via our program. That would actually make things simpler to understand. I have attached the source code below. Follow it as we proceed.

Understanding the source code below:

1. Now, since our main focus would be to create the *insert* function, we could just copy everything we did in the last programming lecture where we learned the iterative search operation. And this would save us a lot of time. And we would have the *createNode* and other functions we did before at one place for our exigency.
2. Create a binary search tree of your choice, we would rather go with the one we already had in the program. Now, let's see the *insert*



Creating the *insert* function:

3. So, the first thing we would like to know is whether inserting this new node is even possible or not. For that, create a void function *insert* and pass the pointer to the root node, and the data of the node you want to insert as its parameters. We will call it
4. Now, we would use two struct pointers to traverse through the array. One of them would be our *root* which we would traverse through the nodes, and the other one would be *prev* which stores the pointer to the previous root. SO, just create a struct Node pointer *prev* to maintain the node you were previously at, at some point in time.
5. Run a while loop that is for until we reach some leaf, and couldn't traverse further. So, run that loop until the root becomes NULL. And inside that loop, make the *prev* equal to the current root since we would definitely move further because this root is not a NULL. We would either move to the left of this root or to the right of this root. But before that check, if this root itself is not equal to the node we are trying to insert. That is, write an if condition to see if there are any duplicates here. If there is, return from the function here itself.
6. Further in the loop, check if the element you want to insert is less than the current root. If it is, update the root to the left element of the struct root. And if it isn't, update the root to the right element of the struct root. And since we have already stored this root in the *prev* node, there isn't any issue updating.
7. And finally, you will have a *prev* node as the outcome at the end after this loop finishes. Now, the only procedure left now is to link these nodes together, that is the *prev* node, the *new* node, and the node *next* to the *prev*
8. Now, before you insert, make sure you create that new struct node using malloc, or simply the *createNode* Fill in the *key* data into this *new* node. Now, simply check if the *prev->data* is less than the key or greater than the key. If it is less, insert our new node to the left of *prev*, else to right of *prev*. And that would be it. We are done

```

    inserting our new node.
void insert(struct node *root, int key){
    struct node *prev = NULL;
    while(root!=NULL){
        prev = root;
        if(key==root->data){
            printf("Cannot insert %d, already in BST", key);
            return;
        }
        else if(key<root->data){
            root = root->left;
        }
        else{
            root = root->right;
        }
    }
    struct node* new = createNode(key);
    if(key<prev->data){
        prev->left = new;
    }
    else{
        prev->right = new;
    }
}
Copy

```

Code Snippet 1: Creating the *insert* function

Here is the whole source code:

```

#include<stdio.h>
#include<malloc.h>

struct node{
    int data;
    struct node* left;
    struct node* right;
};

struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the
heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node

```

```
}
```

```
void preOrder(struct node* root){  
    if(root!=NULL){  
        printf("%d ", root->data);  
        preOrder(root->left);  
        preOrder(root->right);  
    }  
}
```

```
void postOrder(struct node* root){  
    if(root!=NULL){  
        postOrder(root->left);  
        postOrder(root->right);  
        printf("%d ", root->data);  
    }  
}
```

```
void inOrder(struct node* root){  
    if(root!=NULL){  
        inOrder(root->left);  
        printf("%d ", root->data);  
        inOrder(root->right);  
    }  
}
```

```
int isBST(struct node* root){  
    static struct node *prev = NULL;  
    if(root!=NULL){  
        if(!isBST(root->left)){  
            return 0;  
        }  
        if(prev!=NULL && root->data <= prev->data){  
            return 0;  
        }  
        prev = root;  
        return isBST(root->right);  
    }  
    else{  
        return 1;  
    }  
}
```

```
struct node * searchIter(struct node* root, int key){  
    while(root!=NULL){  
        if(key == root->data){  
            return root;  
        }  
    }
```

```

    }
    else if(key<root->data){
        root = root->left;
    }
    else{
        root = root->right;
    }
}
return NULL;
}

```

```

void insert(struct node *root, int key){
    struct node *prev = NULL;
    while(root!=NULL){
        prev = root;
        if(key==root->data){
            printf("Cannot insert %d, already in BST", key);
            return;
        }
        else if(key<root->data){
            root = root->left;
        }
        else{
            root = root->right;
        }
    }
    struct node* new = createNode(key);
    if(key<prev->data){
        prev->left = new;
    }
    else{
        prev->right = new;
    }
}

```

```

int main(){

    // Constructing the root node - Using Function (Recommended)
    struct node *p = createNode(5);
    struct node *p1 = createNode(3);
    struct node *p2 = createNode(6);
    struct node *p3 = createNode(1);
    struct node *p4 = createNode(4);
    // Finally The tree looks like this:
    //    5
    //   /\

```

```

// 3 6
// /\
// 1 4

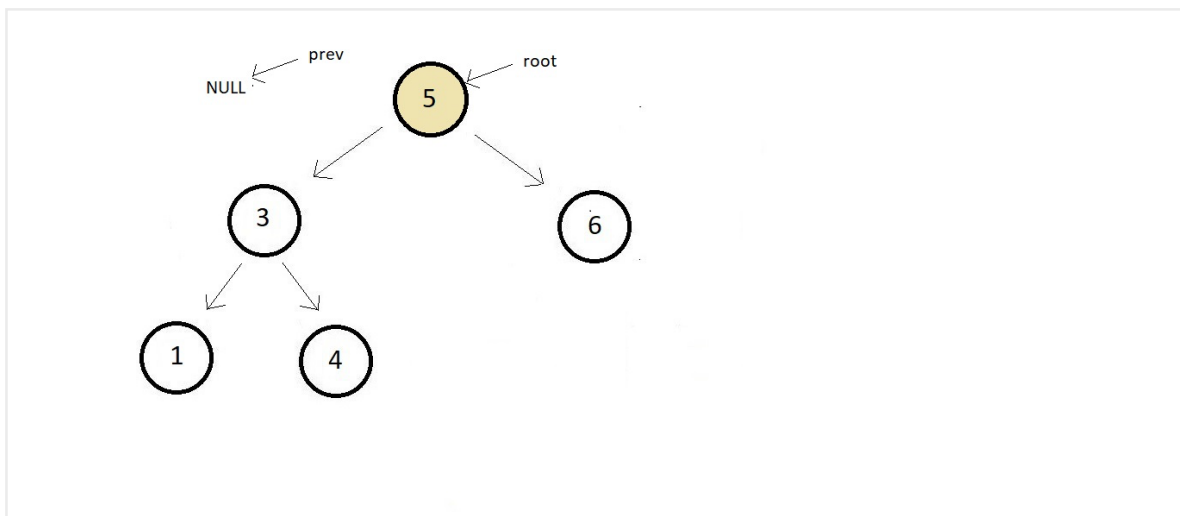
// Linking the root node with left and right children
p->left = p1;
p->right = p2;
p1->left = p3;
p1->right = p4;

insert(p, 16);
printf("%d", p->right->right->data);
return 0;
}
Copy

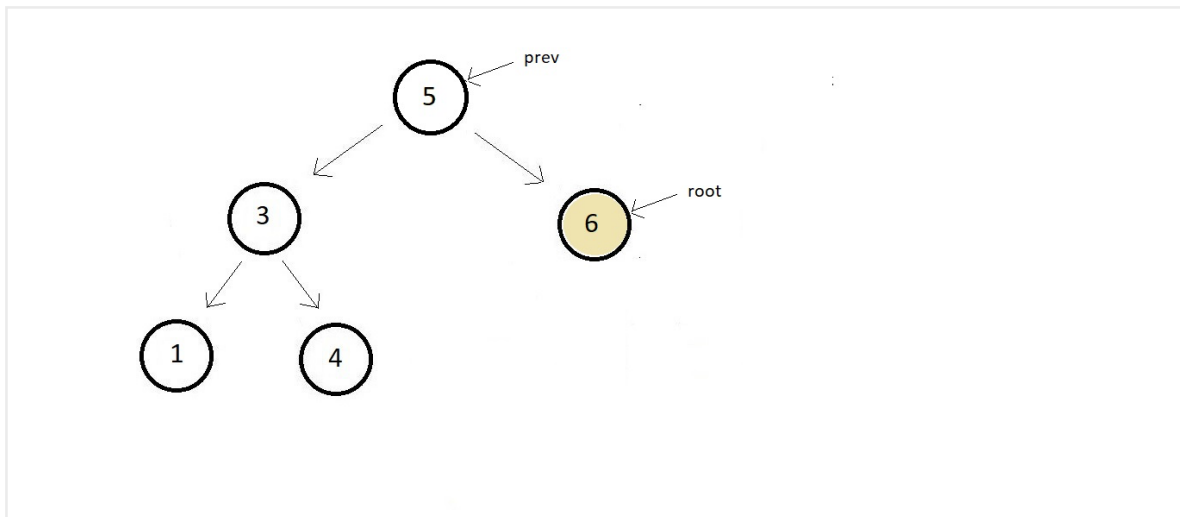
```

Code Snippet 2: Implementing the *insert* function in C

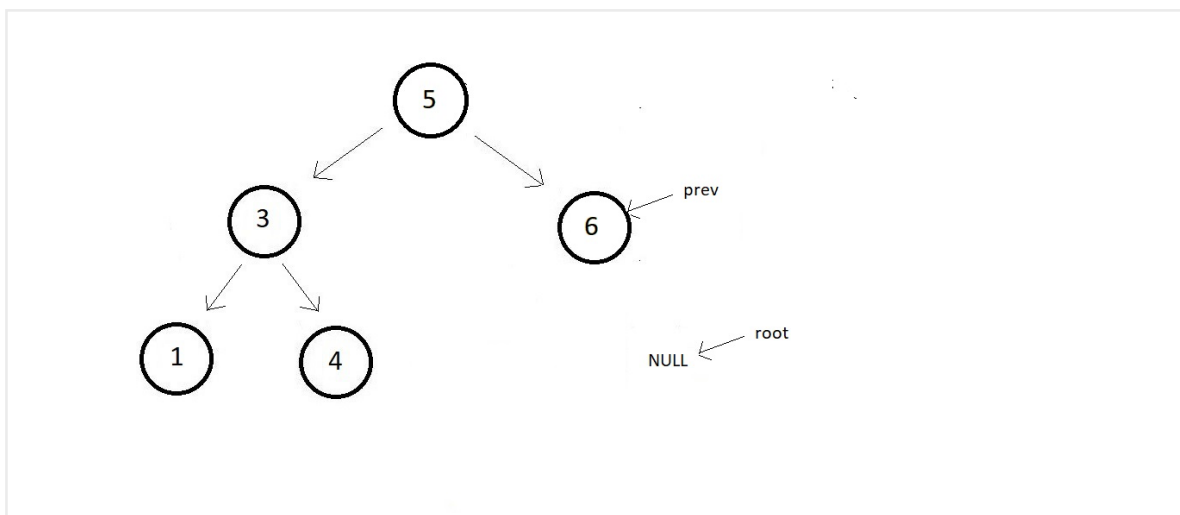
Now, suppose we want to insert element 7 in the above BST we had in the program. Now, let's first dry run that ourselves. And then we'll verify the position of the inserted node via the program. So, firstly, *prev* is NULL. And we start driving in the loop.



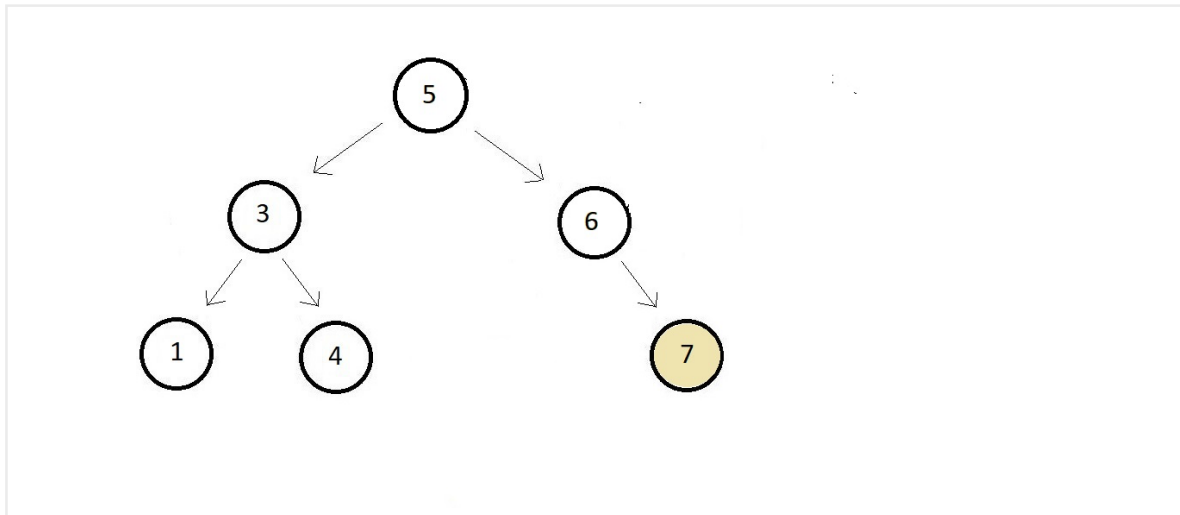
Now, first, we'll make our *prev* point to the root node, and since 7 is greater than 5, we move to the right of the root.



Now, we'll again update our *prev* with root, and since root is still smaller than 7, we'll move to the root's right which is a NULL. So, our loop ends here, and we get node 6 as our *prev* node.



And since 6 is smaller than 7, we'll insert node 7 to the right of node 6. And hence our element 7 gets placed at the position relative to the root as root -> right -> right.



Now, let's verify this by running the *insert* function, and print the data at the root -> right -> right position.

```
insert(p, 7);  
printf("%d", p->right->right->data);
```

Copy

Code Snippet 3: Using the *insert* function

And the output we received was:

7

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

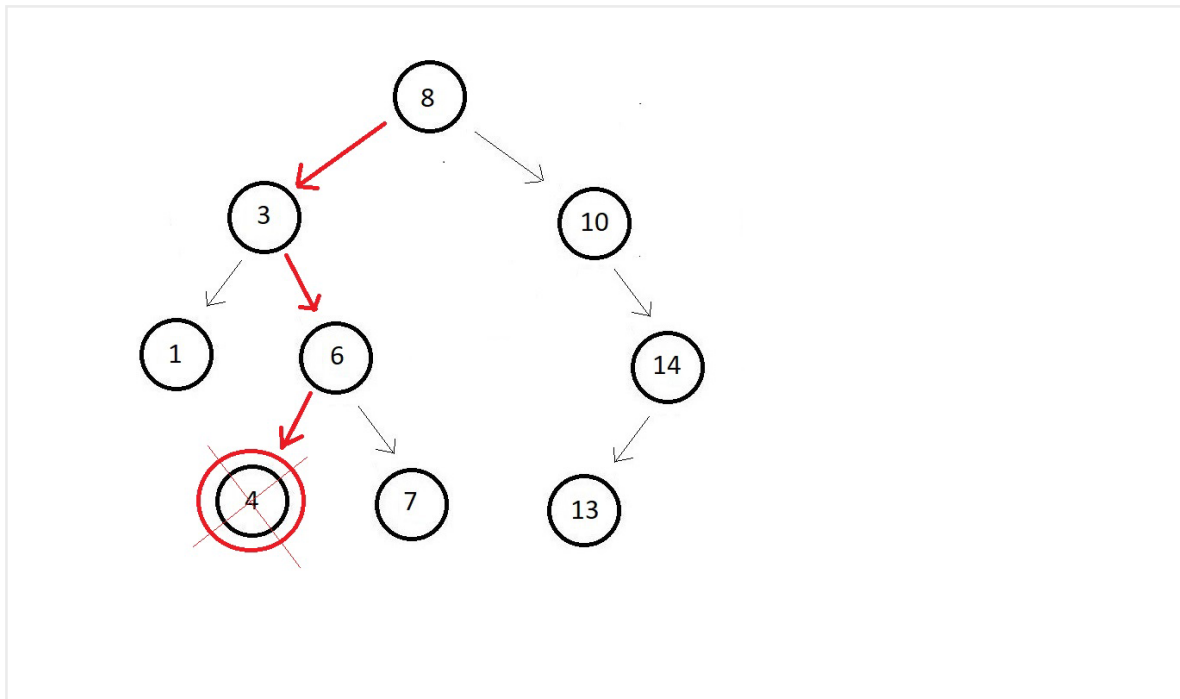
Copy

Figure 1: Output of the above code

Heh, so that got inserted with this ease. And that is all we had in the insertion operation. It is possible to learn this on your own if you practice. Next, we have the deletion function. Having learnt the insertion and the search operations would make learning deletion a cakewalk. You just have to be patient with me.

Deletion in a Binary Search Tree

In the last lecture, we saw insertion in a binary search tree. We learned how to program the insertion function in C. We drew similarities between the insertion and the search operation making it easy to digest. Today, we'll learn our third operation, the deletion operation in a binary search tree. You might be wondering how big a deal is deleting some nodes in a binary search tree. Well, for cases like the one I have drawn below, where you'll be asked to remove say element 4 seems quite an easy job. You'll just search for the element and remove it.



But deleting in a binary search tree is no doubt a tough job like if you consider a case where the node, we'll have to remove might not be a leaf node or the node is the root node. But you should not be the one worrying here. I have some easy way out. We'll explore them in detail now.

So, whenever we talk about deleting a node from binary search tree, we have the following three cases in mind:

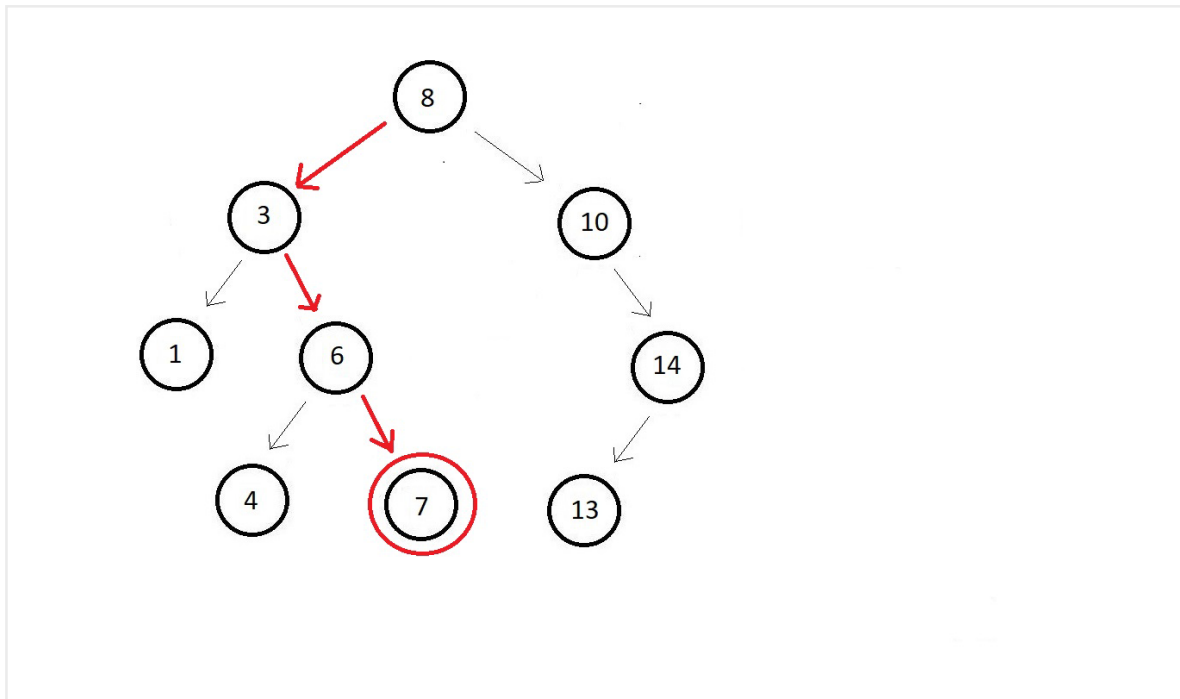
1. The node is a leaf node.
2. The node is a non-leaf node.
3. The node is the root node.

Let's deal with each of these in detail, starting with deleting the leaf node.

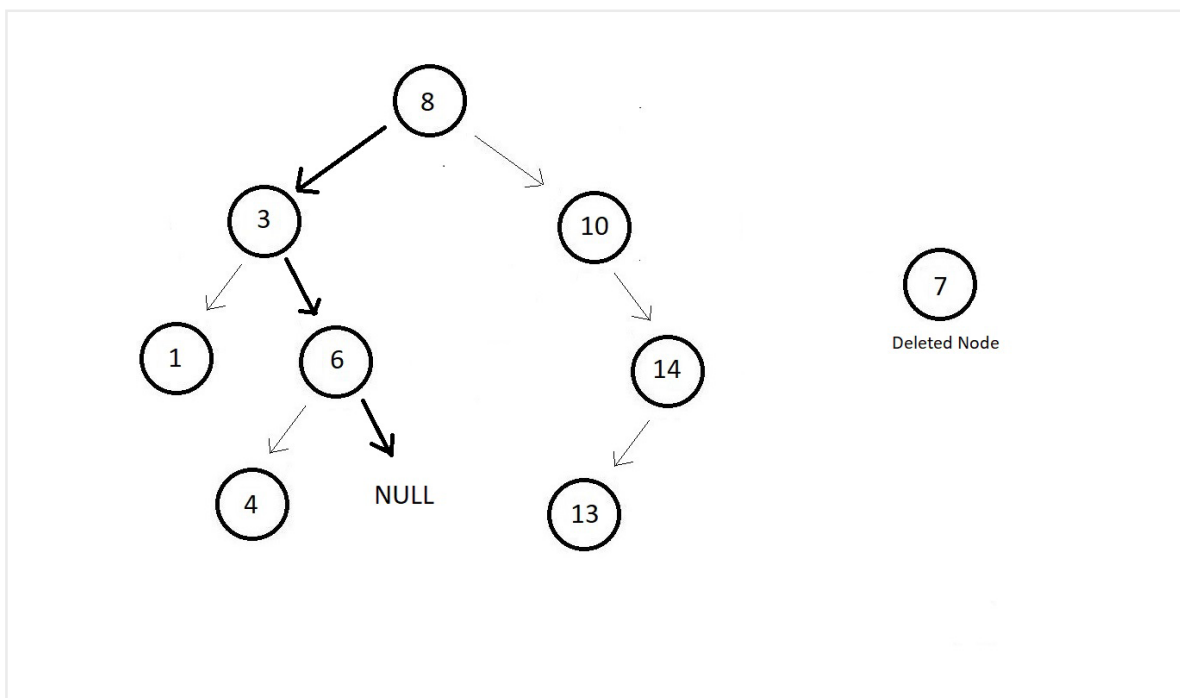
Deleting a leaf node:

Deleting a leaf node is the simplest case in deletion in binary search trees where the only thing you have to do is to search the element in the tree, and remove it from the tree, and make its parent node point to NULL. To be more specific, follow the steps below to delete a leaf node along with the illustrations of how we delete a leaf node in the above tree:

1. Search the node.



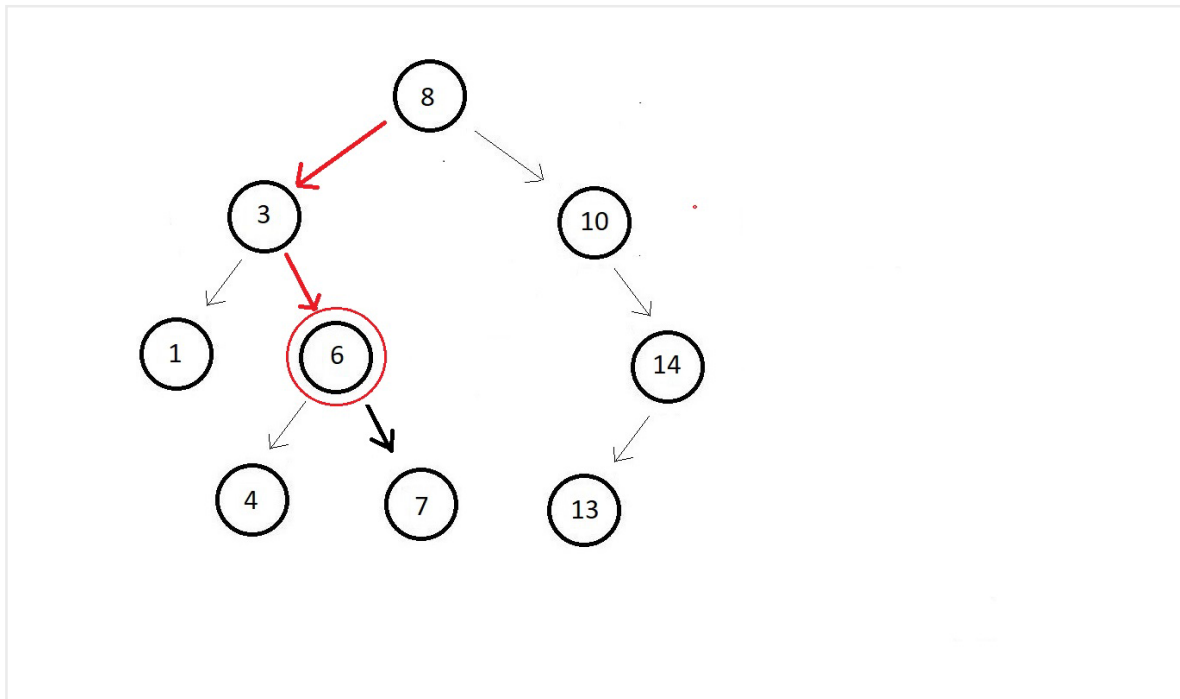
2. Delete the node.



Deleting a non-leaf node:

Now suppose the node is not a leaf node, so you cannot just make its parent point to NULL, and get away with it. You have to even deal with the children of this node. Let's try deleting node 6 in the above binary search tree.

1. So, the first thing you would do is to search element 6.

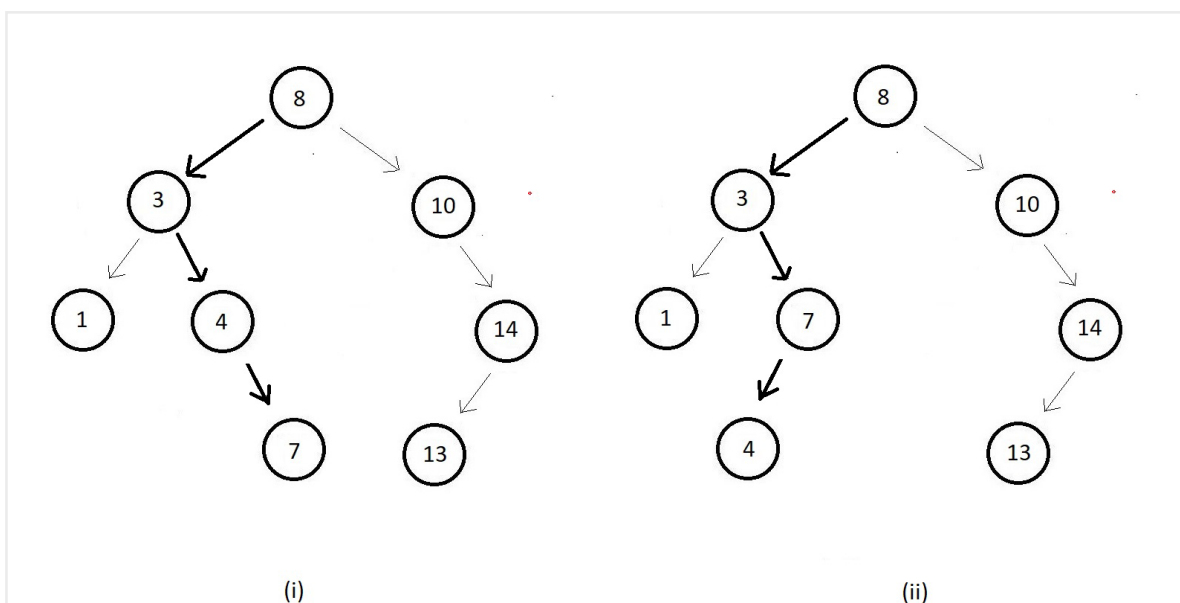


2. Now the dilemma is, which node will replace the position of node 6. Well, there is a simple answer to it. It says, when you delete a node that is not a leaf node, you replace its position with its **InOrder predecessor or Inorder successor**.

So, what does that mean? It means that if you write the InOrder traversal of the above tree, which I have already taught in my previous lectures, the nodes coming immediately before or after node 6, will be the one replacing it. So, if you write the InOrder traversal of the tree, you will get:

$1 \rightarrow 3 \rightarrow 4 \rightarrow \mathbf{6} \rightarrow 7 \rightarrow 8 \rightarrow 10 \rightarrow 13 \rightarrow 14$

So, the InOrder predecessor and the Inorder successor of node 6 are 4 and 7 respectively. Hence you can substitute node 6 with any of these nodes, and the tree will still be a valid binary search tree. Refer to how it looks below.

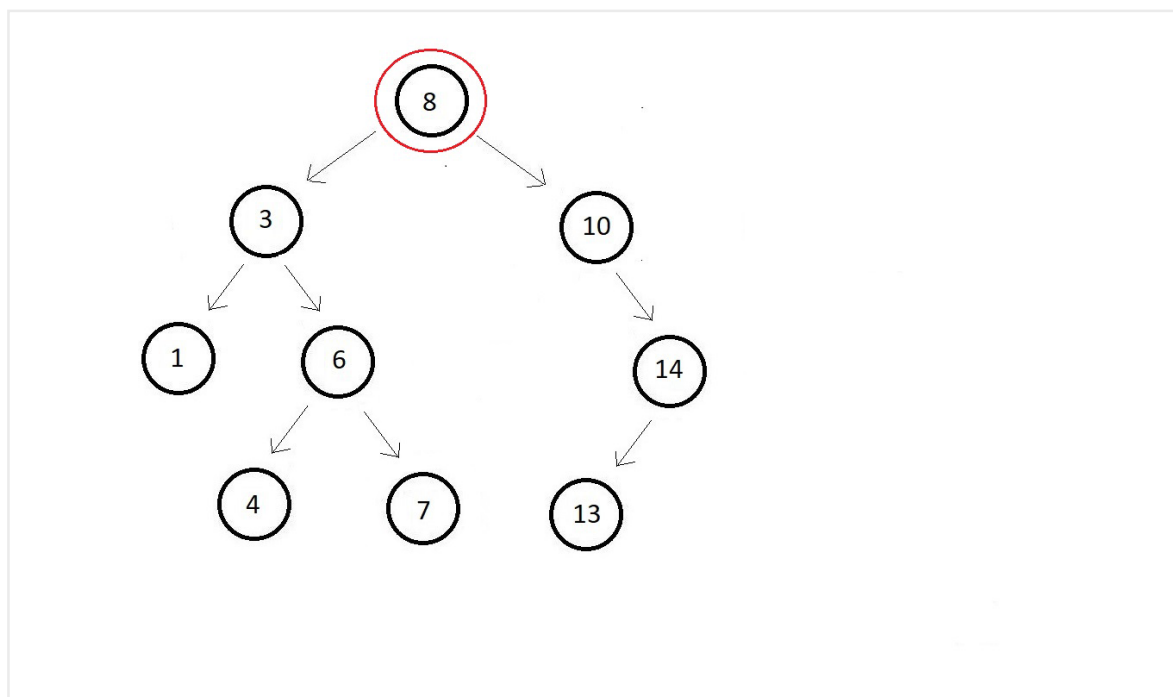


So, both are still binary search trees. In the first case, we replaced node 6

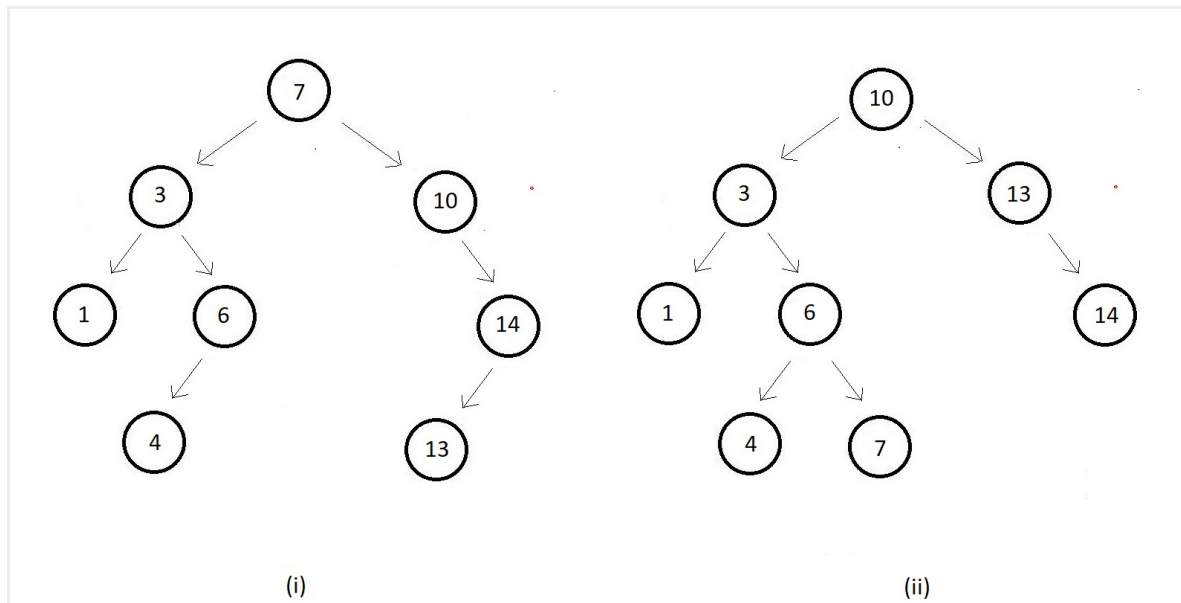
with node 4. And the right subtree of node 4 is 7, which is still bigger than it. And in the second case, we replaced node 6 with node 7. And the left subtree of node 7 is 4, which is still smaller than the node. Hence, a win-win for us.

Deleting the root node:

1. Now, if you carefully observe, the root node is still another non-leaf node. So, the basics to delete the root node remains the same as what we did for a general non-leaf node. But since the root node holds a big size of subtrees along with, we have put this as a separate case here.



2. So, the first thing you do is write the InOrder traversal of the whole tree. And then replace the position of the root node with its **InOrder predecessor or Inorder successor**. So, here the traversal order is, $1 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 10 \rightarrow 13 \rightarrow 14$. So, the InOrder predecessor and the InOrder successor of the root node 8 are 7 and 10 respectively. Hence you can substitute node 8 with any of these nodes, but there is a catch here. So, if you substitute the root node here, with its InOrder predecessor 7, the tree will still be a binary search tree, but when you substitute the root node here, with its InOrder successor 10, there still becomes an empty position where node 10 used to be. So, we still placed the InOrder successor of 10, which was 13 on the position where 10 used to be. And then there are no empty nodes in between. This finalizes our deletion.



So, there are a few steps:

1. First, search for the node to be deleted.
2. Search for the InOrder Predecessor and Successor of the node.
3. Keep doing that until the tree has no empty nodes.

And this case is not limited to the root nodes, rather any nodes falling in between a tree. Well, there could be a case where the node was not found in the tree, so, for that, we would revert the statement that the node could not be found.

So, that was all we had. We saw all three cases in detail. We applied all our previous knowledge of the traversal series to help ourselves here. Next, we will see the programming part of the deletion operation. Practice on your own BST. Try deleting nodes in between the trees.

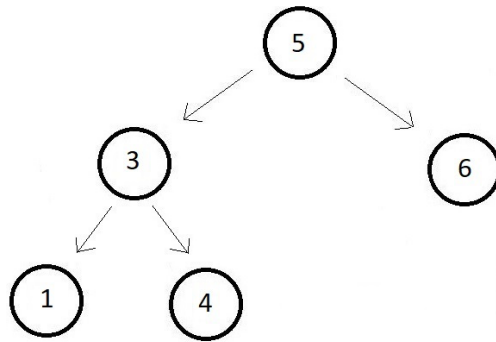
C Code For Deletion in a Binary Search Tree

In the last lecture, we dealt with all the cases of deletion operations in a binary search tree. Despite the complexity of the cases, we made them all appear easy. We saw examples of all the cases. Today, we'll see how to program those deletion cases and overall, the implementation of the deletion operation in a binary search tree using C language.

So, let's just move on to our editors without any further ado. I have attached the source code below. Follow it as we proceed.

Understanding the source code below:

1. Now, since our main focus would be to create the *deleteNode* function, we could just copy everything we did in the last programming lecture where we learnt the *insertion* operation in a binary search tree. And this would save us a lot of time. And we would then have the InOrder and other traversal functions we did before at one place for our convenience.
2. You can always create a binary search tree of your own, and check if it is a BST using the InOrder traversal, and see if it's in ascending order or not. We would rather go with the one we already had in the program. Now, let's see the *deleteNode*



Creating the *deleteNode* function:

3. So, the first thing we would like to know is whether deleting this new node is even possible or not. For that, create a struct Node pointer function *deleteNode* and pass the pointer to the root node, and the data of the node you want to delete as its parameters. We will call it *value*

4. So, the next thing you would do is see whether our *value* is greater than or less than the root node's data. If it's less than the root node's data, we'll simply recursively call the function to its left subtree, and pass the same *value* into it. And if it's greater, we'll do the same thing with the right subtree. So, this basically reduces our problem. We do this until we reach the node we want to delete, or to the NULL, signifying the node was not there in the tree.

5. So, the deletion strategy says, when you find the node you want to delete, if you remember what we learnt, we can substitute it with its InOrder predecessor or with its InOrder successor. And we prefer doing it here using the InOrder predecessor. So, the first thing you would do here is to find the Inorder predecessor of the root node you want to delete and store it in a struct node pointer *iPre*. We would have a separate function for this, named *inOrderPredecessor* which we'll see later. Suppose it returns the pointer to the InOrder Predecessor of the root node.

6. Having received the pointer to the InOrder Predecessor of the root node, you just copy the data of that predecessor node to our root node. And call the *deleteNode* function recursively to the left subtree of the root node, and with the value of the InOrder predecessor's data. Now, observe this carefully. Why did we choose the left subtree? Because the InOrder predecessor of a node always lies on the left subtree of the node. And since you have replaced your root node with its InOrder predecessor, you have to now delete it from the left subtree.

7. And since these functions are recursive, we would need a base condition, and if you could realize, this recursive deletion of the replacing nodes must stop when it reaches a leaf node. So, the base conditions are when our

given root is NULL, here we simply return NULL or both it's left and the right subtree is NULL, this is where the node we want to delete is the leaf node, so we'll simply free this node. And simply return the root node at the end.

```
struct node *deleteNode(struct node *root, int value){
```

```
    struct node* iPre;
    if (root == NULL){
        return NULL;
    }
    if (root->left==NULL&&root->right==NULL){
        free(root);
    }

    //searching for the node to be deleted
    if (value < root->data){
        deleteNode(root->left,value);
    }
    else if (value > root->data){
        deleteNode(root->right,value);
    }
    //deletion strategy when the node is found
    else{
        iPre = inOrderPredecessor(root);
        root->data = iPre->data;
        deleteNode(root->left, iPre->data);
    }
}
```

Copy

Code Snippet 1: Creating the *deleteNode* function

8. Now, this deleteNode function would not simply work alone. We would need an *inOrderPredecessor* function we have used in the program.

Creating the *inOrderPredecessor* function:

9. Create a struct node pointer function *inOrderpredecessor* and pass into it the pointer to the root node you want to find the InOrder predecessor of as its parameter. I hope you all would have realised the fact that an Inorder predecessor of a node is the rightmost node of its left subtree.

10. So, simply update root to its left subtree, and use a while loop to iterate through all the right subtree, until we reach a leaf node. This gives our InOrder predecessor of the given node. Here, return the root.

```
struct node *inOrderPredecessor(struct node* root){
    root = root->left;
    while (root->right!=NULL)
    {
```

```

        root = root->right;
    }
    return root;
}
Copy

```

Code Snippet 2: Creating the inOrderPredecessor function

Here is the whole source code:

```

#include<stdio.h>
#include<malloc.h>

struct node{
    int data;
    struct node* left;
    struct node* right;
};

struct node* createNode(int data){
    struct node *n; // creating a node pointer
    n = (struct node *) malloc(sizeof(struct node)); // Allocating memory in the
heap
    n->data = data; // Setting the data
    n->left = NULL; // Setting the left and right children to NULL
    n->right = NULL; // Setting the left and right children to NULL
    return n; // Finally returning the created node
}

void preOrder(struct node* root){
    if(root!=NULL){
        printf("%d ", root->data);
        preOrder(root->left);
        preOrder(root->right);
    }
}

void postOrder(struct node* root){
    if(root!=NULL){
        postOrder(root->left);
        postOrder(root->right);
        printf("%d ", root->data);
    }
}

void inOrder(struct node* root){
    if(root!=NULL){
        inOrder(root->left);
        printf("%d ", root->data);
    }
}

```

```

        inOrder(root->right);
    }
}

```

```

int isBST(struct node* root){
    static struct node *prev = NULL;
    if(root!=NULL){
        if(!isBST(root->left)){
            return 0;
        }
        if(prev!=NULL && root->data <= prev->data){
            return 0;
        }
        prev = root;
        return isBST(root->right);
    }
    else{
        return 1;
    }
}

```

```

struct node * searchIter(struct node* root, int key){
    while(root!=NULL){
        if(key == root->data){
            return root;
        }
        else if(key<root->data){
            root = root->left;
        }
        else{
            root = root->right;
        }
    }
    return NULL;
}

```

```

void insert(struct node *root, int key){
    struct node *prev = NULL;
    while(root!=NULL){
        prev = root;
        if(key==root->data){
            printf("Cannot insert %d, already in BST", key);
            return;
        }
        else if(key<root->data){
            root = root->left;
        }
    }
}

```



```

        else{
            root = root->right;
        }
    }
    struct node* new = createNode(key);
    if(key<prev->data){
        prev->left = new;
    }
    else{
        prev->right = new;
    }
}

struct node *inOrderPredecessor(struct node* root){
    root = root->left;
    while (root->right!=NULL)
    {
        root = root->right;
    }
    return root;
}

struct node *deleteNode(struct node *root, int value){

    struct node* iPre;
    if (root == NULL){
        return NULL;
    }
    if (root->left==NULL&&root->right==NULL){
        free(root);
        return NULL;
    }

    //searching for the node to be deleted
    if (value < root->data){
        root-> left = deleteNode(root->left,value);
    }
    else if (value > root->data){
        root-> right = deleteNode(root->right,value);
    }
    //deletion strategy when the node is found
    else{
        iPre = inOrderPredecessor(root);
        root->data = iPre->data;
        root->left = deleteNode(root->left, iPre->data);
    }
    return root;
}

```

```

}

int main(){

    // Constructing the root node - Using Function (Recommended)
    struct node *p = createNode(5);
    struct node *p1 = createNode(3);
    struct node *p2 = createNode(6);
    struct node *p3 = createNode(1);
    struct node *p4 = createNode(4);
    // Finally The tree looks like this:
    //   5
    //  /\
    // 3  6
    // /\
    // 1  4

    // Linking the root node with left and right children
    p->left = p1;
    p->right = p2;
    p1->left = p3;
    p1->right = p4;

    inOrder(p);
    printf("\n");
    deleteNode(p, 3);
    inOrder(p);

    return 0;
}

```

Copy

Code Snippet 3: Implementing the *deleteNode* function in C

Now, let's verify this function by running the *deleteNode* function with the value being 3, and to see if the node really got deleted, we'll run the *inOrder* function before and after the deletion operation.

```

inOrder(p);
printf("\n");
deleteNode(p, 3);
inOrder(p);

```

Copy

Code Snippet 4: Using the *deleteNode* function

And the output we received was:

```

1 3 4 5 6
1 4 5 6

```

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 1: Output of the above code

So, yes, node 3 got deleted from the tree. And our tree didn't lose its identity, it remained a binary search tree. You can run the same function for all the other nodes to see if it works for all the cases. And try this for even bigger trees. Go through the last lecture again if you couldn't get hold of the basics of deletion. Next, we have our new data structure in the list, called the AVL trees.

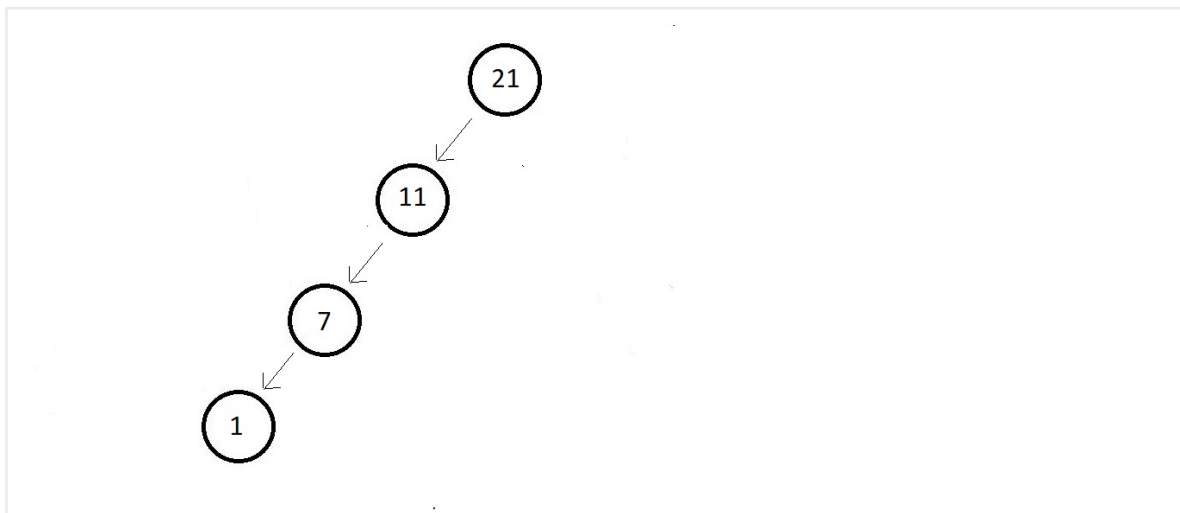
AVL Trees - Introduction

In the last lecture, we finished learning our operations on a binary search tree. We learned to search, insert, and all the cases of deletion in a binary search tree. Today, we'll start learning about the AVL trees.

Well, the operations we have discussed lately have been observed to work faster when the tree is highly distributed, or where the height is actually close to $\log n$ and the complexity tends to $O(\log n)$, but they come close to $O(n)$ when our tree becomes sparse, and looks unnecessarily lengthened. This is where AVL trees come to the rescue. I'll explain more on why AVL trees are considered a different topic worth teaching and why we even need AVL trees.

Why AVL trees?

I'll take an example to help you all understand the concept. Suppose you are told to create a Binary Search tree out of some elements given to you. Suppose the numbers were {1, 11, 7, 21}. And to avoid any labour, you simply sort the elements and write them as shown in the figure below.

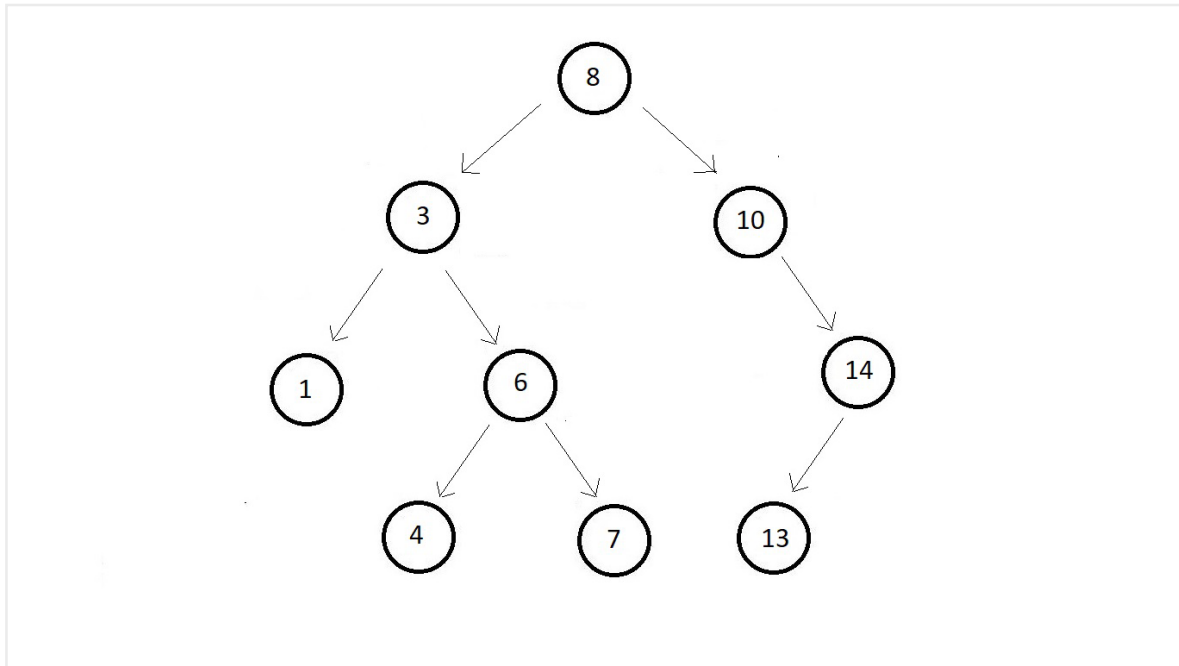


And to be honest, you cannot deny the fact that this is indeed a binary search tree, though skewed. There is no violation of any of the rules of a binary search tree. But any operation here has a complexity $O(n)$. But I can guarantee, you expected something more balanced, something more dense. Well, that is what an AVL tree is.

An AVL tree is needed because:

1. Almost all the operations in a binary search tree are of order $O(h)$ where h is the height of the tree.
2. If we don't plan our trees properly, this height can get as high as n where n is the number of nodes in the Binary Search Tree (Skewed tree).
3. So, to guarantee an upper bound of $O(\log n)$ for all these operations we use balanced trees.

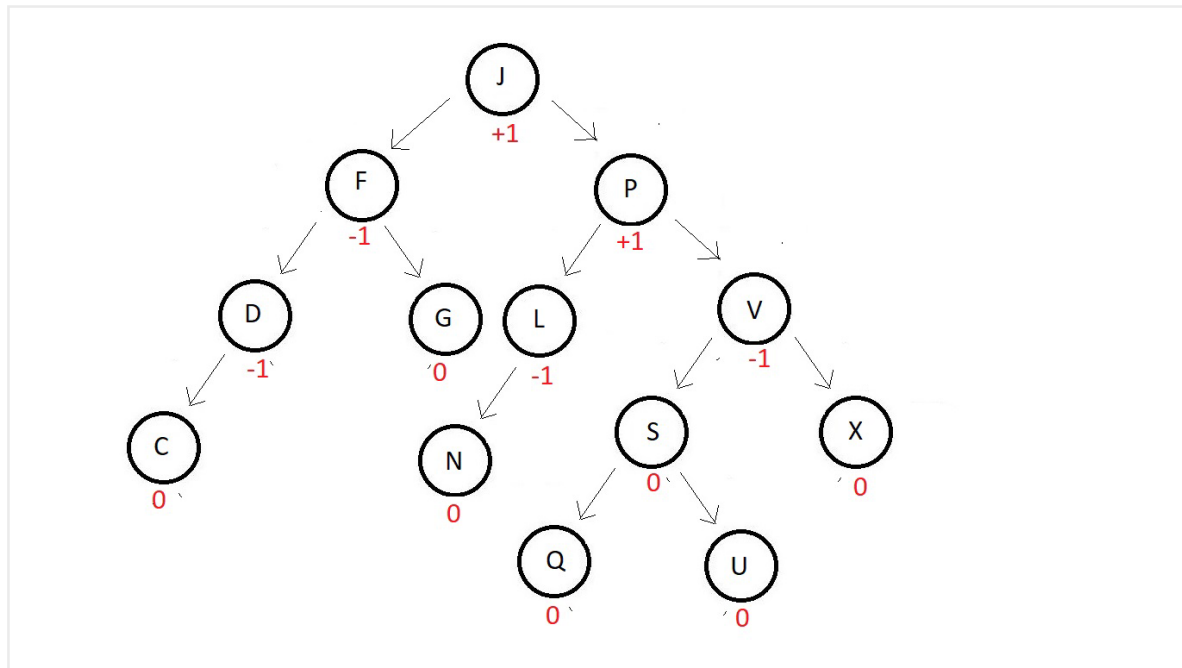
This is actually very practical. Because when a binary search tree takes the form of a list, our operations, say searching, starts taking more time. Consider the Binary search tree below.



To search 1, in this binary search tree, you would need only 3 operations, while to search in a list type binary search tree having the same elements, this would have taken 9 operations.

What are AVL trees?

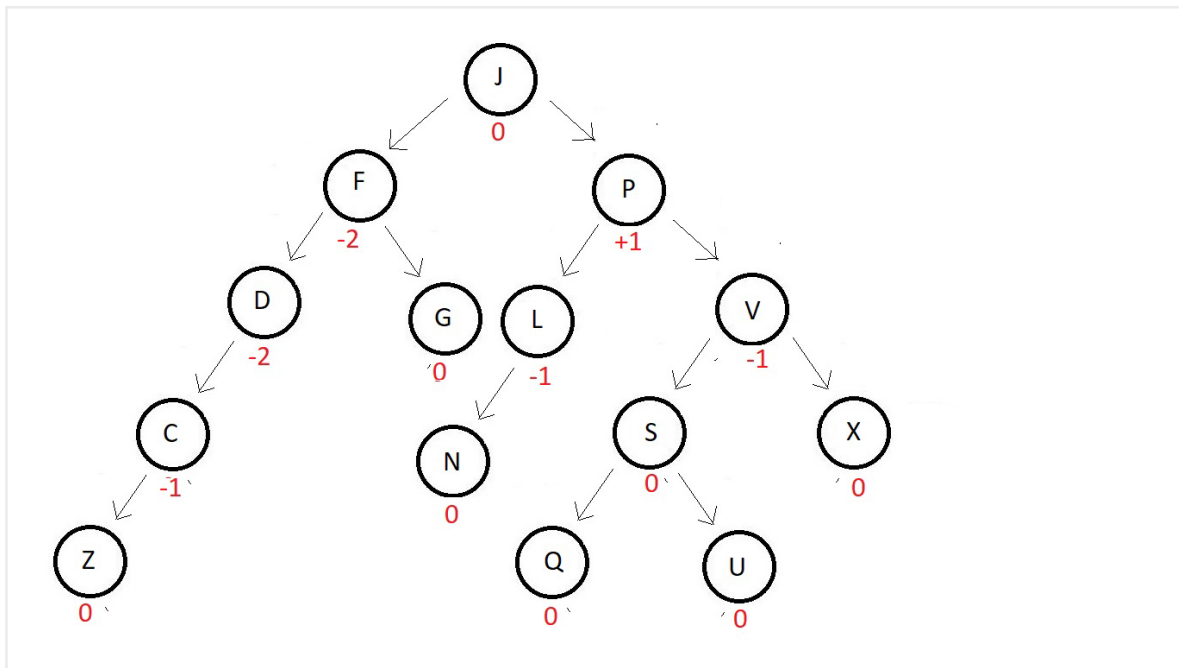
1. AVL trees are height balanced binary search trees. Because most of the operations work on $O(h)$, we would want the value of h to be minimum possible, which is $\log(n)$.
2. Height difference between the left and the right subtrees is less than 1 or equal in an AVL tree.
3. For AVL trees, there is a balance factor BF, which is equal to the height of the left subtree subtracted from the height of the right subtree. If we consider the below binary search tree, you can see the balance factor mentioned beside each node. Carefully observe each of those.



You can see, none of the nodes above have a balance factor more than 1 or less than -1. So, for a balance tree to be considered an AVL tree, the value of $|BF|$ should be less than or equal to 1 for each of the nodes, i.e., $|BF| \leq 1$.

4. And even if some of the nodes in a binary search tree have a $|BF|$ less than or equal to 1, those nodes are considered balanced. And if all the nodes are balanced, it becomes an AVL.

One thing before we finish. An AVL tree gets disturbed sometime when we try inserting a new element in it. For example, in the above AVL tree, if we try inserting an element Z at the end of the leftmost element, the balanced factor gets updated for each of the nodes following above. And the tree is no more an AVL tree. See the updated tree below.



And to avoid this unbalancing, we have an operation called **rotation** in AVL trees. This helps maintain the balancing of nodes even after a new element gets inserted. I'll show you how in the next video.

So, this was all we had for today. We will see the operations, the insertion, and the rotation in the AVL tree in the next video. Take a list of elements and make a balanced tree out of it yourself, and find the balancing factor of each node. And keep practicing.

I appreciate your support throughout. I hope you enjoyed the tutorial. If you genuinely appreciate my work, please let your friends know about this course too. If you haven't checked out the whole playlist yet, move on to codewithharry.com or my YouTube channel to access it. See you all in the next tutorial where we'll see a few operations, **rotation, and insertion in AVL trees**.

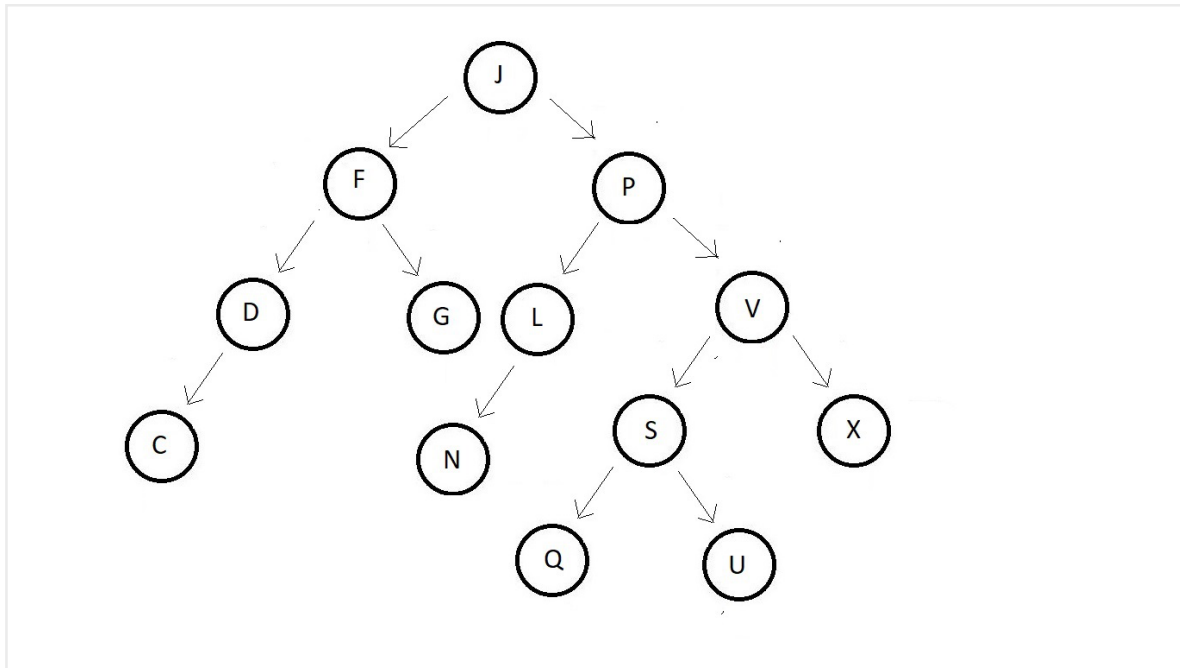
Insertion and Rotation in AVL Tree

In the last lecture, we got introduced to a new variant of the binary search tree, called the AVL trees. We saw why we needed AVL trees and how we judge whether a binary search tree is an AVL tree or not. Today, we'll learn about the rotations in an AVL tree, and why rotation is needed when you insert a new element in an AVL tree.

Before we see insertion and rotations in AVL trees, we'll give ourselves a quick brief revision of the characteristics of an AVL tree. These are:

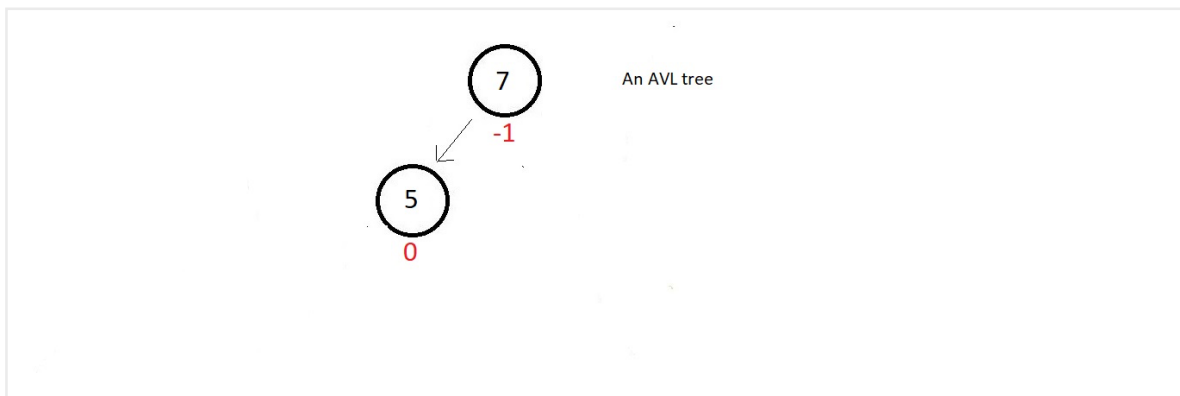
1. AVL trees are height-balanced binary search trees.
2. It gives an upper bound of $O(\log n)$ to all the operations possible in a Binary Search Tree.
3. Balance factor, $BF = \text{Height of Right Subtree} - \text{Height of Left Subtree}$, and for a binary search tree to be an AVL tree, $|BF|$ should be less than or equal to 1 for all of its nodes.

An example of an AVL tree is:

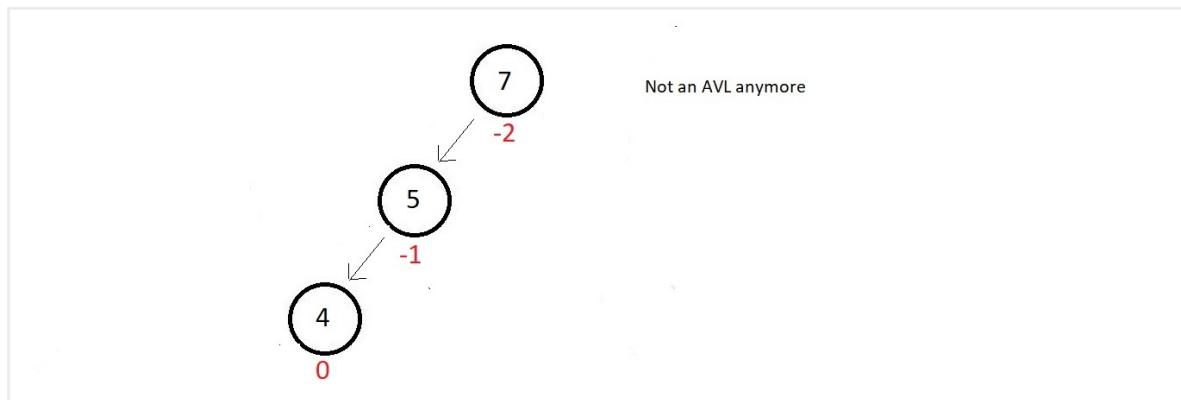


Now, before we proceed to see what different types of rotation in an AVL tree we have, we would first like to know why rotation is even done.

In an AVL tree, rotations are performed to avoid the unbalancing of a node caused by insertion. Now, suppose we have a small AVL tree having just these two nodes.



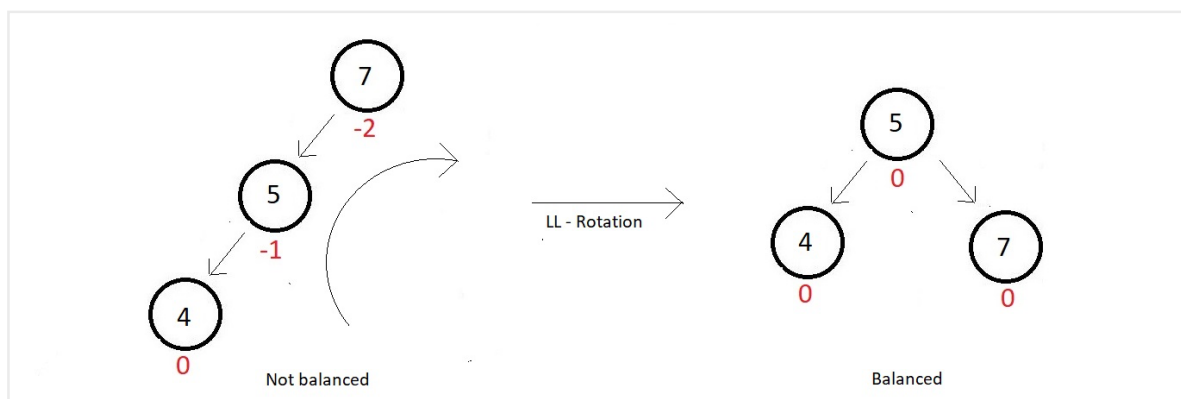
And you can see, the balance factor of both the nodes are good, but as soon as we insert a new node having data 4, our updated tree becomes unbalanced to the left. The absolute balance factor of node 7 becomes greater than 1.



So, the method you will follow to make this tree an AVL again is called **rotation**. Now, rotations can be of different types, one of them being the **LL rotation**.

LL Rotation:

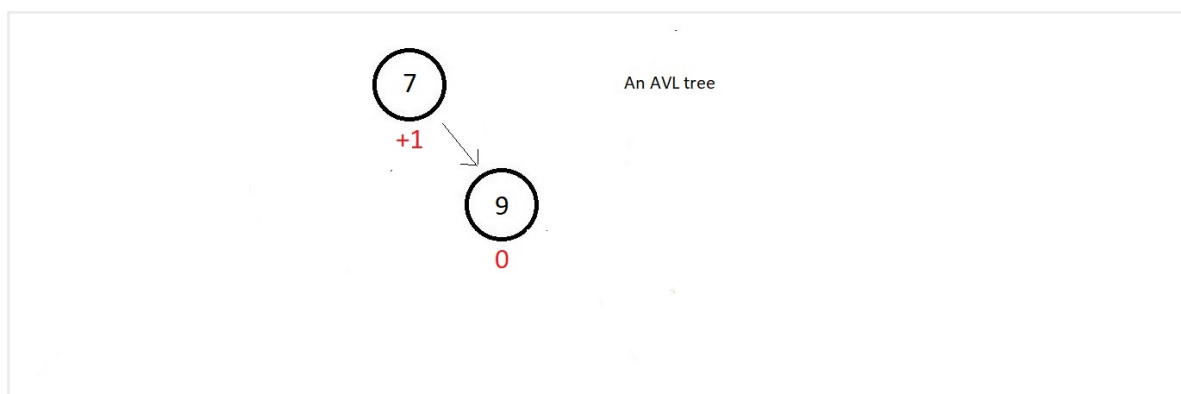
The name LL, just because we inserted the new element to the left subtree of the root. In this rotation technique, you just simply rotate your tree one time in the clockwise direction as shown below.



So, our tree got balanced again, with a perfect balance factor at each of its nodes. Next, we have the RR rotation.

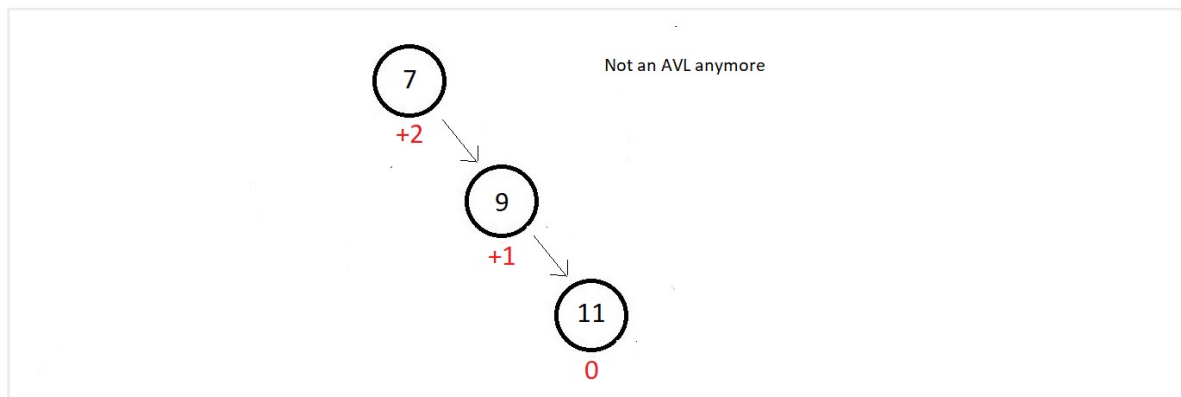
RR Rotation:

Now, suppose we have a small AVL tree having just these two nodes.

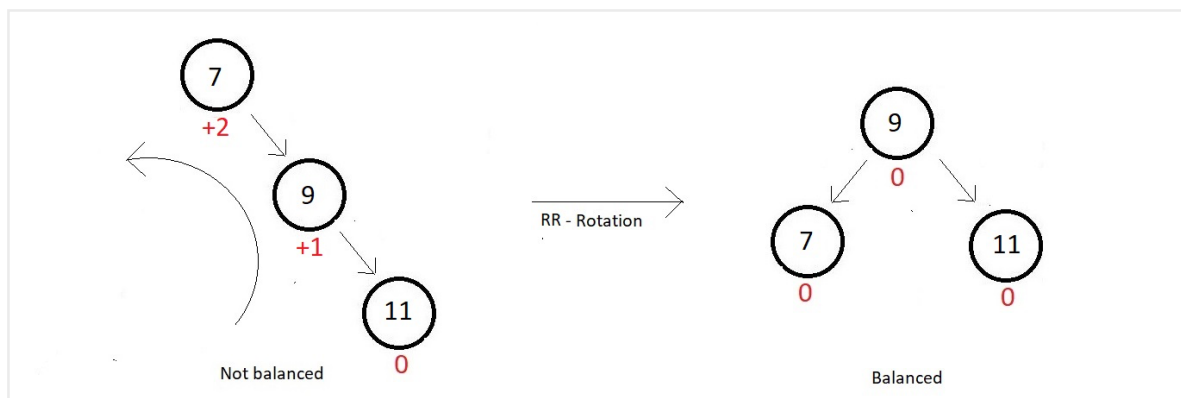


And you can see, the balance factor of both the nodes are good, but as soon as we insert a new node having data 11, our updated tree becomes

unbalanced to the right. The absolute balance factor of node 7 becomes greater than 1.



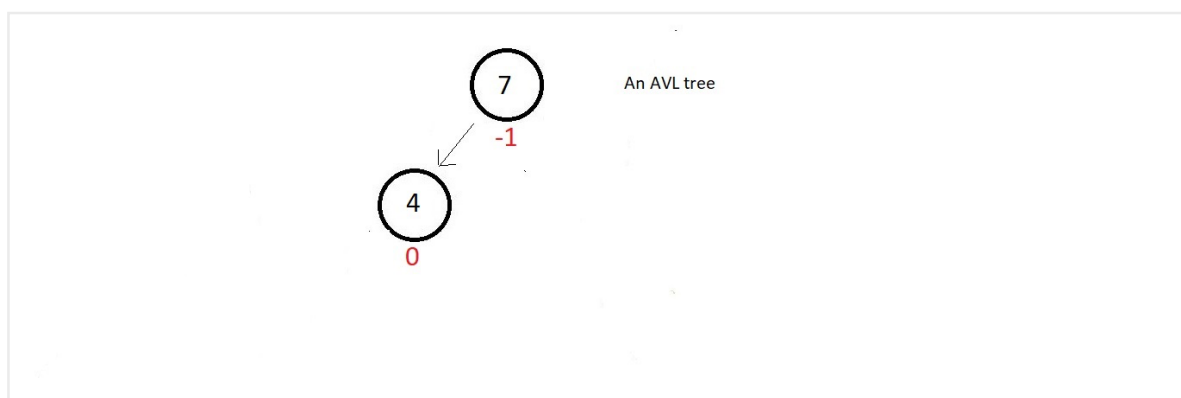
So, the method you will follow now to make this tree an AVL again is called the **RR rotation**. The name RR, just because we inserted the new element to the right subtree of the root. In this rotation technique, you just simply rotate your tree one time in an anti-clockwise direction as shown below.



So, our tree got balanced again, with a perfect balance factor at each of its nodes. Next, we have the LR rotation.

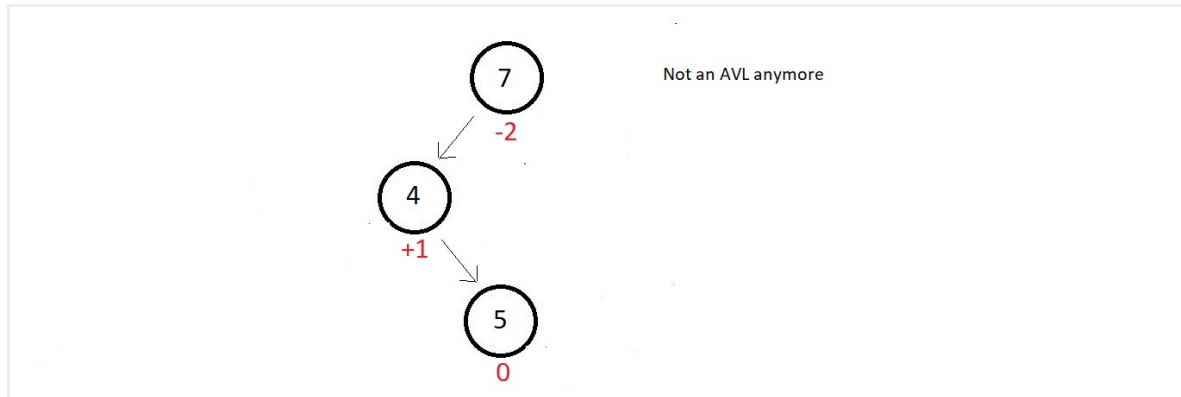
LR Rotation:

Now, suppose we have a small AVL tree having just these two nodes.



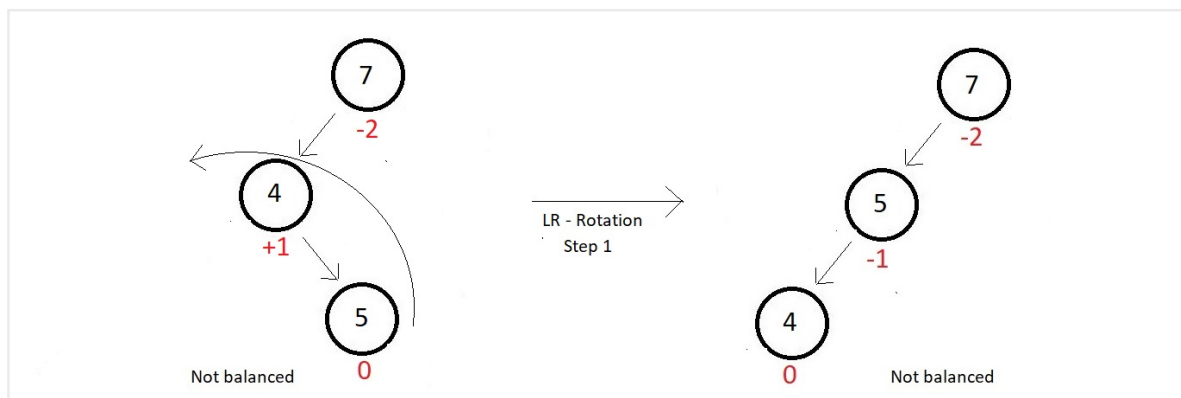
And you can see, the balance factor of both the nodes are good, but as soon as we insert a new node having data 5, our updated tree becomes unbalanced to the left. The absolute balance factor of node 7 becomes

greater than 1.

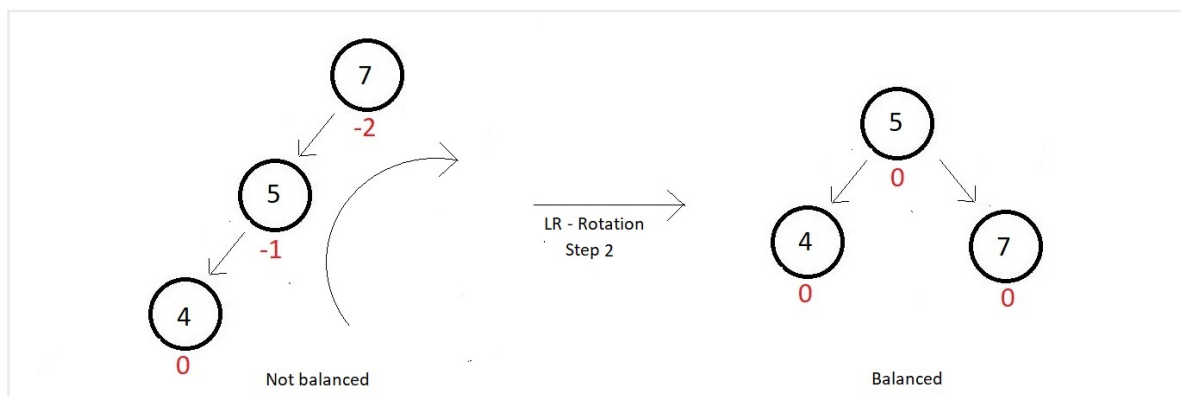


So, the method you will follow now to make this tree an AVL again is called the **LR rotation**. The name LR, just because we inserted the new element to the right to the left subtree of the root. In this rotation technique, there is a subtle complexity, which says, first rotate the left subtree in the anticlockwise direction, and then the whole tree in the clockwise direction. Follow the two steps illustrated below:

Step 1:



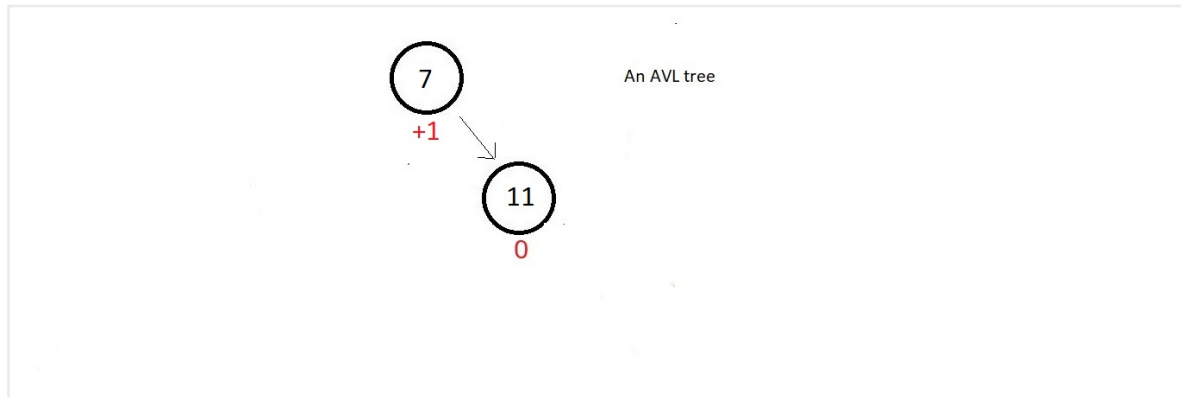
Step 2:



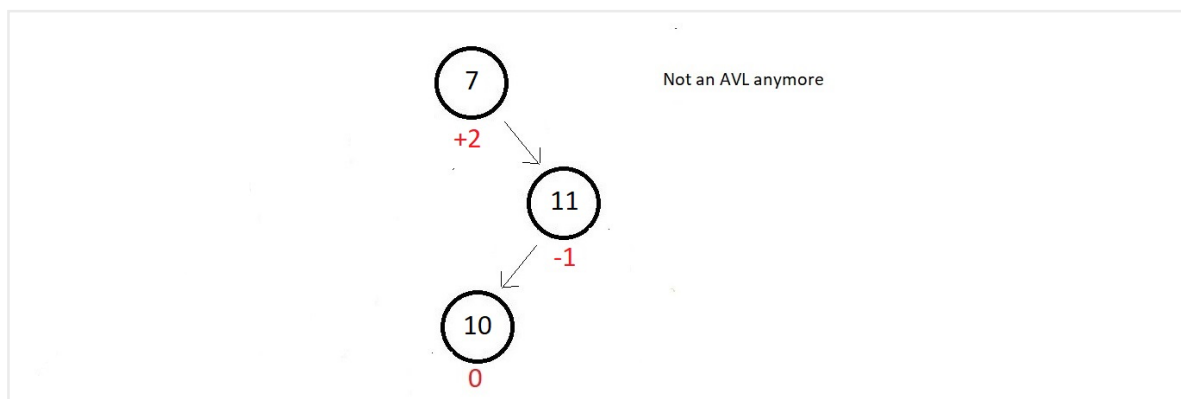
So, our tree got balanced again, with a perfect balance factor at each of its nodes. Although it was a bit clumsy, it was achievable. Next, we have the RL rotation.

RL Rotation:

Now, suppose we have a small AVL tree having just these two nodes.

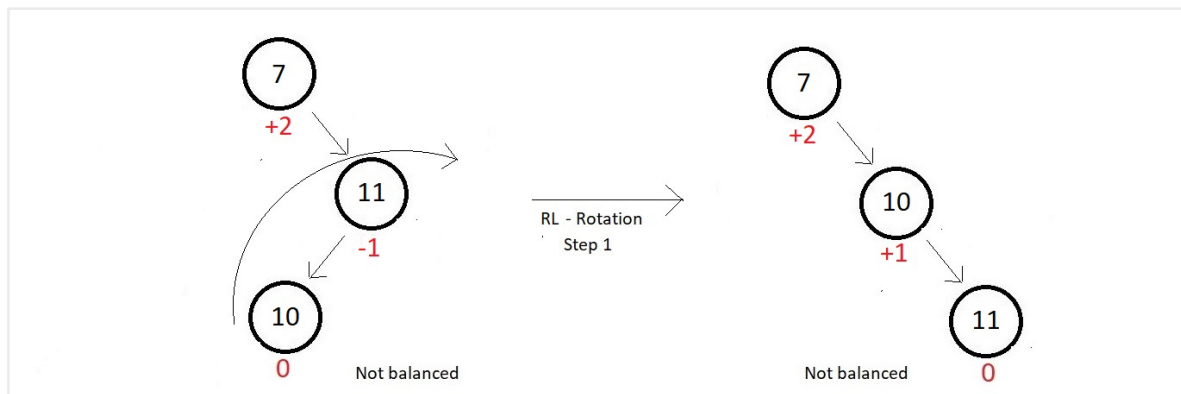


And you can see, the balance factor of both the nodes are good, but as soon as we insert a new node having data 10, our updated tree becomes unbalanced to the right. The absolute balance factor of node 7 becomes greater than 1.

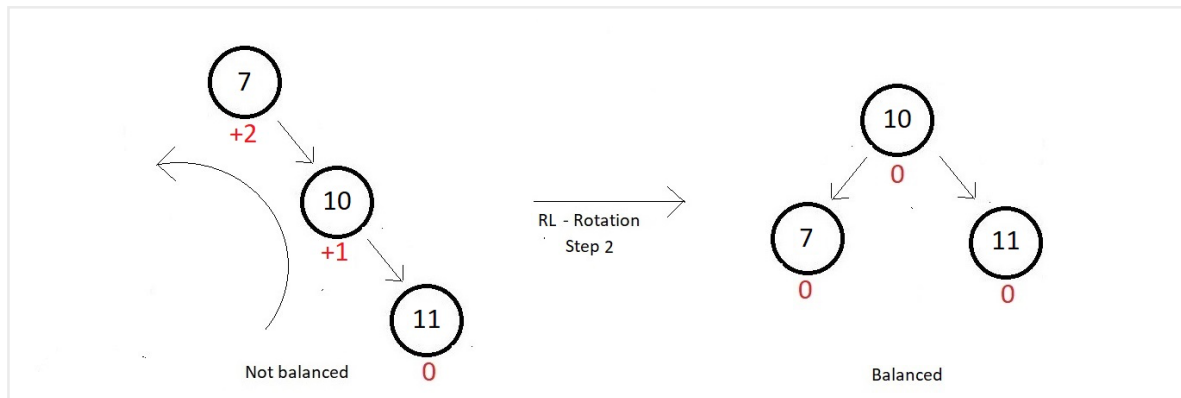


So, the method you will follow now to make this tree an AVL again is called the **RL rotation**. The name RL, just because we inserted the new element to the left to the right subtree of the root. We follow the same technique we used above, which says, first rotate the right subtree in the clockwise direction, and then the whole tree in the anticlockwise direction. Follow the two steps illustrated below:

Step 1:



Step 2:



So, our tree got balanced again, with a perfect balance factor at each of its nodes. And those were our different types of rotation techniques that helped gain the balance of our AVL trees back after the insertion of new elements.

You all might be wondering what would have happened if the trees we selected to demonstrate the types of rotation had been complex. Well, to understand those complex problems, knowing the basics was important. That's why we choose the smallest possible AVL trees. No doubt, we'll follow those complexities in our next video. Stay tuned.

AVL Trees - LL LR RL and RR rotations

In the last lecture, we saw why AVL trees are important and how rotations in an AVL tree balances it back after an unbalanced insertion. We learned all the different types of rotations we have in AVL trees. We practiced them on the smallest possible AVL tree, but today, we'll push ourselves further with some complex AVL trees and complex situations that arise after insertion. So, before we proceed with the examples of the rotations we learned, we should know how a binary search tree that is unbalanced can be balanced with a few Rotate Operations.

Rotate Operations:

We can perform Rotate operations to balance a binary search tree such that the newly formed tree satisfies all the properties of a binary search tree.

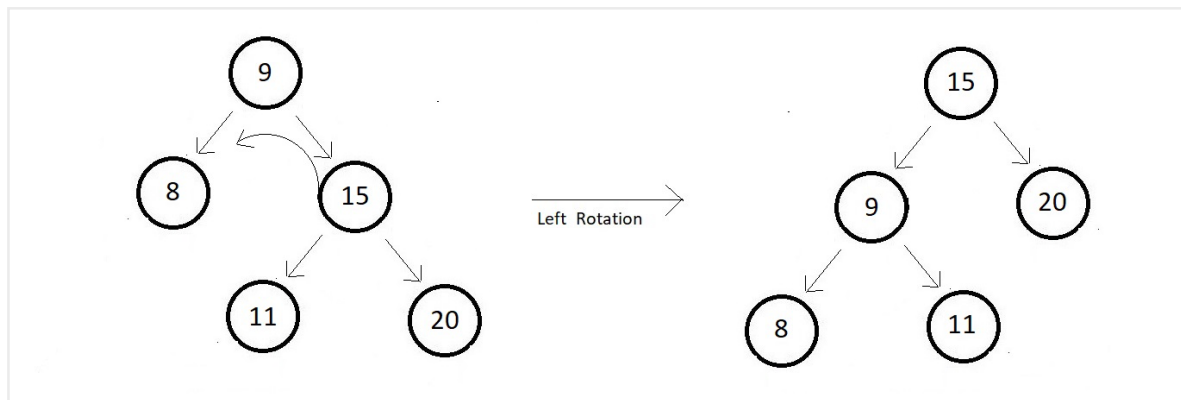
Following are the two basic Rotate operations:

1. Left Rotate Operations.
2. Right Rotate Operation

Left Rotate Operations:

In this Rotate Operation, we move our unbalanced node to the left.

Consider a binary search tree given below, and the newly formed tree after its left rotation with respect to the root.

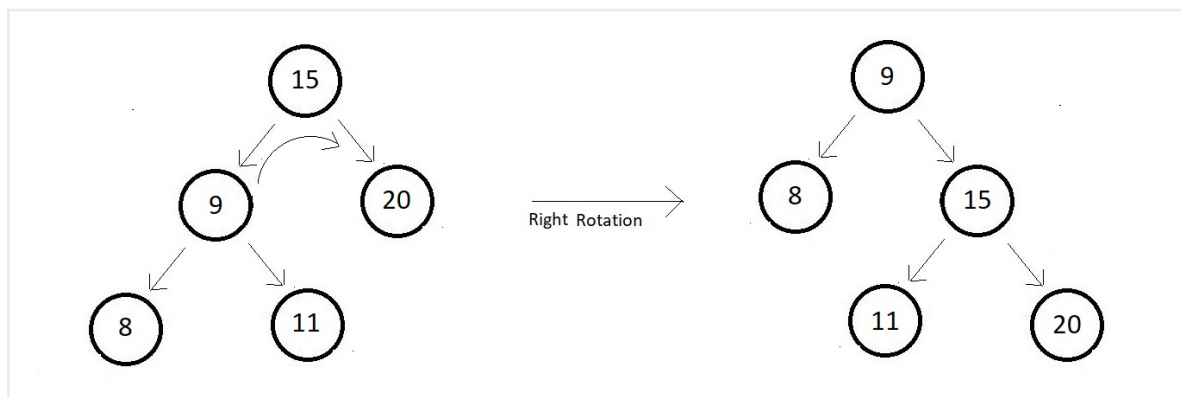


One thing to observe here is that node 11 had to change its parent after the rotation to be able to maintain the balance of the tree.

Right Rotate Operations:

In this Rotate Operation, we move our unbalanced node to the right.

Consider a binary search tree given below, and the newly formed tree after its right rotation with respect to the root.



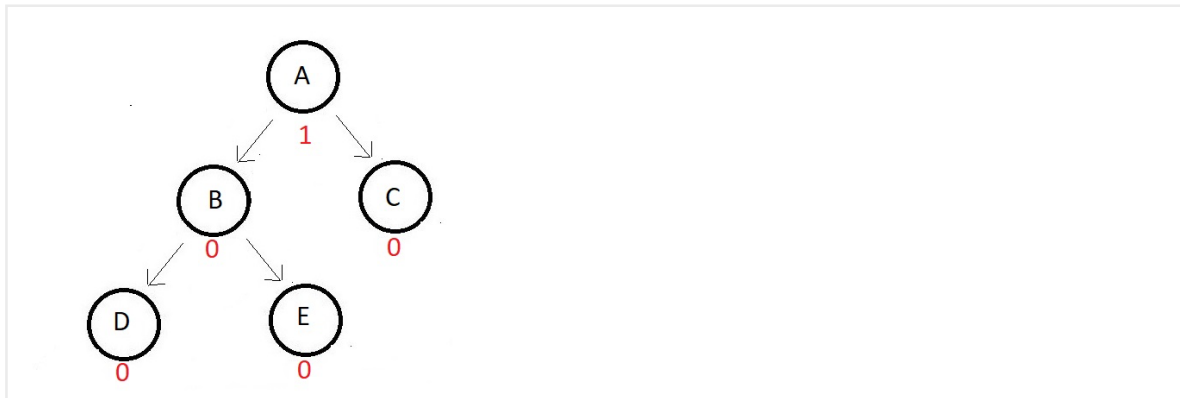
Again, as you can see that node 11 had to change its parent after the rotation to be able to maintain the balance of the tree. And rotating a tree to its left, and then again to the right, yields the same original tree as you can see from the above two examples.

Let's move back to the balancing of the AVL tree after insertion. So, when it comes to complex trees, in order to balance an AVL tree after insertion, we can follow the below-mentioned rules:

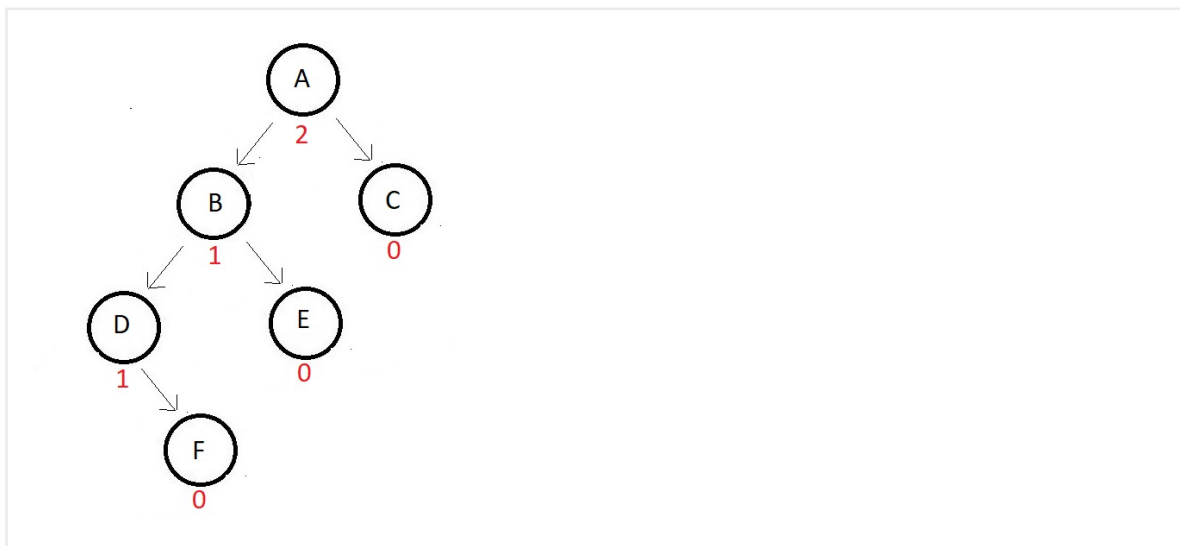
1. For Left-Left insertion - Right rotate once with respect to the first imbalance node.
2. For Right-Right insertion - Left rotate once with respect to the first imbalance node.
3. For Left-Right insertion - Left rotate once and then Right rotate once.
4. For Right-Left insertion - Right rotate once and then Left rotate once.

We'll now see how a complex tree gets balanced again after an insertion.

Consider the binary search AVL tree below:

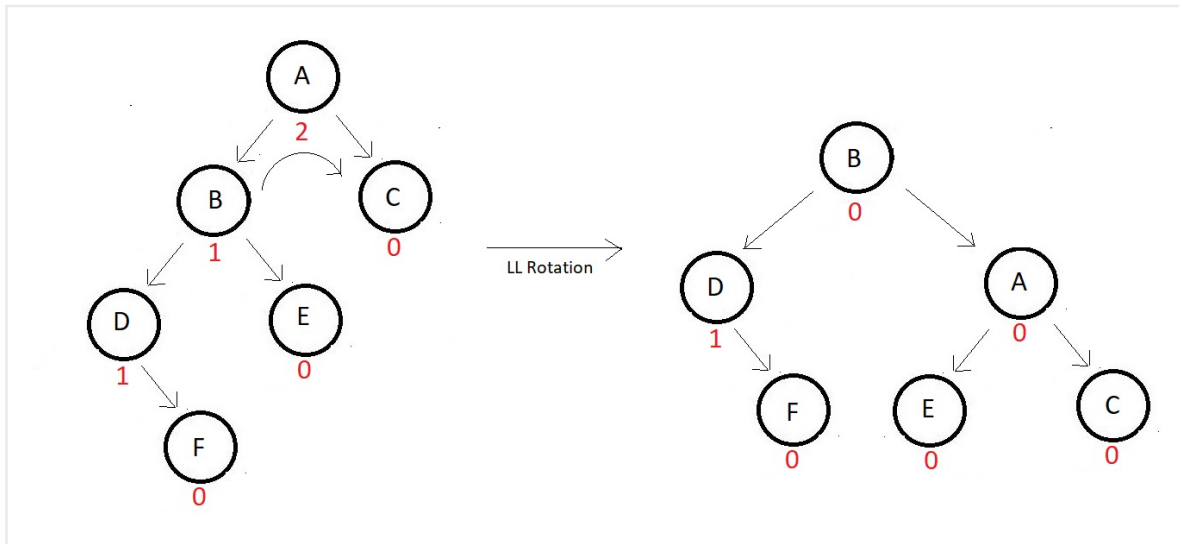


The absolute balance factor of each node is written beside, and you can see how balanced the values are. Now suppose we need to insert an element that gets its position to the right of node D. Now the updated tree looks something like this.



And the tree got imbalanced. Now, you follow these steps.

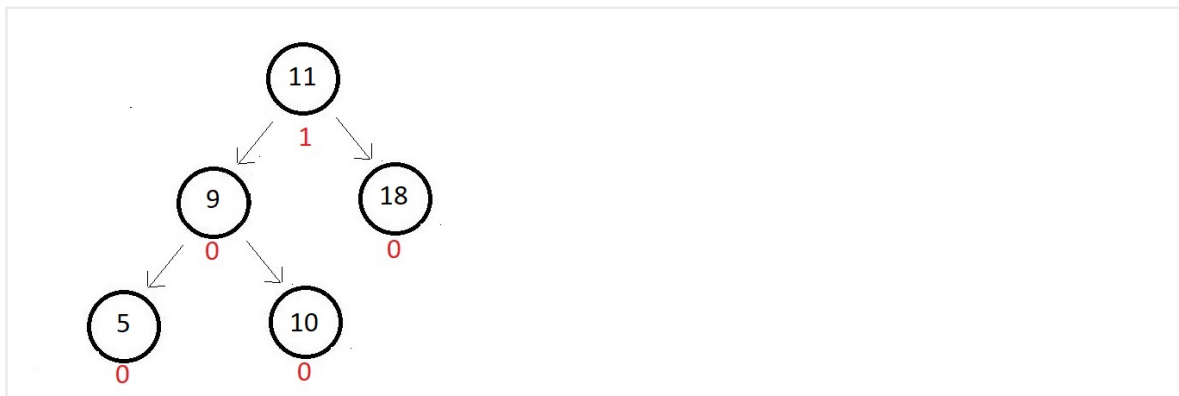
1. The first thing you would do is search for the node which got imbalanced first. We start iterating from the node we inserted at and move upwards looking for that first imbalance node. Here node A is the one we were searching for.
2. Second, you see what type of insertion was this with respect to the node we found. Here, the insertion happened to the left to the left of node A. So, this belongs to the first rule we saw above.
3. Do what the rule says. Here the rule says to right rotate once with respect to the first imbalance node. So, the tree after rotating to the right becomes:



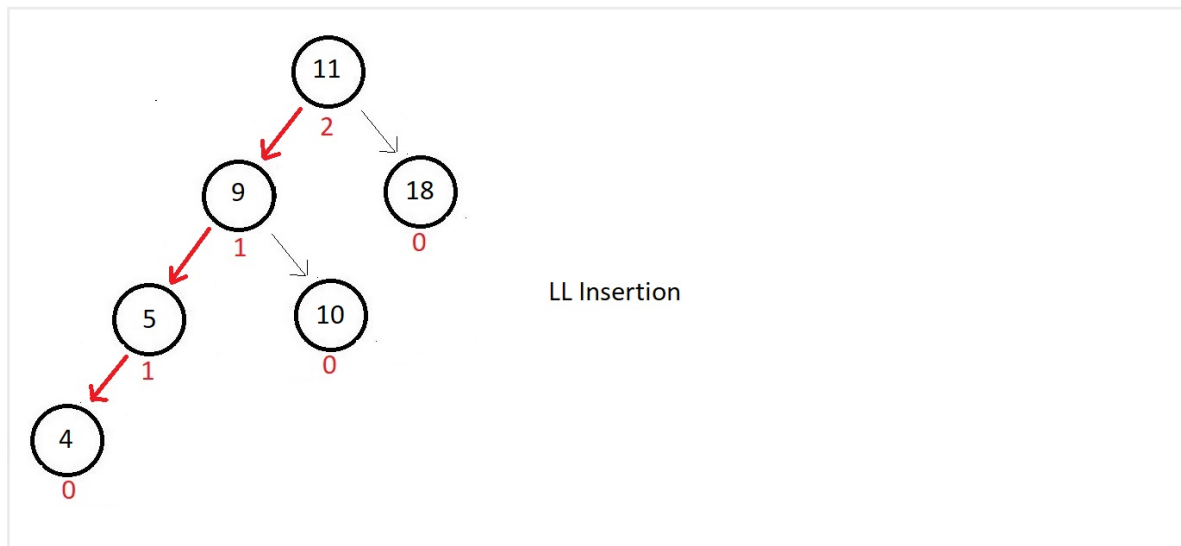
And similarly, had this been a type of right-right insertion, we would have first searched the first imbalanced node, and then would have rotated left with respect to it. But since this was just a demonstration, we would deal with two out of these four in detail now with actual numbers. Due to the similarity between the first and second rotations, we will take the LL rotation first and then one of the LR or RL rotations second.

LL Rotation:

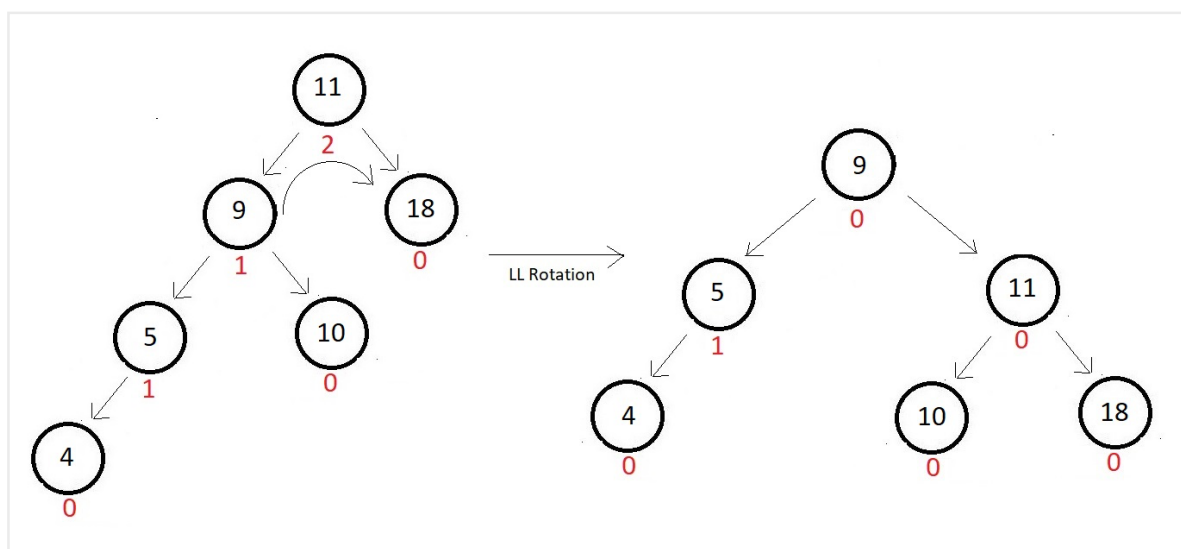
LL rotations were discussed already in the last lecture. We would just use it to balance a relatively complex AVL tree. Consider the one given below.



Absolute balance factors of each of the nodes are written beside. The tree is balanced and good for now. But now, we want to insert a node with data 4. So, that would get inserted to the left to node 5. The updated tree and their balance factors are:



And since this is a case of left-left insertion, we would rotate right once with respect to the root node, since that's the first one to get imbalanced. And in that process, we might lose the position of node 10. So, we give it a new position to the left of node 11 to accommodate it again into the tree. And our tree gets balanced again.

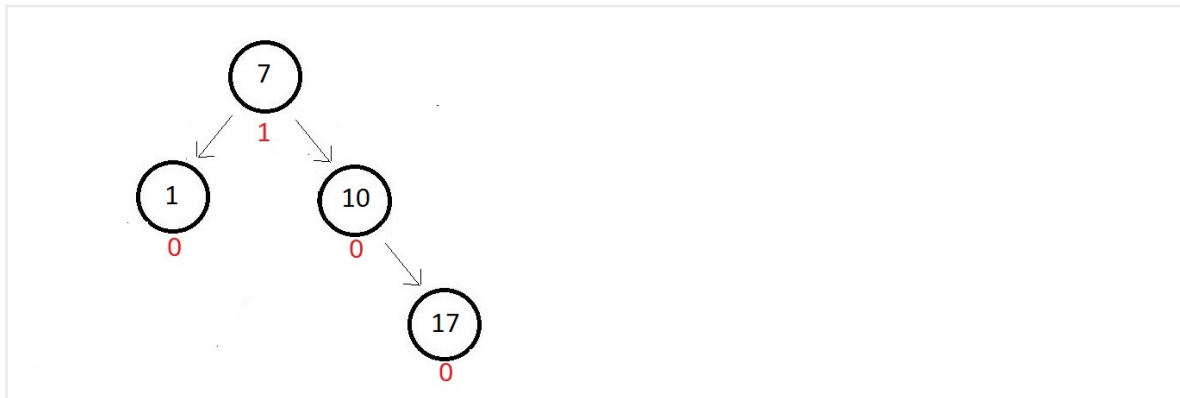


This is just a coincidence that our root node is the one we are rotating with respect to. We could come across examples where the first imbalanced node is not the root. So, we would rotate with respect to the one we'll find first, not the root.

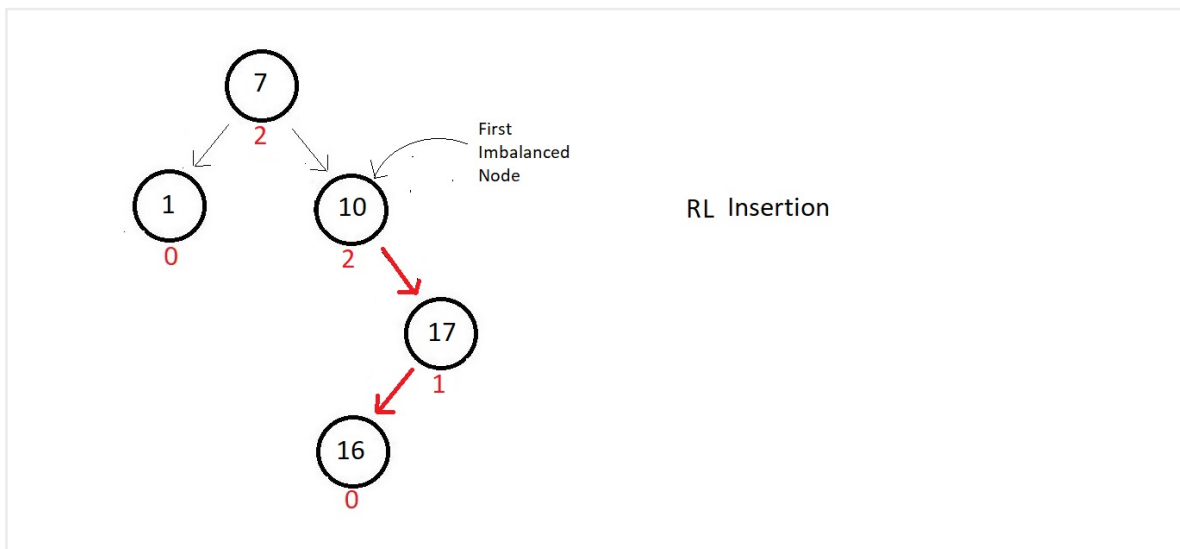
Let's now deal with one of the RL insertion cases. LR would be more or less the same, so, we'll just ignore that.

RL Rotation:

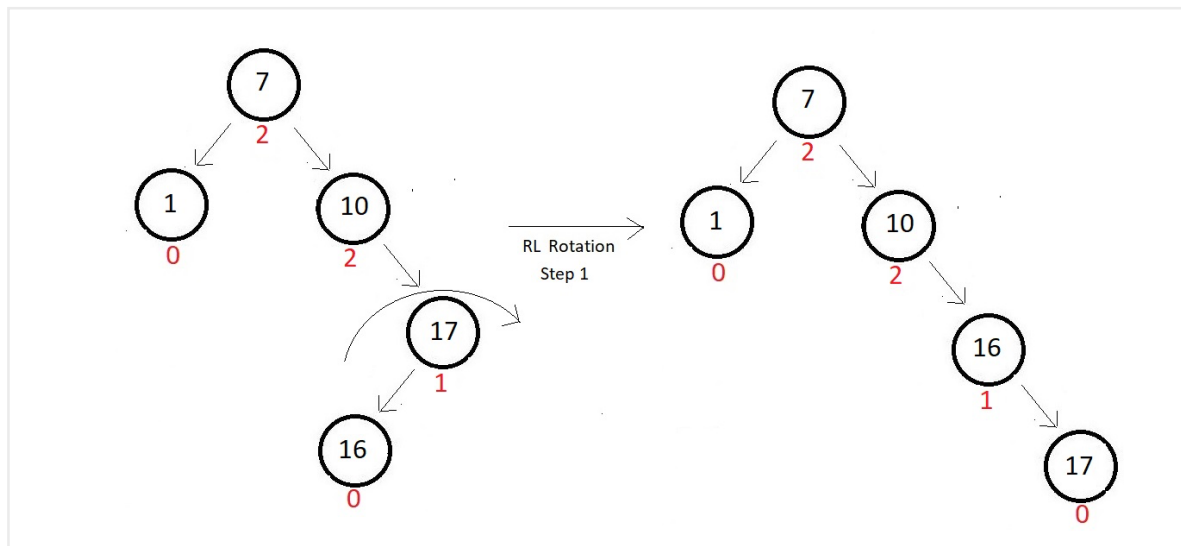
RL rotations were also discussed in the last lecture. We would just use it to balance a relatively complex AVL tree. Consider the one given below.



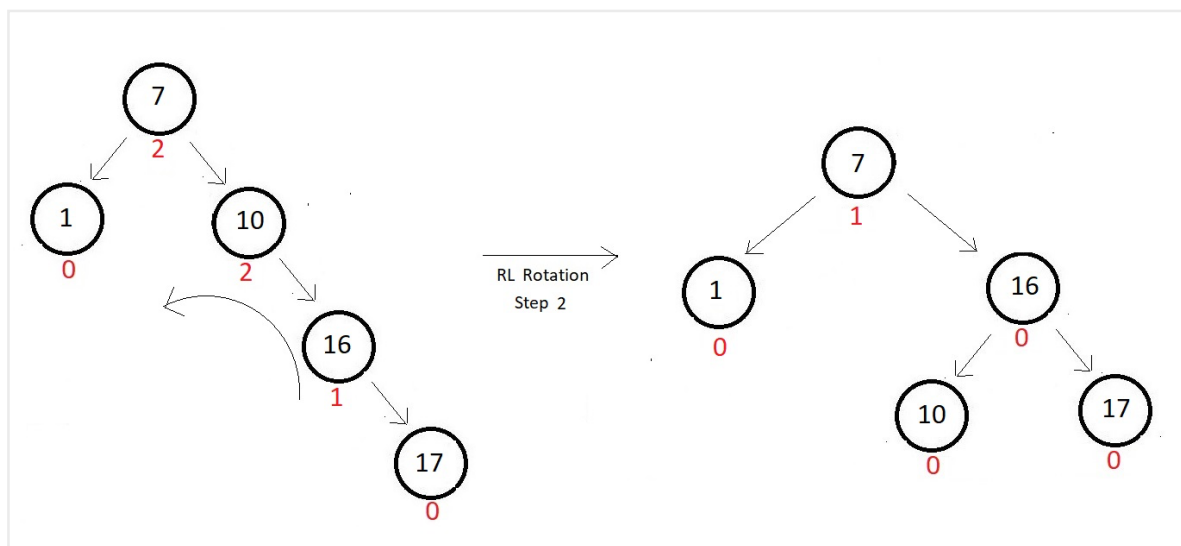
The absolute balance factors of each of the nodes are written beside. The tree is balanced and good for now. But now, we want to insert a node with data 16. So, that would get inserted to the left of node 17. The updated tree and their balance factors are:



And since this is a case of right-left insertion with respect to the first imbalanced node which is node 10, we would first rotate right once with respect to the child of the first imbalanced node which comes into the path of the insertion node. Follow the figure below.



And now, we rotate left with respect to the node we found first imbalanced, here 10. And this would do our job. Our tree gets balanced again.



So, that was what we had for today. One important thing you shouldn't miss while balancing trees. Root nodes have no special significance while you are balancing your trees. It is just about the node you find imbalanced first. It could be the root node sometimes, but it couldn't be either. So, just be careful about that. Practice on your own. Balance your trees yourselves. Hustle on.

I appreciate your support throughout.

C Code For AVL Tree Insertion & Rotation (LL, RR, LR & RL Rotation)

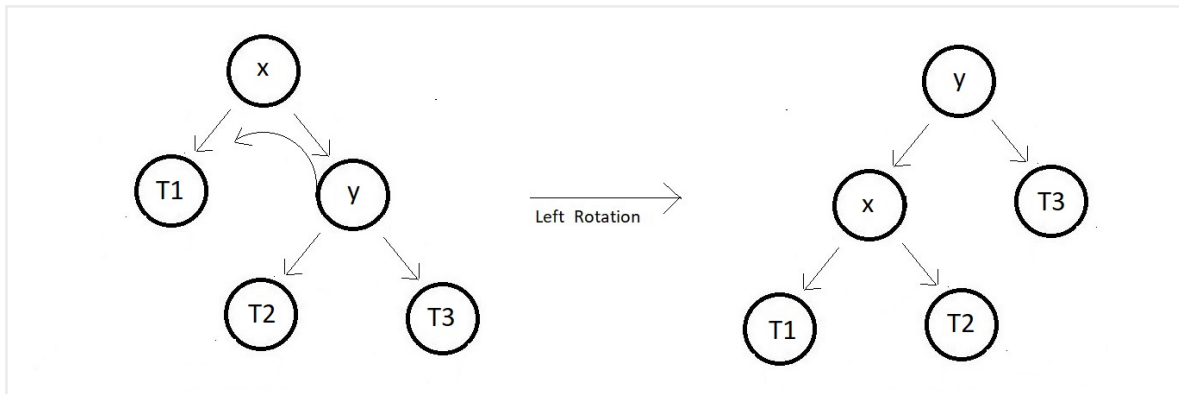
In the last lecture, we saw some good examples and complex ones to fix violations that arise after insertion operation using different rotation techniques. We learned using all the different types of strategies we use in AVL trees to balance an unbalanced tree caused after the four types of rotation situations. Today, we'll see the implementation of the same in C language.

So, before we proceed with the programming implementation of the things we learned, let's first give ourselves a very quick revision of a few things.

The two basic Rotate operations were:

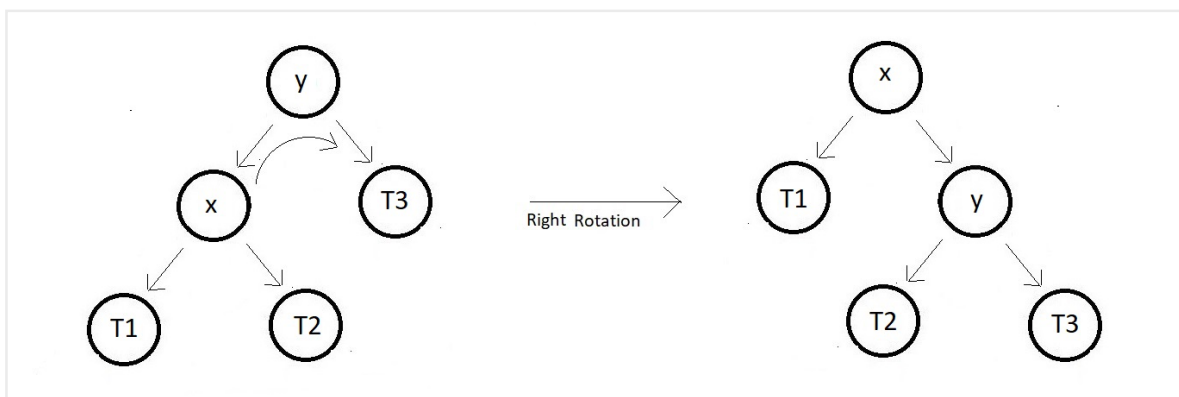
1. Left Rotate Operations.

Here, we move our unbalanced node to the left. Follow the example below.



2. Right Rotate Operation

Here, we move our unbalanced node to the right. Follow the example below.



We'll use the same tree we have illustrated above to guide us through our implementation of left and right rotation functions. We have already learnt in the last lecture the situations we had to use these two rotations techniques in. When you study the code, keep these illustrations in mind. So, let's just move on to our editors without any further ado. I have attached the source code below. Follow it as we proceed.

Understanding the source code below:

1. The first thing would be to create a struct Node which we earlier used to create a node in a tree. So, create a struct Node. Inside the structure, we would have four embedded elements, first is an integer variable *data* to store the data of the node, second is a struct Node pointer called *left* to store the address of the left child node, and third is again a struct Node pointer called *right* to store the address of the right child node. Fourth is an integer variable storing the height of that node for the ease of calculating the absolute balance factor. Absolute balance factor, if you remember, should not exceed 1, otherwise, that node becomes imbalanced.

```
struct Node
{
```

```

int key;
struct Node *left;
struct Node *right;
int height;
};

```

Copy

Code Snippet 1: Creating the struct Node

Creating the getHeight function:

2. Create an integer *getHeight*, and pass the pointer to the struct node you want to get the height of as the only parameter. Check if the struct node pointer is not NULL, return 0 if it is NULL, otherwise return the height element of the struct node. And that would be it.

```

int getHeight(struct Node *n){
    if(n==NULL)
        return 0;
    return n->height;
}

```

Copy

Code Snippet 2: Creating the getHeight function

Creating the createNode function:

3. Create a struct Node* function *createNode*, and pass the data/key for the node as the only parameter. Create a struct Node pointer *node*. Reserve a memory in the heap for the *node* using malloc. Make the left and the right element of the struct *node* point to NULL, and fill the data in the data element. Now, set the height element of this *node* as 1, because any new node would be a leaf node that has a height of 1. And now, return the pointer *node* you created. This would simply create a node as a part of the tree.

```

struct Node *createNode(int key){
    struct Node* node = (struct Node *) malloc(sizeof(struct Node));
    node->key = key;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    return node;
}

```

Copy

Code Snippet 3: Creating the createNode function

Here come the three most important functions of all when we implement the AVL tree. First is the *getBalanceFactor*, the second is the *leftRotate* and the third is the *rightRotate*.

Creating the getBalanceFactor function:

4. Create a struct Node* function *getBalanceFactor*, and pass the pointer to the struct node you want to get the balance factor of as the only parameter.

Now the balance factor of a node is just the value we receive after subtracting the height of the right child with that of the left child of the node. So, first check whether the node is not NULL, if it is, simply returns 0. Otherwise, return the value received after subtracting the height of the right child with that of the left child. This was a utility function to find the balance factor for a node.

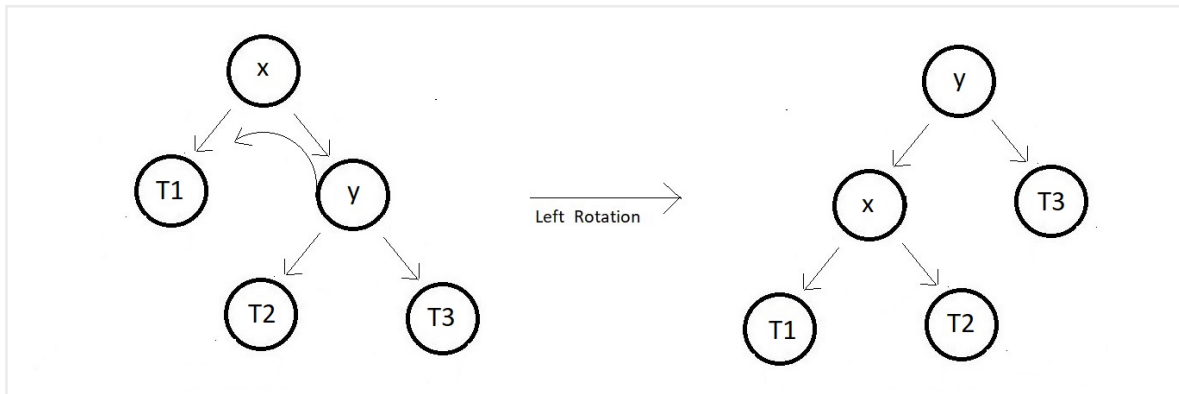
```
int getBalanceFactor(struct Node * n){
    if(n==NULL){
        return 0;
    }
    return getHeight(n->left) - getHeight(n->right);
}
```

Copy

Code Snippet 4: Creating the getBalanceFactor function

Follow the illustrations below while we write the left and the right rotate functions respectively.

Creating the leftRotate function:



5. Create a struct Node* function leftRotate, and pass the pointer to the imbalanced struct node you want to left rotate as the only parameter. Consider this node x as represented in the figure above. As you could see, the left child of x is y. So, define a struct node pointer variable y and initialize it with `x->right`. As seen in the figure, we would make x the left child of x, and T2 the right child of x. So, we would also store `y->left` in another struct node pointer variable T2.

Now, simply assign x to the left element of y, and T2 to the right element of x. And this would finish our left rotation function. We must, however, remember to change the new heights of the nodes that have been shifted, which are x and y. Height of x becomes one added to the maximum of the heights of both of its children. Similarly, the height of y becomes one added to the maximum of the heights of both of its children. And now simply return y, since y becomes the new root for the subtree. Don't forget to explicitly define a max function before using it.

```
struct Node* leftRotate(struct Node* x){
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;
```

```

x->right = T2;

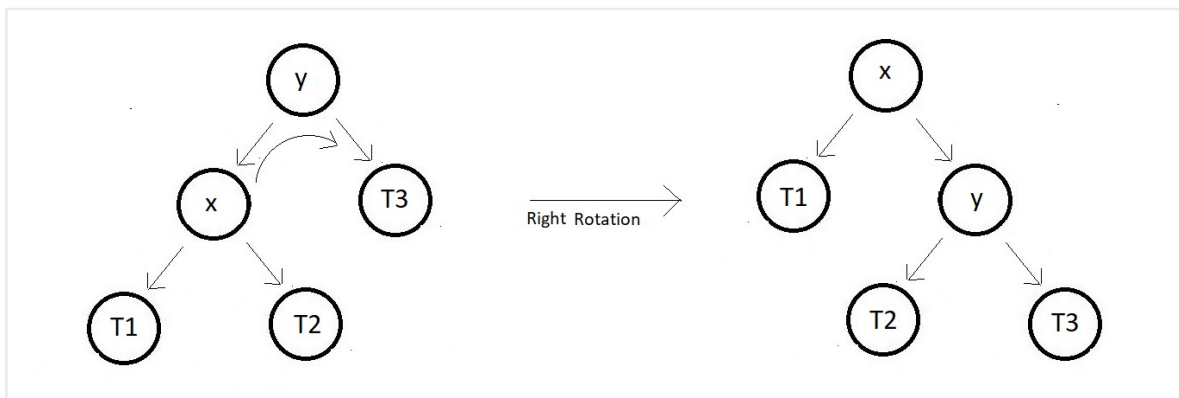
x->height = max(getHeight(x->right), getHeight(x->left)) + 1;
y->height = max(getHeight(y->right), getHeight(y->left)) + 1;

return y;
}
Copy

```

Code Snippet 5: Creating the leftRotate function

Creating the rightRotate function:



6. Create a struct Node* function rightRotate, and pass the pointer to the imbalanced struct node you want to right rotate as the only parameter. Consider this node y as represented in the figure above. As you could see, the left child of y is x. So, define a struct node pointer variable x and initialize it with `y->left`. As seen in the figure, we would make y the right child of x, and T2 the left child of y. So, we would also store `x->right` in another struct node pointer variable T2.

Now, simply assign y to the right element of x, and T2 to the left element of y. And this would finish our right rotation function. In addition, we must remember to adjust the new height for the nodes that have been shifted, which are x and y. Height of x becomes one added to the maximum of the heights of both of its children. Similarly, the height of y becomes one added to the maximum of the heights of both of its children. And now simply return x, since x becomes the new root for the subtree.

```

struct Node* rightRotate(struct Node* y){
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    x->height = max(getHeight(x->right), getHeight(x->left)) + 1;
    y->height = max(getHeight(y->right), getHeight(y->left)) + 1;
}

```

```
    return x;
}
```

Copy

Code Snippet 6: Creating the rightRotate function

Creating the insert function:

7. Create a struct Node* function *insert*, and pass the pointer to the root of the AVL tree and the data we want to insert as two of its parameters. Now, we would follow the simple Binary Search Tree insertion operation which we had already covered in our past lectures. There, we would just recursively iterate to the best-fit position where the new data should be inserted, and once found, we create a new node, and return the pointer to this node.

And since we have recursively gone down the tree, we would backtrack after inserting. While backtracking, we update the heights of all the nodes we followed to reach the apt position.

Once this new node gets inserted, the balance of the tree might get disturbed, so we would first create an integer variable *bf*, and store in it the balance factor of the current node we are on while backtracking. And if this node gets imbalanced, four of the possible cases arise.

Left-Left case:

Here, the new node would have been inserted on the left to the left child of the current node. So, if our *bf* has a value greater than 1, and the new node has data less than the data of the left child of the node itself, it is the case of left-left rotation, and hence we call the rightRotate function once to fix this disturbance.

Right-Right case:

Here, the new node would have been inserted on the right to the right child of the current node. So, if our *bf* has a value less than -1, and the new node has data greater than the data of the right child of the node itself, it is the case of right-right rotation, and hence we call the leftRotate function once to fix this disturbance.

Left-Right case:

Here, the new node would have been inserted on the right to the left child of the current node. So, if our *bf* has a value greater than 1, and the new node has data greater than the data of the left child of the node itself, it is the case of left-right rotation, and hence we call first the leftRotate function passing the left subtree and then call the rightRotate function on this updated node to fix this disturbance.

Right-Left case:

Here, the new node would have been inserted on the left to the right child of the current node. So, if our *bf* has a value less than -1, and the new node has data less than the data of the right child of the node itself, it is the case of right-left rotation, and hence we call first the rightRotate function passing the right subtree and then call the leftRotate function on this updated node to fix this disturbance.

And then finally return this node.

8. The Last step would be to fetch the preOrder traversal function from our previous programming lecture, to actually see how our tree would get all balanced after all the insertions we would do.

```
struct Node *insert(struct Node* node, int key){
    if (node == NULL)
        return createNode(key);

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    node->height = 1 + max(getHeight(node->left), getHeight(node->right));
    int bf = getBalanceFactor(node);

    // Left Left Case
    if(bf>1 && key < node->left->key){
        return rightRotate(node);
    }
    // Right Right Case
    if(bf<-1 && key > node->right->key){
        return leftRotate(node);
    }
    // Left Right Case
    if(bf>1 && key > node->left->key){
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    // Right Left Case
    if(bf<-1 && key < node->right->key){
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}
```

Copy

Code Snippet 7: Creating the insert function

Here is the whole source code:

```
#include <stdio.h>
#include <stdlib.h>

struct Node
{
    int key;
    struct Node *left;
    struct Node *right;
```



```

    int height;
};

int getHeight(struct Node *n){
    if(n==NULL)
        return 0;
    return n->height;
}

struct Node *createNode(int key){
    struct Node* node = (struct Node *) malloc(sizeof(struct Node));
    node->key = key;
    node->left = NULL;
    node->right = NULL;
    node->height = 1;
    return node;
}

int max (int a, int b){
    return (a>b)?a:b;
}

int getBalanceFactor(struct Node * n){
    if(n==NULL){
        return 0;
    }
    return getHeight(n->left) - getHeight(n->right);
}

struct Node* rightRotate(struct Node* y){
    struct Node* x = y->left;
    struct Node* T2 = x->right;

    x->right = y;
    y->left = T2;

    x->height = max(getHeight(x->right), getHeight(x->left)) + 1;
    y->height = max(getHeight(y->right), getHeight(y->left)) + 1;

    return x;
}

struct Node* leftRotate(struct Node* x){
    struct Node* y = x->right;
    struct Node* T2 = y->left;

    y->left = x;

```

```

x->right = T2;

x->height = max(getHeight(x->right), getHeight(x->left)) + 1;
y->height = max(getHeight(y->right), getHeight(y->left)) + 1;

return y;
}

struct Node *insert(struct Node* node, int key){
    if (node == NULL)
        return createNode(key);

    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    node->height = 1 + max(getHeight(node->left), getHeight(node->right));
    int bf = getBalanceFactor(node);

    // Left Left Case
    if(bf>1 && key < node->left->key){
        return rightRotate(node);
    }
    // Right Right Case
    if(bf<-1 && key > node->right->key){
        return leftRotate(node);
    }
    // Left Right Case
    if(bf>1 && key > node->left->key){
        node->left = leftRotate(node->left);
        return rightRotate(node);
    }
    // Right Left Case
    if(bf<-1 && key < node->right->key){
        node->right = rightRotate(node->right);
        return leftRotate(node);
    }
    return node;
}

void preOrder(struct Node *root)
{
    if(root != NULL)
    {
        printf("%d ", root->key);
        preOrder(root->left);
    }
}

```

```

        preOrder(root->right);
    }
}

int main(){
    struct Node * root = NULL;

    root = insert(root, 1);
    root = insert(root, 2);
    root = insert(root, 4);
    root = insert(root, 5);
    root = insert(root, 6);
    root = insert(root, 3);
    preOrder(root);
    return 0;
}
Copy

```

Code Snippet 8: Implementing AVL tree and its insertion operation

And now having finished all the functions we needed, we can create an AVL tree in main and verify if all these functions actually work. So, simply create a struct node pointer root and initialize it with NULL. And now we are good to insert as many nodes as we want. We would insert 6 of them. And then call the preOrder traversal on the same.

```
struct Node * root = NULL;
```

```

root = insert(root, 1);
root = insert(root, 2);
root = insert(root, 4);
root = insert(root, 5);
root = insert(root, 6);
root = insert(root, 3);
preOrder(root);

```

Copy

Code Snippet 9: Creating an AVL tree and using the insert function

The output we received was:

```
4 2 1 3 5 6
```

```
PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>
```

Copy

Figure 1: Output of the above program

So, our AVL tree is actually working all fine. And this was the implementation of the rotations we learned. I know this was something actually I would grade tough, but giving respect to the complexity of the

topic, I would recommend you all to go through these again and practice writing the code yourselves.

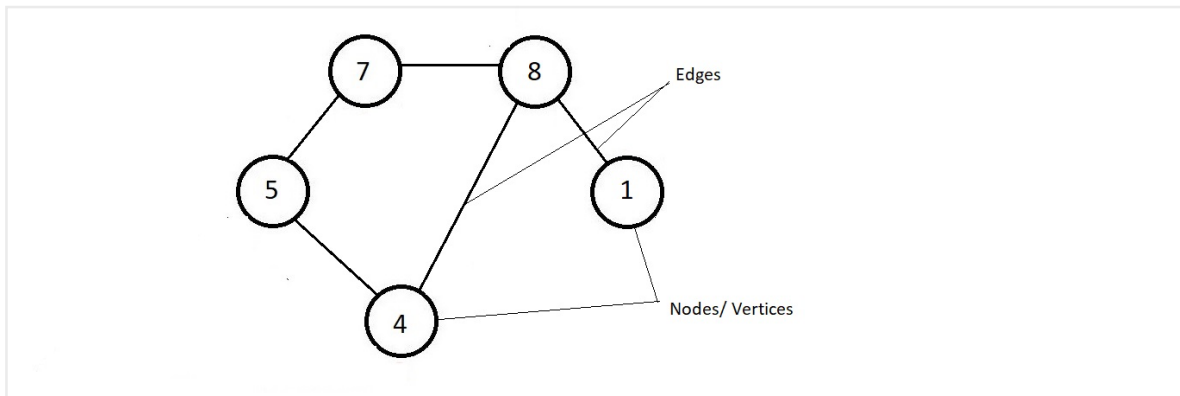
Introduction to Graphs | Graph Data Structure

As we continue on to our topic of data structures and algorithms, we'll examine one of the most important subjects of all, **graphs**. Graphs are an important data structure we should learn. Today, we'll start with the introduction of graphs, and the reasons why we should learn them.

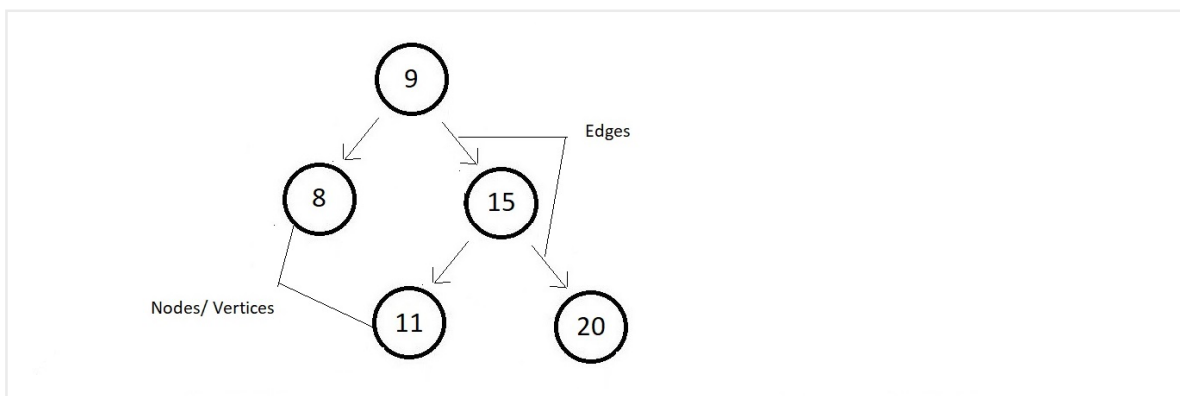
What is a graph?

All the data structures we have learned so far have either been linear data structures or nonlinear data structures. Linear data structures included arrays, linked lists, and stacks, while nonlinear hierarchical data structures included Binary Search Trees and AVL Trees. Graphs are an example of non-linear data structures. A graph is a collection of nodes connected through edges.

Example of a simple graph:



This was one of the general types of graphs we would learn. You might be surprised to learn that trees are also a type of graph. Well, there is nothing as such to be surprised about. Follow the illustration of a tree below.

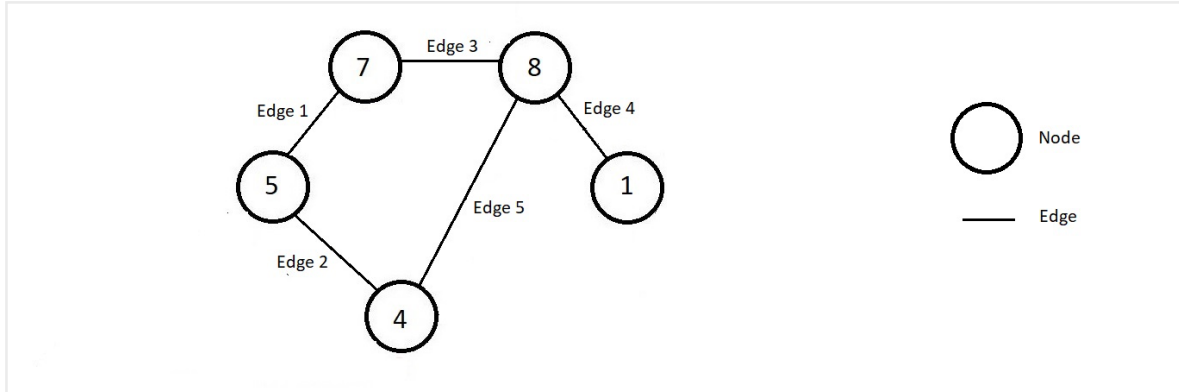


A tree is also a collection of nodes and edges, and hence is a graph only. Let's now quickly see all those elements of a graph.

Nodes/ Vertices & Edges:

A vertex or node is one fundamental unit/entity of which graphs are formed.

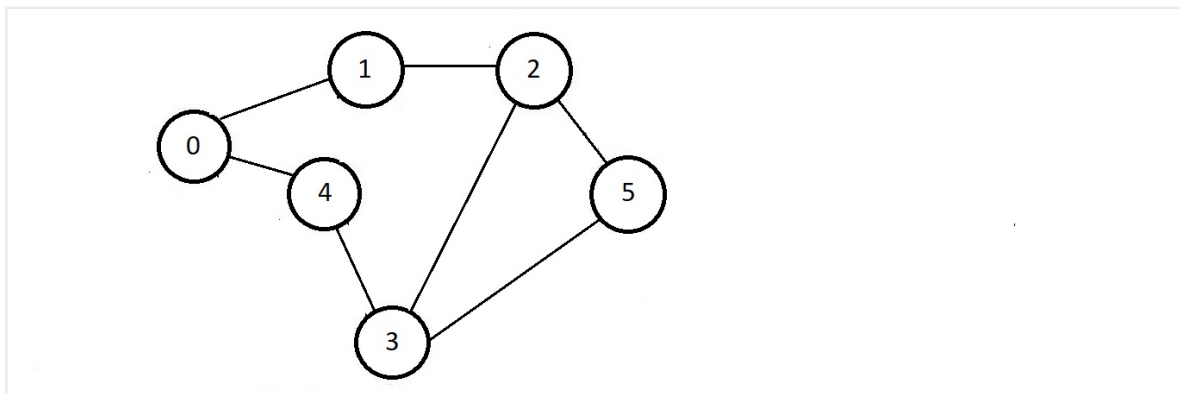
Nodes are the data containing parts, connected with each other using edges. An edge is uniquely defined by its 2 endpoints or the two nodes it is connecting. You can take the analogy of people in a network, or a widespread chain, where people could be representing the nodes and their connections in the network could be represented using edges. It is a structure.



Formal Definition:

A graph $G = (V, E)$ is technically a collection of vertices and edges connecting these vertices.

Follow the below-illustrated graph for reference:



Here, V is the set of all the vertices. Therefore $V = \{0, 1, 2, 3, 4, 5\}$, and E is the set of all edges, therefore $E = \{(0, 1), (1, 2), (0, 4), (4, 3), (3, 5), (2, 5), (2, 3)\}$. Every edge connects the pair they are represented with. Edge $(0, 1)$ connects node 0 to node 1. And hence, a graph is always represented using its set of vertices, V and its set of edges, E in the form $G = (V, E)$.

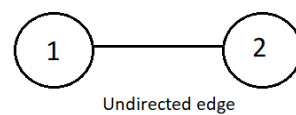
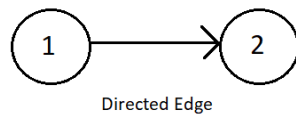
Applications of graphs:

Graphs have a wide range of applications in our world. Learning graphs also becomes a necessity for anyone pursuing software development due to its applications. Graphs are used to model paths in a city, as often seen in Google Maps. They are used to model social networks such as Facebook or LinkedIn. Graphs are also used to monitor website backlinks, internal employee networks, etc.

Types of Edges:

1. Directed Edge -A directed edge is an edge connecting two nodes but strictly defines the way it is connected from and to. Below example shows node 1 connecting to node 2, and not vice - versa. You can take the analogy of Facebook's follow feature, where if you follow someone, then it's not that the other person who followed you too. It's just one way. Another great example is that of a hyperlink, where one can even link google.com to their own websites on the internet, but then google.com would not necessarily link your website to their page :)

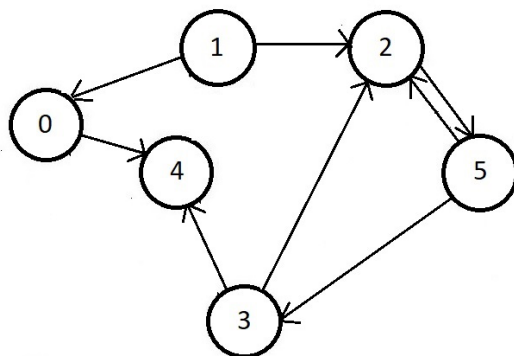
2. Undirected Edge - An undirected edge is an edge connecting two nodes from the way. Below example shows node 1 connecting to node 2, and at the same time node 2 connecting to node 1. You can take the analogy of Facebook's friend feature, where if you make someone a friend, then you too become their friend, so it's both ways.



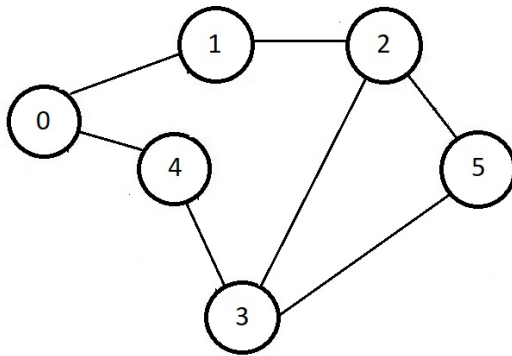
According to the type of edges that a graph has, graphs can be further divided into two types.

Types of Graphs:

1. Directed graph - Directed graphs are graphs having each of its edges directed. One of the examples of directed graphs is:



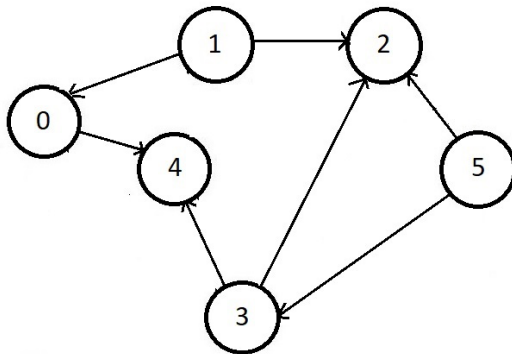
2. Undirected graph - Directed graphs are graphs having each of its edges undirected. One of the examples of directed graphs is:



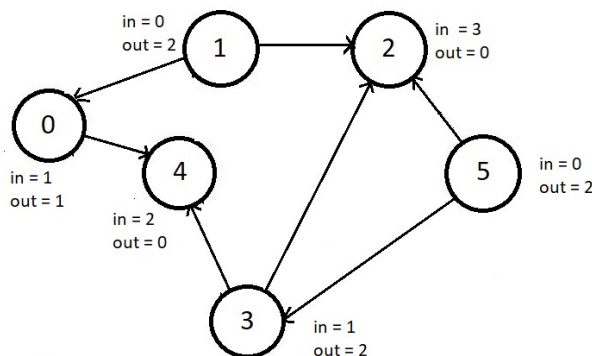
You may sometimes get to see both directed and undirected edges in the same graph, but those are rarely studied. For now, we would restrict ourselves to directed and undirected graphs only.

Indegree and Outdegree of a node:

As their name suggests, indegree of a node is the number of edges coming to the node, and outdegree of a node is the number of edges originating from that node. Consider the directed graph below:



We can write the indegree represented by *in* and outdegree represented by *out*, in the above graph for each of these nodes.



Let's talk about one real-life example of graphs - a graph of users - FACEBOOK!

- Although the users using Facebook need not understand the graph theory, once you create your profile there, Facebook uses graphs to

model relationships between nodes.

- We can apply graph algorithms to suggest friends to people, calculate the number of mutual friends, etc.
Suppose you have friends X, Y, and Z on Facebook. And you are one common friend to all of the three. Now, Facebook observes the connections and would suggest your friends to get connected with each other seeing your connection with them. And you would be shown as one mutual friend to all three of them, and this is the concept behind the mutual friends and friends suggestions system.
- Other examples of graphs include the result of a web crawl for a website or for the entire world wide web, city routes as seen on Google Maps, etc. Furthermore, different search engines have different web network models.

So this was the introductory lecture on Graph Theory. We'll be seeing modelling these graphs and their different algorithms in the upcoming lectures. You just stay tuned.

Thank you for being with me throughout the session.

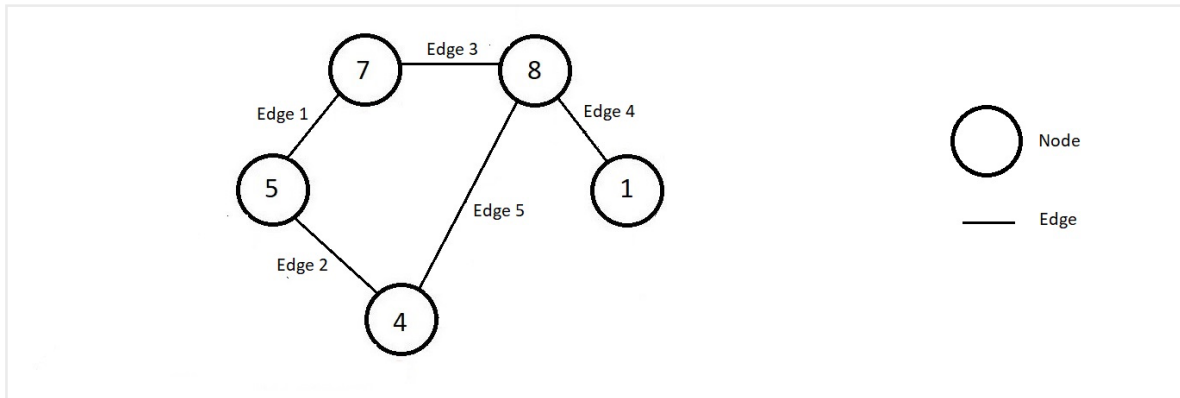
Representation of Graphs - Adjacency List, Adjacency Matrix & Other Representations

In the last lecture, we introduced to you our new topic, graphs. We saw the applications of graphs and discussed their elements including their nodes/ vertices and edges. Moving ahead with our discussion on graphs, today we'll learn how graphs are represented, and various terminologies related to graphs.

Representation of graphs is actually a very important concept and once you understand how graphs are represented, learning other graph algorithms becomes much easier. The first lecture detailed the following points:

1. Graphs and their nodes & edges, directed and undirected edges and graphs.
2. Applications including their use to model real-world problems like managing a social network, website links, etc.
3. Another major application is their use to solve problems like is there a path between two locations on a map and if there is, which one is the shortest.

A simple undirected graph looks like this:



Let's now move to see how we represent graphs in various ways.

Ways to represent a graph:

Any representation should basically be able to store the nodes of a graph and their connections between them. And this can be accomplished in so many ways but primarily the most used way to represent a graph is,

1. **Adjacency List** - Mark the nodes with their neighbours
2. **Adjacency Matrix** - $A_{ij} = 1$, if there is an edge between i and j , 0 otherwise.

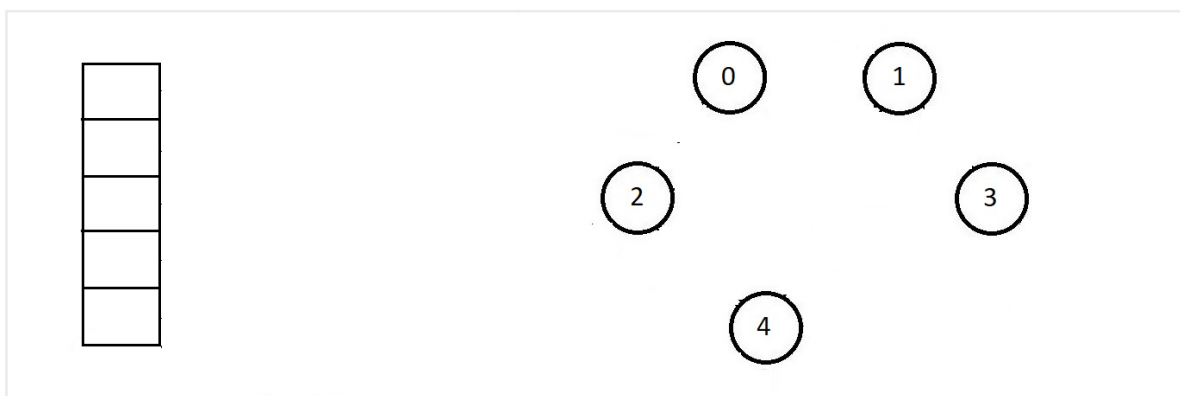
We'll see the above two in detail. Meanwhile, other representations include:

1. **Edge Set** - Store the pair of nodes/vertices connected with an edge.
Example: $\{(0, 1), (0, 4), (1, 4)\}$.
2. Other implementations to represent a graph also exist. For example, Compact list representation, cost adjacency list, cost adjacency matrix, etc.

Let's now deal with each of these individually.

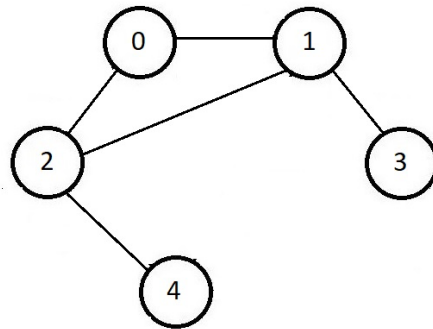
Adjacency List:

In this method of representation of graphs, we store the nodes, along with the list of their neighbors. Basically, we maintain a list of all the nodes, and along with it, we store the list of nodes a particular node is connected with. Consider the nodes I've illustrated below.

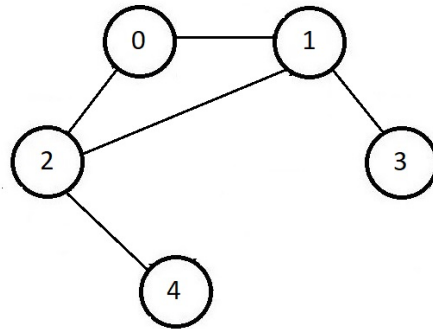
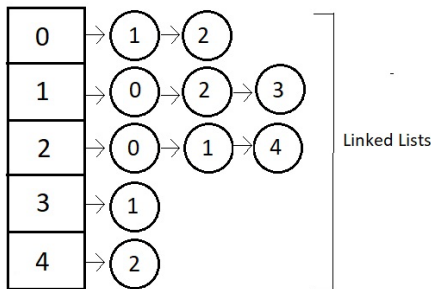


So first, we store the nodes we have in the graph.

0
1
2
3
4



Next, we look for the connections of each of these nodes we stored. Starting with 0, you can see that 0 is connected with both 1 and 2. So, we will store that beside node 0. Next, 1 is connected with all 0, 2, and 3. 2 is connected with both 0 and 4, and 3 is connected with only 1, and 4 is connected with only 2. So, this information gets stored as shown below.



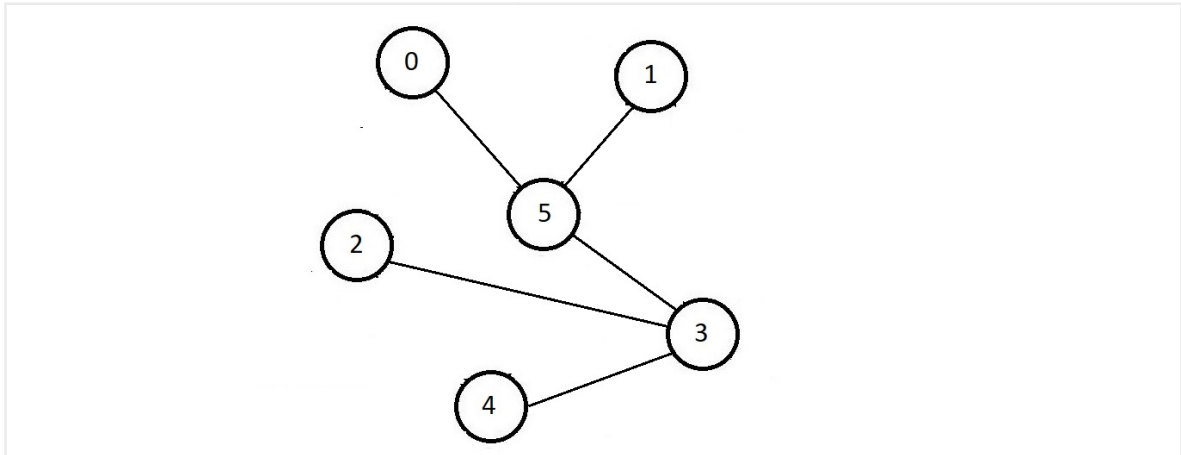
And the information about the connections of each of the nodes gets stored in separate linked lists as shown in the figure above. And each of the nodes itself acts as a pointer stored in an array pointing to the head of each of the linked lists. So, in this case, we would have an array of length 5 where the first index stores a pointer to the head of the adjacency linked list of the node 0.

And this was the adjacency list representation of graphs, and I can say that this is one of the most used representation methods. Next is the adjacency matrix.

Adjacency Matrix:

Adjacency matrix is another method of representation of graphs, where we represent our graph in the form of a matrix where cells are either filled with 0 or 1. Let's call the cell falling on the intersection of i th row and j th column be A_{ij} , then the cell would be filled with 1 if there is an edge between node i and j , otherwise, the cell would be filled with a 0.

Consider the graph illustrated below:



Now, we'll make a 6x6 matrix as follows:

	0	1	2	3	4	5
0						
1						
2						
3						
4						
5						

Now, we'll iterate through each of the cells, and see if there is an edge between the row number and the column number or not. If there is an edge, we'll place 1 in that cell, otherwise a zero. Since there are no self-loops even, we'll mark 0 to the cell having $i=j$. The filled adjacency matrix for the above graph would be:

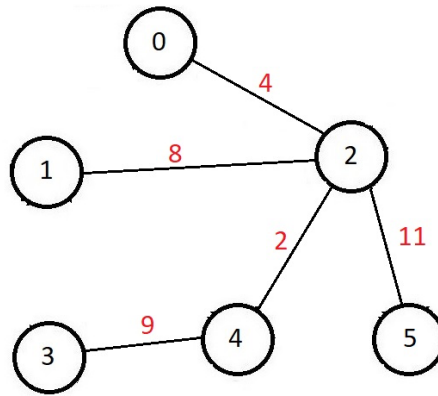
	0	1	2	3	4	5
0	0	0	0	0	0	1
1	0	0	0	0	0	1
2	0	0	0	1	1	0
3	0	0	1	0	1	1
4	0	0	0	1	0	0
5	1	1	0	1	0	0

Now, one can very easily find whether there is an edge between any two nodes by simply looking for the cell representing the two nodes and checking if there is a 1 or a 0. So, this was the adjacency matrix representation of graphs.

We can further extrapolate the application of the adjacency matrix by replacing these ones in the matrix with the weights for a weighted graph. Weighted graphs have a value/cost for each of the respective edges. These costs could represent anything, be it distance or time or cost literally. There is one traveling salesman problem where we store the shortest path from a city to some other city. Let's look at the cost adjacency matrix in detail.

Cost Adjacency Matrix:

The cost adjacency matrix is another method of representation of weighted graphs, where we represent our graph in the form of a matrix where cells are either filled with 0 or the cost of the edge. Let's call the cell falling on the intersection of i th row and j th column be A_{ij} , then the cell would be filled with the cost of the edge between node i and j if there is an edge between node i and j , otherwise, the cell would be filled with a 0 and if the cost could also be 0, then we'll fill -1 in the cell where there is no edge. Consider the graph illustrated below:



Now, we'll make a 6x6 matrix as follows and iterate through each of the cells, and see if there is an edge between the row number and the column number or not. If there is an edge, we'll place 1 in that cell, otherwise a zero. Since there are no self-loops even, we'll mark 0 to the cell having $i=j$. The filled adjacency matrix for the above graph would be, assuming the cost could be zero as well:

	0	1	2	3	4	5
0	-1	-1	4	-1	-1	-1
1	-1	-1	8	-1	-1	-1
2	4	8	-1	-1	2	11
3	-1	-1	-1	-1	9	-1
4	-1	-1	2	9	-1	-1
5	-1	-1	11	-1	-1	-1

So, this was the cost adjacency matrix representation of graphs. Other implementations are not that frequently used, so let's go through them quickly and in brief.

Edge Set: Store the pair of nodes/vertices connected with an edge.

Example: $\{(0, 1), (0, 2), (1, 2)\}$ for a graph having nodes 0, 1 and 2 all connected with each other.

Cost Adjacency List: Similar to the adjacency list, but instead of just storing the node value, we'll also store the cost of the edge too in the linked list.

Compact List Representation: here, the entire graph is compressed and stored in just one single 1D array.

Graph traversal & Graph traversal algorithms

In the last lecture, we discussed in detail the different ways to represent a

graph. We saw two of the most popular representation methods namely the adjacency list and the adjacency matrix. Today, we'll be interested in learning what graph traversal is and about different graph traversal algorithms. We'll also look at the reasons why these traversal techniques are so important.

What is graph traversal?

Graph traversal refers to the process of visiting (checking and/or updating) each vertex (node) in a graph. A smaller graph seems relatively easy to traverse. But when it comes to traversing a graph with a huge number of nodes, the process needs to be automated. Doing things manually increases the chances of missing some vertices or so. And visiting nodes of a graph becomes important when you need to change something for some nodes, or just need to retrieve the value present there or something else. And this is where our graph traversing algorithms come to the rescue. Sequences of steps that are used to traverse a graph are known as graph traversal algorithms. There are two algorithms that are used for traversing a graph. They are:

1. Breadth-First Search (BFS)
2. Depth First Search (DFS)

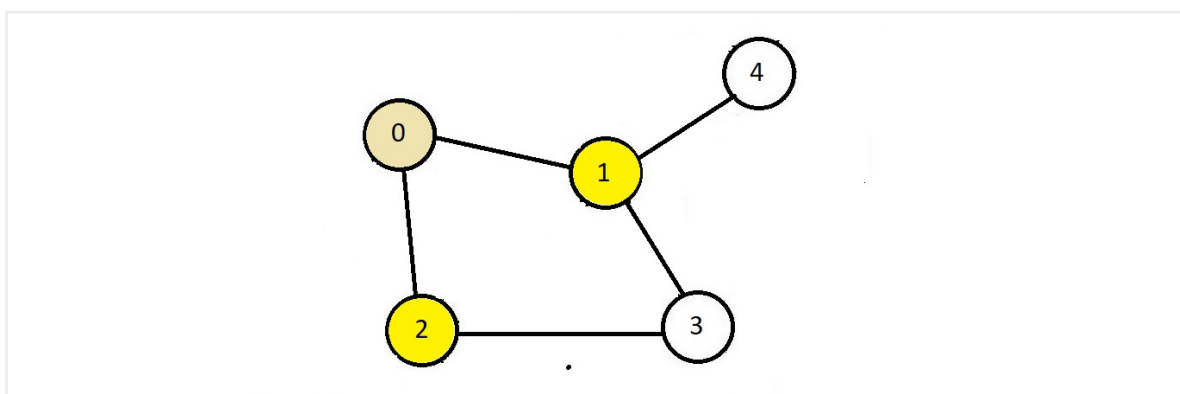
We'll see these algorithms in much detail in the coming lectures, but for today, we'll just sketch out a rough idea about these algorithms.

Breadth-First Search (BFS)

There are a lot of ways to traverse a graph, but the breadth-first search says, start with a node, and first, visit all the nodes connected to this node. But before we actually delve deep into this, there are a few terminologies we need to familiarise ourselves with.

Exploring a vertex (node):

- In a typical graph traversal algorithm, we traverse through (or visit) all the nodes of the graph and add it to the collection of all visited nodes.
- Exploring a vertex in a graph means visiting all the connected vertices. Follow the illustration of the graph I have mentioned below. We'll use this for understanding the breadth-first search.

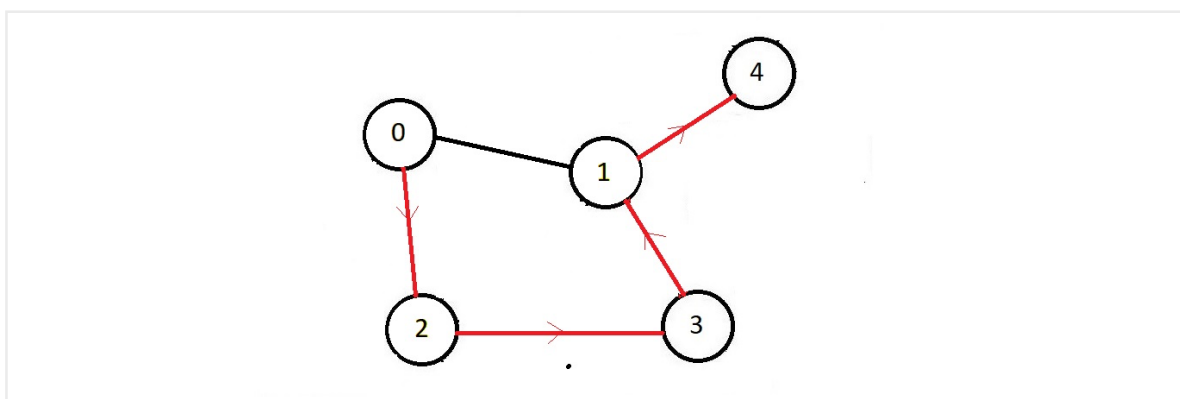


In breadth-first search, we use the queue data structure, and let me tell you that we won't be implementing the queue data structure all over again,

rather we'll presume that we have all its utility functions pre-defined. Here, suppose we start with node 0 and put it in the list of visited nodes, then we visit nodes 2 and 1 since they are directly connected to node 0. And then gets visited node 3 and 4. So, the order of exploring would be, 0, 2, 1, 3, 4. In breadth-first search, we visit from left to right all the nodes which lie on the same level. Here, node 0 was on one level, followed by nodes 2 and 1, and then nodes 3 and 4.

Depth First Search (DFS)

Follow the illustration of a graph I have mentioned below. We'll use this for understanding the depth-first search.



In depth-first search, we use the stack data structure, and we'll not implement a stack either. Suppose they are pre-defined and available for us to use. Here, suppose we start with node 0 and put it in the list of visited nodes, then we visit node 2 and then visit nodes further connected to node 2. So, we visit node 3 and since it is further connected to node 1 and node 1 is connected to 4, we'll follow them in the same order. So, the order of exploring would be, 0, 2, 3, 1, 4.

In depth-first search, we visit from top to down all the nodes which are connected with each other. Here, node 0 was the parent, which is connected to node 2, which is again connected to node 3, and then node 1 and then node 4.

So, this was the basic idea of each of these two algorithms. We'll discuss them in great detail in the upcoming lectures. We'll first understand the algorithm, then followed by their applications and the places where we prefer one over the other.

Breadth First Search (BFS) Graph Traversal in Data Structures

In the last lecture, we learned about what graph traversal is as well as two of the traversal techniques in brief. These were the Breadth-First Search and the Depth First Search. Today, we'll be dealing solely with the Breadth-First Search abbreviated as BFS in greater detail. I'll also provide you all with some techniques to derive the Breadth-First Search of any graph while only looking at it.

Before we actually move on to writing the algorithm of the Breadth-First Search or programming its implementation, we'll first visualize how

Breadth-First Search works.

Graph traversal refers to the process of visiting (checking and/or updating) each vertex(node) in a graph. And since traversing a very large graph manually becomes impossible, some algorithms are used to accomplish this task. One such algorithm of graph traversal is Breadth-First Search.

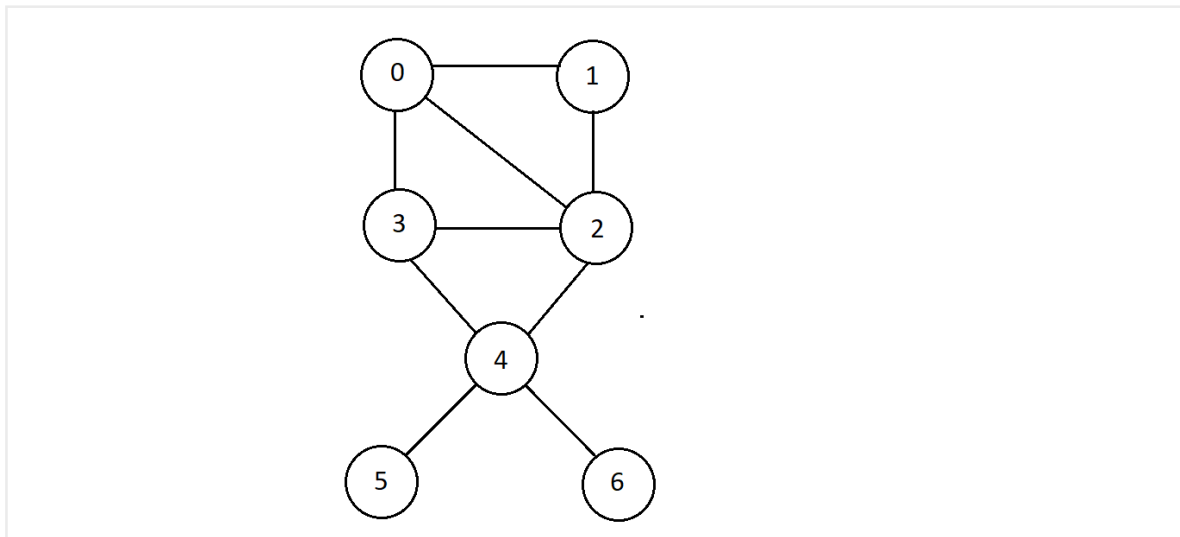
What is Breadth-First Search?

In Breadth-First Search, we start with a node (not necessarily the smallest or the largest) and start exploring its connected nodes. The same process is repeated with all the connecting nodes until all the nodes are visited.

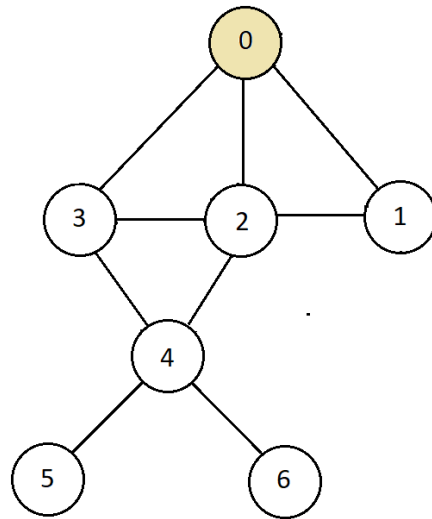
We should first learn the concept of the BFS spanning tree in order to understand the Breadth-First Search in a very intuitive way.

Method 1: BFS Spanning Tree:

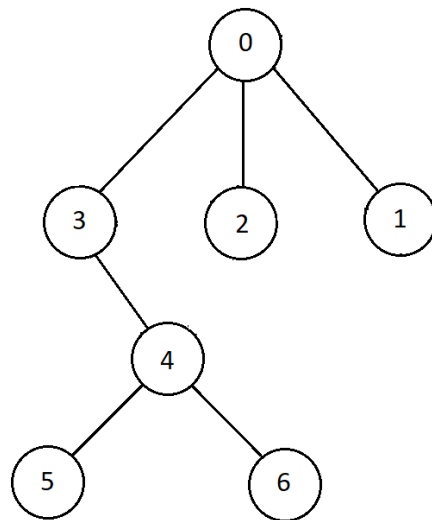
To understand what BFS Spanning Tree means, consider the graph I've illustrated below.



Now, choose any node, say 0, and try to construct a tree with this chosen node as its root. Or basically, hang this whole tree in a gravity-driven environment with this node and assume those edges are not rigid but flexible. So, the graph would now look something like this.



Now, mark dashed or simply remove all the edges which are either sideways or duplicate (above a node) to turn this graph into a valid tree, and as you know for a graph to be a tree, it shouldn't have any cycle. So, we can remove the edges between nodes 2 and 3, and then between nodes 1 and 2 being sideways. Then also between 2 and 4. You could have rather removed the one between node 3 and 4 instead of 2 and 4, but both ways work since these are duplicate to node 4. The tree we receive after we do these above-mentioned changes is,



This constructed tree above is known as a BFS Spanning Tree. And surprisingly, the level order traversal of this BFS spanning tree gives us the Breadth-First Search traversal of the graph we started originally with. And if you remember what a level order traversal of a tree is, we simply write the nodes in the same level from left to right. So, the level order traversal of the above BFS Spanning Tree is **0, 3, 2, 1, 4, 5, 6**.

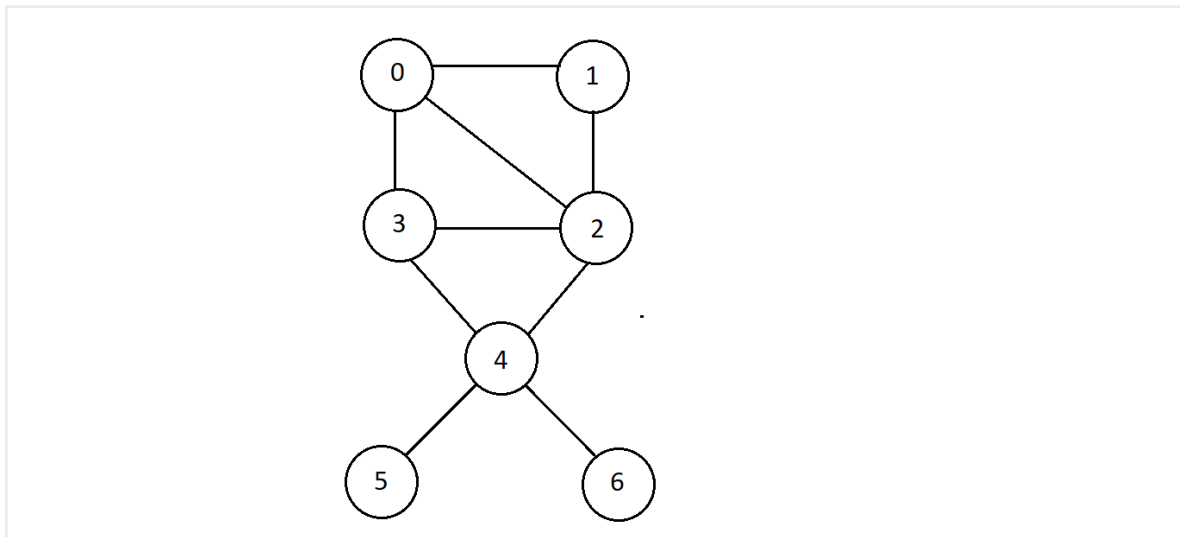
And a BFS Spanning Tree is not unique to a graph. We could have removed, as discussed above, the edge between nodes 3 and 4, instead of nodes 2 and 4. That would have yielded a different BFS Spanning Tree.

Therefore, given a graph, this is probably the easiest way, I believe, to write the Breadth-First Search traversal of the graph. Here, the chances of

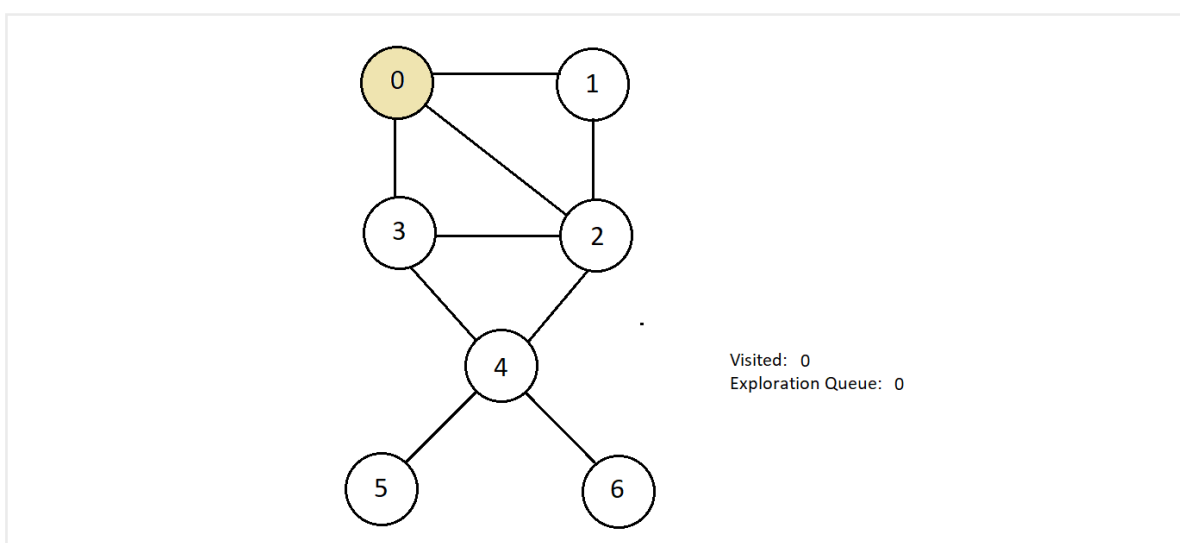
making a mistake are minimal. Despite the simplicity of the above technique, there is a very convenient method we follow when we implement the Breadth-First Search traversal algorithm in our program. Let us now discuss that.

Method 2: Conventional Breadth-First Search Traversal Algorithm:

Consider the same graph we covered above. Let me illustrate that again.

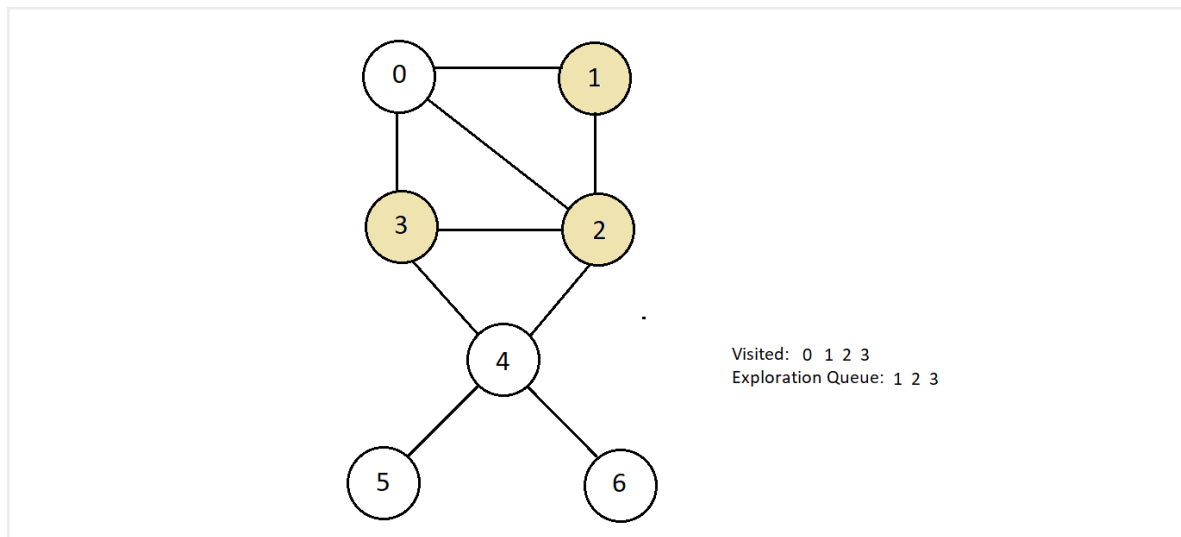


Considering we could begin with any source node; we'll start with 0 only. Let's define a queue named **exploration queue** which would hold the nodes we'll be exploring one by one. We would maintain another array holding the status of whether a node is already **visited** or not. Since we are starting with node 0, we would enqueue 0 into our exploration queue and mark it visited.

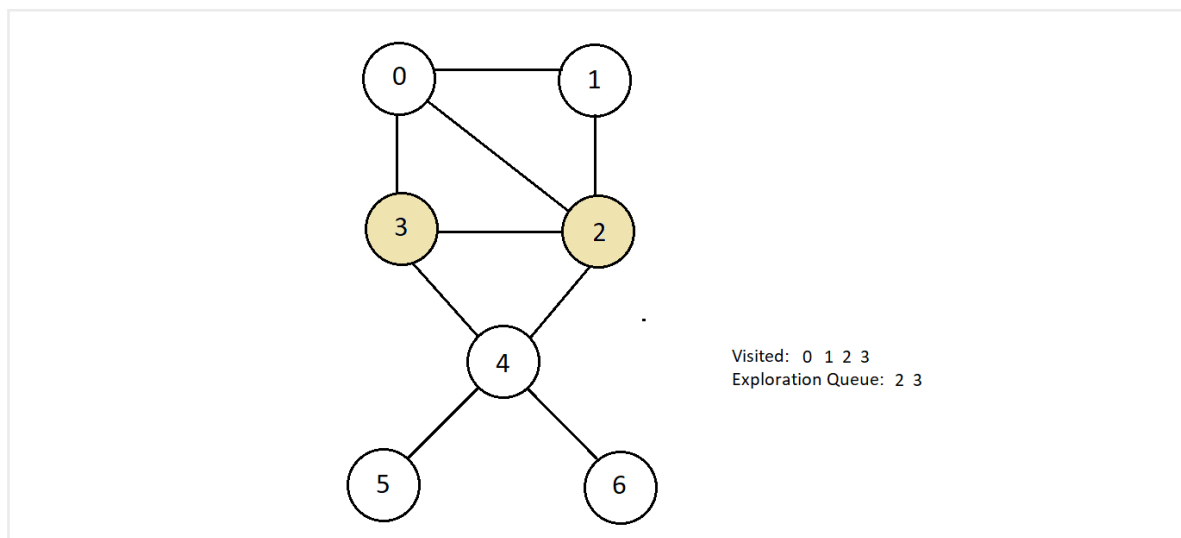


Now, we'll start visiting all the nodes connected to node 0, and remove node 0 from the exploration queue, enqueueing all the currently visited nodes which were nodes 1, 2, and 3. We are pushing them inside the exploration queue because these might further have some unvisited nodes

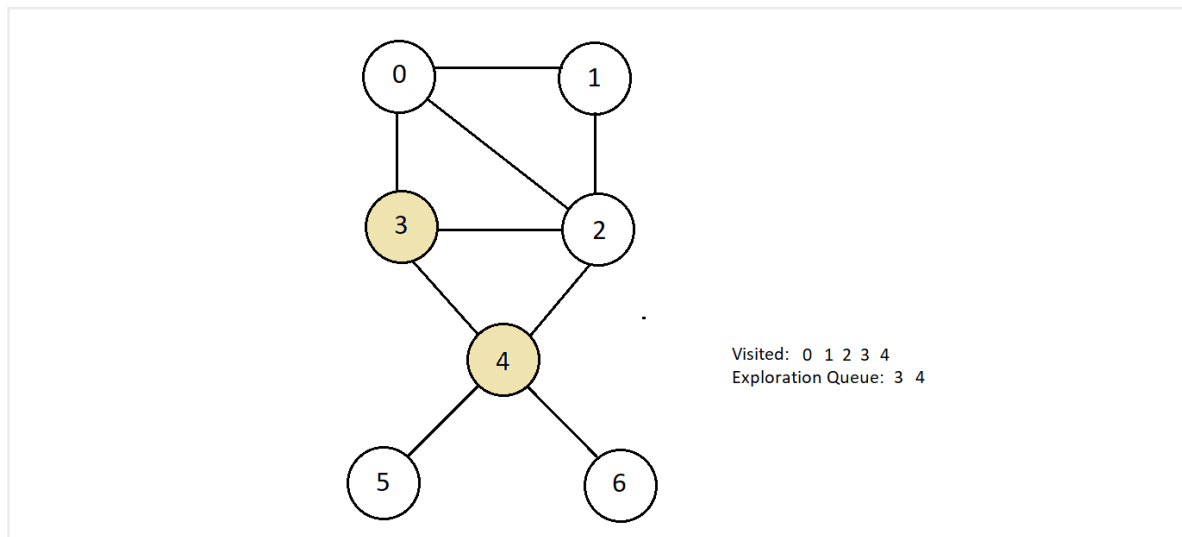
connected to them. Mark these nodes visited as well.



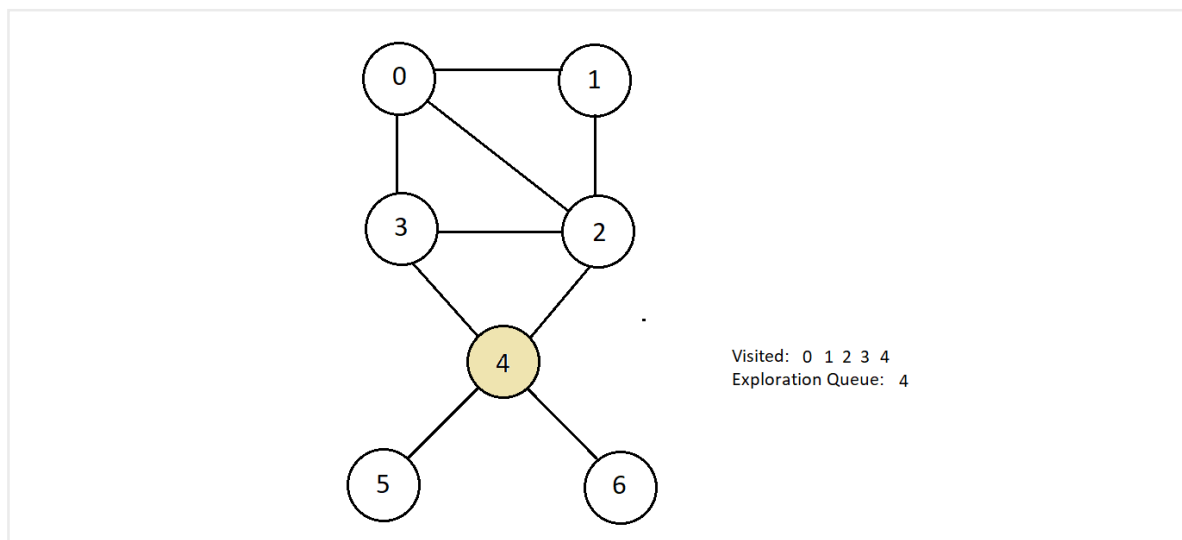
After this, we have node 1 at the top in the exploration queue, so we'll take it out and visit all unvisited nodes connected to it. Unfortunately, there aren't any. Therefore, we'll continue exploring further.



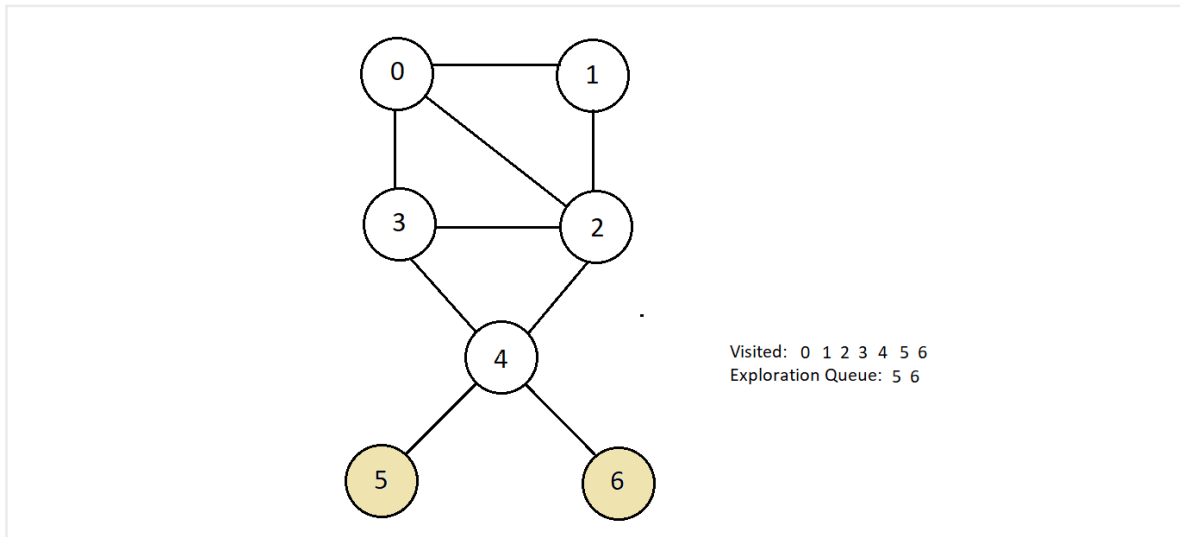
Next, we have node 2. And the only unvisited node connected to node 2 is node 4. So, we'll mark it visited and will also enqueue it in our exploration queue.



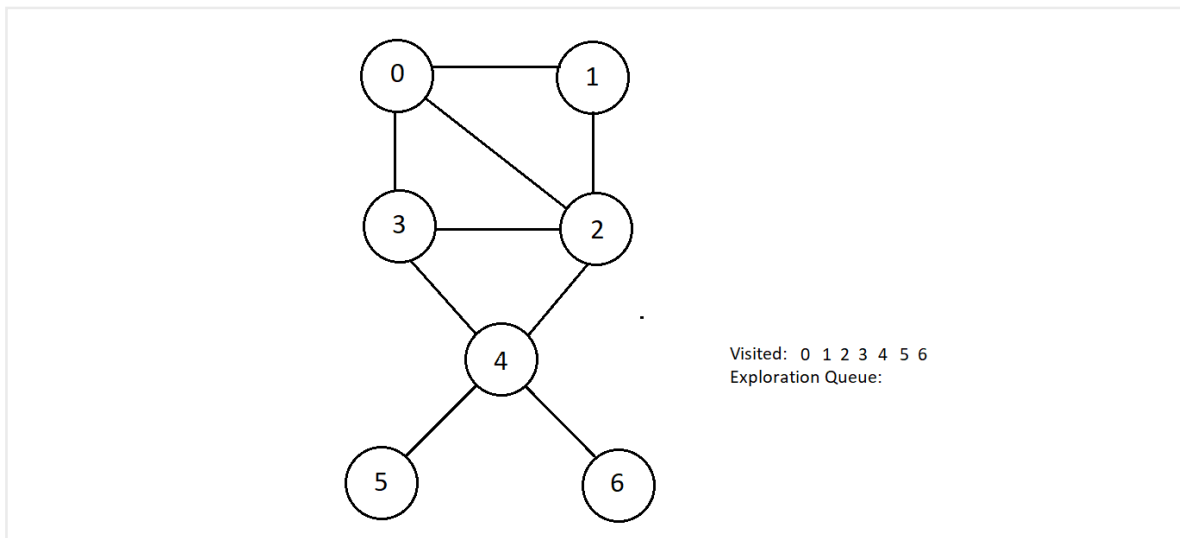
Node 3 is the next in line. Since, all nodes 1, 2, and 4 which are the nodes connected to it are already visited, we have nothing to do here while we are on node 3.



Next, we have node 4 on the top in the exploration queue. Let's get it out and see what nodes are connected and unvisited to it. So, we got nodes 5 and 6. Mark them visited and push them inside the exploration queue.



And now we can explore the other two nodes left in the queue, and since all nodes are already visited, we'll get nothing in them. And this got our queue emptied and every node traversed in Breadth-First Search manner.



And the order in which we marked our nodes visited is the Breadth-First Search traversal order. Here, it is **0, 1, 2, 3, 4, 5, 6**. So basically, the visited array maintains whether the node itself is visited or not, and the exploration queue maintains whether the nodes connected to a node are visited or not. This was the difference.

Let's now see how the process we did manually above can be automated in C although we will be using pseudocode for now. In the next lecture, we'll discuss it in more detail.

1. We'll take a whole graph and the information about its nodes and edges as the input along with the source node s .
2. We'll mark the node s visited and then create a queue, and enqueue s in it.
3. We'll then initiate a while loop which will run until the queue is not

empty. At each iteration, we will take out the first element out of the queue, and visit all the vertices already not visited and connected to it while enqueueing every new node we visit in the queue.

4. Below is the pseudocode, and I encourage you all to read it and try to implement the same in C.

```
- Input: A graph  $G = (V, E)$  and source node  $s$  in  $V$ 
- Algorithm:

- Mark all nodes  $v$  in  $V$  as unvisited
- Mark source node  $s$  as visited
-  $enq(Q, s)$  //First-in first-out Queue
- while( $Q$  is not empty)
{
     $u := deq(Q)$ ;
    for each unvisited neighbour  $v$  of  $u$  {
        mark  $v$  as visited;
         $enq(Q, v)$ ;
    }
}
```

Few important points before we leave:

1. We can start with any vertex.
2. There can be multiple Breadth-First Search results for a given graph
3. The order of visiting the vertices can be anything.
4. **Quiz:** Try to find another valid Breadth-First Search for the same graph we discussed above. (Hint: Start with nodes other than 0).

We are now just left with the programming part, which will be covered super soon in the next lecture. Keep revising the concept until then.

BFS Implementation in C | C Code For Breadth First Search

In the last lecture, we learned about the concepts of Breadth-First Search in graph traversal. We saw two of its methods, first one using the BFS spanning tree, and the second one being one of the most conventional ones where we maintained a visited array and an exploration queue while we explored new nodes to see what we have visited already and what we have yet to explore respectively. Today we will implement Breadth-First Search in C language.

If you recall, we gave you all a pseudocode in the end and asked you all to read and try implementing the same in C. So, let me know if you could. The pseudocode was:

```

- Input: A graph  $G = (V, E)$  and source node  $s$  in  $V$ 
- Algorithm:

- Mark all nodes  $v$  in  $V$  as unvisited
- Mark source node  $s$  as visited
-  $enq(Q, s)$  //First-in first-out Queue
- while( $Q$  is not empty)
    {
         $u := deq(Q)$ ;
        for each unvisited neighbour  $v$  of  $u$  {
            mark  $v$  as visited;
             $enq(Q, v)$ ;
        }
    }

```

1. Now, the first thing we did was, we took the input. The input comprises the information concerning the graph, its nodes/vertices, and edges, and the source node we'll start the traversal with.
2. Then, we'll mark the source node s visited and then create an exploration queue, and enqueue the source code s in it.
3. We'll then initiate a while loop which will run until the queue is not empty. At each iteration, we will take out the first element out of the queue using the utility function dequeue and suppose that element is u . Then, we visit all the vertices which are directly connected to u and are already not visited.
4. And every time we visit a new node, we enqueue them in our exploration queue. Eventually, the queue becomes empty, and our traversal ends. And the visited array is the order of our Breadth-First Search traversal.

So, this was the theory part. We are now ready to move on to our programming segment. Having said that, let's move directly to our editors. I have attached the source code below. Follow it as we proceed.

Understanding the source code below:

1. The first thing you should do is, bring the implementation part of the queue from our previous lectures. In this way, we can save time and reduce repetition. I have literally copied everything from my previous snippet including the queue, and its utility functions implemented using arrays.

Initializing a queue:

2. Next thing you should do is initialize a queue, and dynamically assign it a memory in heap using malloc and define its size, say 400. Mark both its front and rear at zeroth index.

```

// Initializing Queue (Array Implementation)
struct queue q;
q.size = 400;
q.f = q.r = 0;

```

```
q.arr = (int*) malloc(q.size*sizeof(int));
```

Copy

Code Snippet 1: Initializing a queue

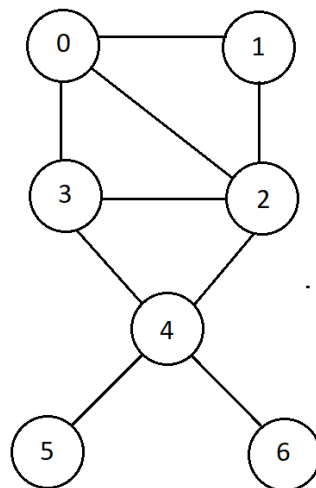
BFS Implementation:

3. Before we start programming the BFS implementation of the graph, we should first define a graph. And out of all the methods we have studied to represent a graph, we will be using the adjacency matrix one to define our graph here.

4. Define an integer variable *node* which we will use to traverse the graph. Define another integer variable *i* which is the node we are currently at. So, since we would be the default starting with any node of our choice, we would initialize it with node 1.

5. Now, define an integer array *visited* to store the status of a node (of size 7 here), and the values corresponding to a node is 0, if it is unvisited and 1, if it is visited. And since, no node is already visited when we first start, therefore we'll fill this array with all zeros.

6. Create an adjacency matrix corresponding to the graph I've illustrated below.



An adjacency matrix holds a value 1 for a cell that is at the intersection of the *i*th row and *j*th column if there is an edge between node *i* and node *j*. So, the adjacent matrix corresponding to the graph above is:

	0	1	2	3	4	5	6
0	0	1	1	1	0	0	0
1	1	0	1	0	0	0	0
2	1	1	0	1	1	0	0
3	1	0	1	0	1	0	0
4	0	0	1	1	0	1	1
5	0	0	0	0	1	0	0
6	0	0	0	0	1	0	0

7. Next, we would print the source node we have chosen, here node 1. Also, mark this node visited, which means make the i th index of the visited array 1. And since we are now interested in exploring the nodes connected to this source node, we would first enqueue this node into the queue q we created in step 2.

8. Create a while loop, and make it run while our queue is not empty. Queue becoming empty is the condition where we finish our traversal. We use the utility function *isEmpty* for the same.

Inside this loop, pluck the top element of the queue using *dequeue* and store it in an integer variable *node*. Run another *for* loop using j as the iterator, and since the size of our graph is 7, we'll make this loop run from 0 to 7.

Now, every time we find edges *node* & j connected (that is cell $a[node][j]$ equal to 1) and the node j unvisited, we mark it visited, and enqueue this newly visited node j in the queue. We'll print these nodes while we visit them to determine the BFS traversal order of the graph.

This way we will explore all the nodes, and the queue will ultimately become empty.

// BFS Implementation

```
int node;
int i = 1;
int visited[7] = {0,0,0,0,0,0,0};
int a [7][7] = {
    {0,1,1,1,0,0,0},
    {1,0,1,0,0,0,0},
    {1,1,0,1,1,0,0},
    {1,0,1,0,1,0,0},
    {0,0,1,1,0,1,1},
    {0,0,0,0,1,0,0},
    {0,0,0,0,1,0,0}
};
printf("%d", i);
visited[i] = 1;
```

```

enqueue(&q, i); // Enqueue i for exploration
while (!isEmpty(&q))
{
    int node = dequeue(&q);
    for (int j = 0; j < 7; j++)
    {
        if(a[node][j] ==1 && visited[j] == 0){
            printf("%d", j);
            visited[j] = 1;
            enqueue(&q, j);
        }
    }
}
}
Copy

```

Code Snippet 2: Implementing Breadth-First Search

Here is the whole source code:

```

#include<stdio.h>
#include<stdlib.h>

struct queue
{
    int size;
    int f;
    int r;
    int* arr;
};

int isEmpty(struct queue *q){
    if(q->r==q->f){
        return 1;
    }
    return 0;
}

int isFull(struct queue *q){
    if(q->r==q->size-1){
        return 1;
    }
    return 0;
}

void enqueue(struct queue *q, int val){
    if(isFull(q)){

```

```

        printf("This Queue is full\n");
    }
    else{
        q->r++;
        q->arr[q->r] = val;
        // printf("Enqueued element: %d\n", val);
    }
}

```

```

int dequeue(struct queue *q){
    int a = -1;
    if(isEmpty(q)){
        printf("This Queue is empty\n");
    }
    else{
        q->f++;
        a = q->arr[q->f];
    }
    return a;
}

```

```

int main(){
    // Initializing Queue (Array Implementation)
    struct queue q;
    q.size = 400;
    q.f = q.r = 0;
    q.arr = (int*) malloc(q.size*sizeof(int));

    // BFS Implementation
    int node;
    int i = 1;
    int visited[7] = {0,0,0,0,0,0,0};
    int a [7][7] = {
        {0,1,1,1,0,0,0},
        {1,0,1,0,0,0,0},
        {1,1,0,1,1,0,0},
        {1,0,1,0,1,0,0},
        {0,0,1,1,0,1,1},
        {0,0,0,0,1,0,0},
        {0,0,0,0,1,0,0}
    };
    printf("%d", i);
    visited[i] = 1;
    enqueue(&q, i); // Enqueue i for exploration
    while (!isEmpty(&q))
    {
        int node = dequeue(&q);
    }
}

```

```

    for (int j = 0; j < 7; j++)
    {
        if(a[node][j] == 1 && visited[j] == 0){
            printf("%d", j);
            visited[j] = 1;
            enqueue(&q, j);
        }
    }
}
return 0;
}

```

Copy

Code Snippet 3: Implementing BFS using queue in C

We'll see if our algorithm actually works, and if it reverts the Breadth-First Search traversal order of the graph we fed into the program. As you could observe, we have started our traversal considering node 0 as the root node. And when we ran the program, the output we received was:

0123456

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 1: Output when the root node is 0

But if we change our root node from 0 to 1, the output changes to:

1023456

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 2: Output when root node is 1

I encourage you all to try this yourself. It shall work for any node you start with, and any graph you feed into the program. I hope I made this clear for you all. You can go through the lectures again for a better understanding of things. Depth First Search is what we'll cover next.

Depth First Search (DFS) Graph Traversal in Data Structures

In the last lecture, we completed learning about the first method of graph traversal which was the Breadth-First Search as well as its implementation in C. Today, we'll be dealing solely with the Depth First Search abbreviated as DFS in greater detail.

Before we actually move on to writing the algorithm of the Depth First Search or programming its implementation, we'll first visualize how Depth First Search works.

As we know, graph traversal becomes impossible when done manually for very large graphs. Therefore, some algorithms are used to automate this task. One such algorithm we learned was BFS, and another one, which we would learn today is the Depth First Search.

What is Depth First Search?

In Depth First Search, we start with a node and start exploring its connected nodes, keeping on suspending the exploration of previous vertices. And those suspended nodes are explored once we finish exploring

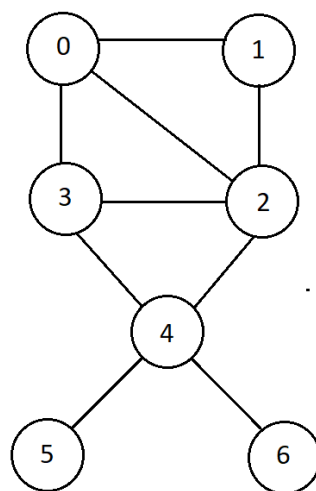
the node below. And this way we explore all the nodes of the graph. If we now compare what we are doing differently from BFS, we would find that there is one major thing that shouldn't go unnoticed. In BFS, we explore all the nodes connected to a node, then explore the nodes we visited in a horizontal manner, while in DFS, we start with the first connected node, and similarly go deep, so this looks like visiting the nodes vertically. This might sound confusing for now, so let's make it more intuitive for you to understand.

DFS Procedure:

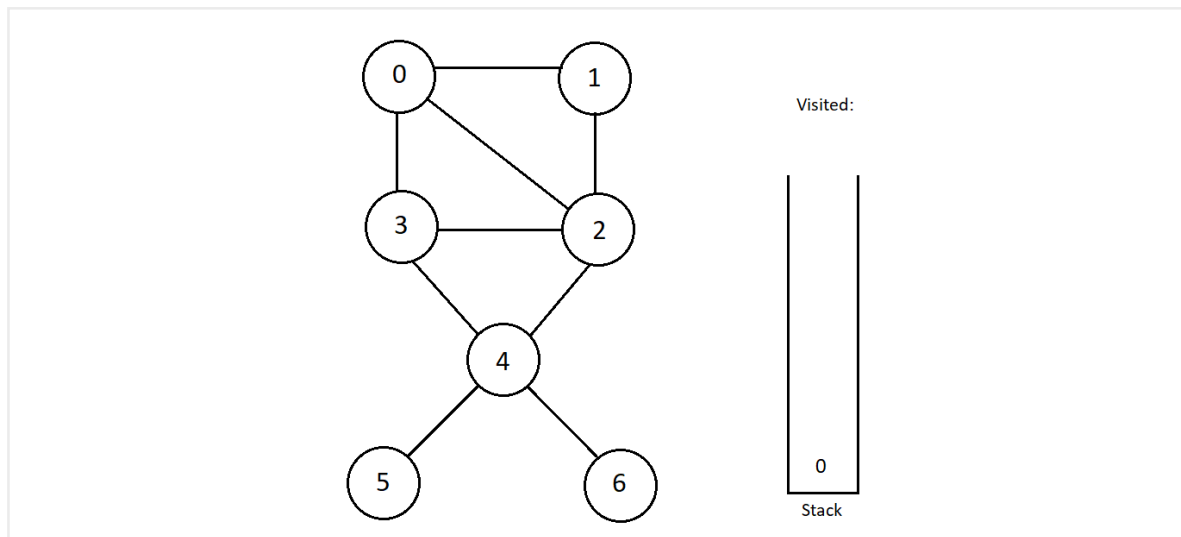
The procedure we follow to accomplish the depth-first search traversal of a graph is:

1. Let's define a stack named which would hold the nodes we'll be suspending to explore one by one later. And stack, if you remember, is a data structure in which the element you push the last comes out the first. Choose any node as the source node and push it into the top of the stack we created. We would maintain another array holding the status of whether a node is already **visited** or not.
2. Take the top item of the stack and mark it visited or add it to the visited list.
3. Create a list of the nodes directly connected to the vertex we visited. Push the ones which are still not visited into the stack.
4. Repeat steps 2 and 3 until the stack is not empty.

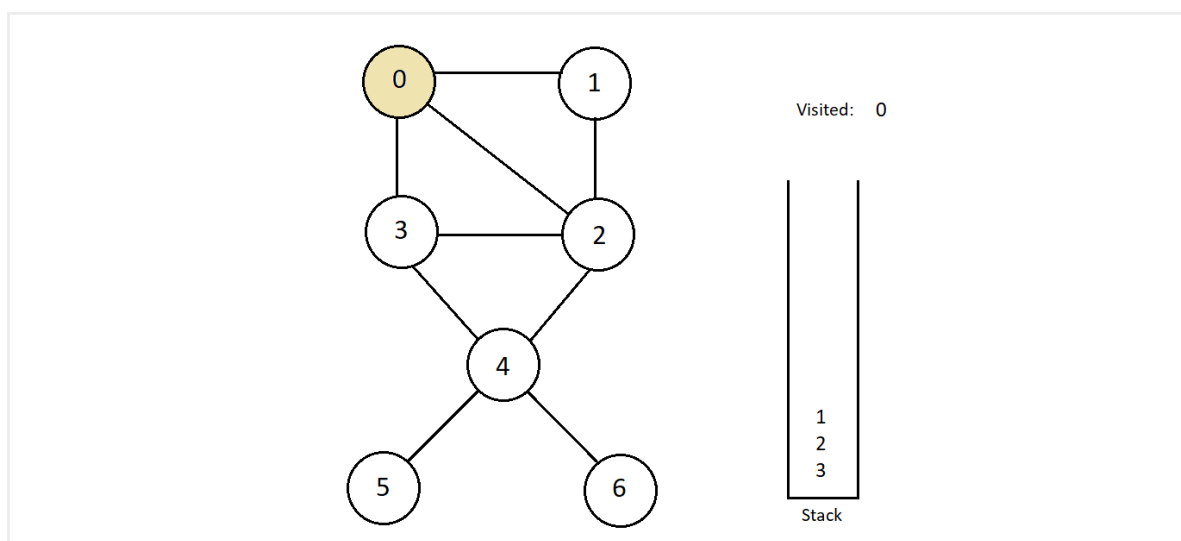
To understand the procedure of the Depth First Search, consider the graph I've illustrated below. For better contrast, I took the same graph as in the last lecture.



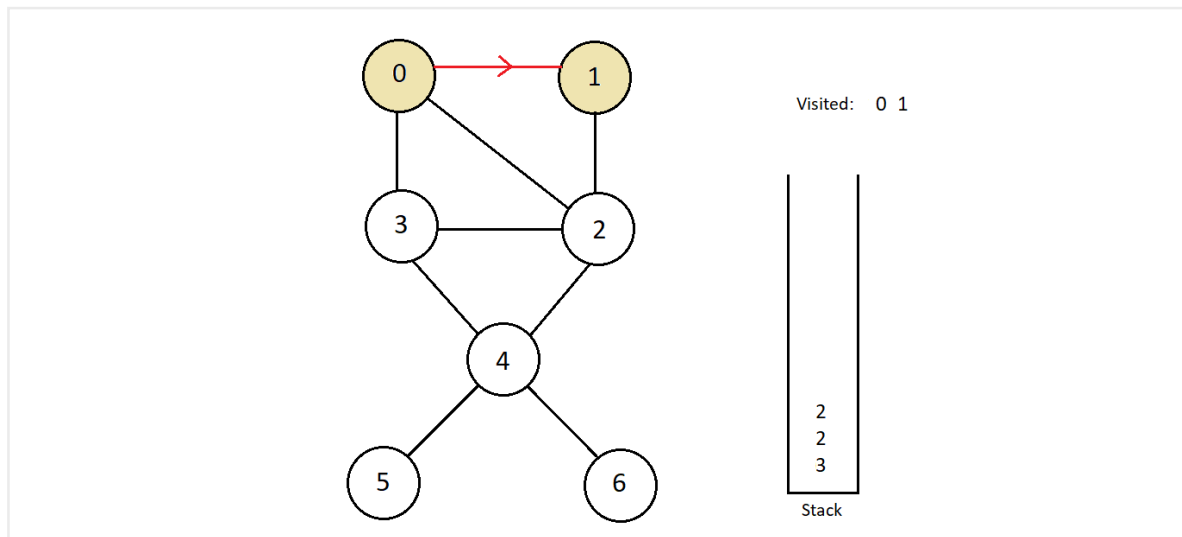
Considering the fact that we could begin traversing with any source node; we'll start with 0 only. So, following step 1, we would push this node into the stack and begin our Depth First Search traversal.



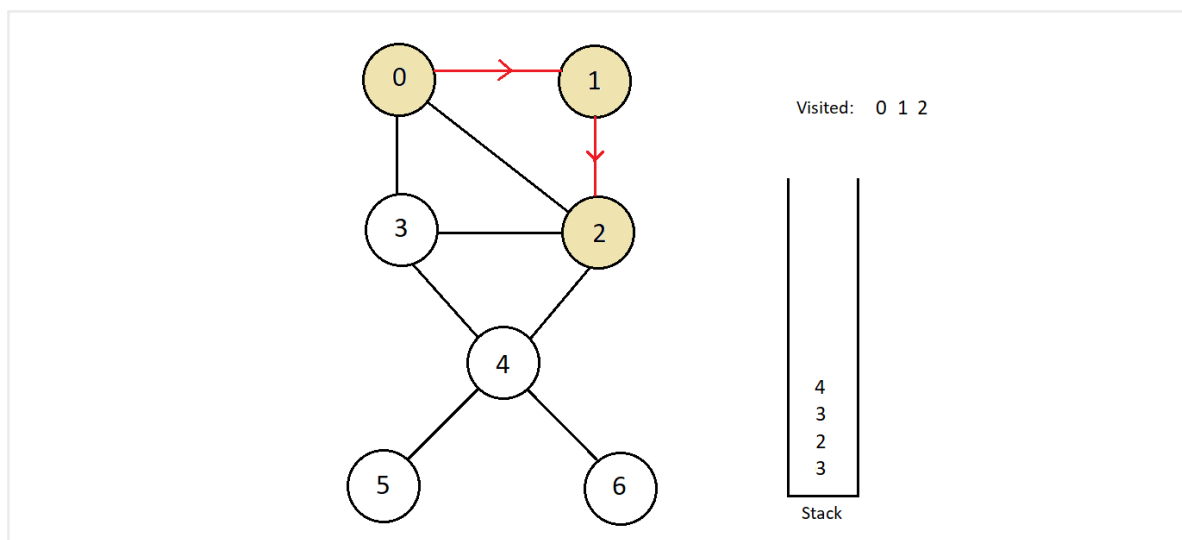
The next step says, pop the top element from the stack which is node 0 here, and mark it visited. Then, we'll start visiting all the nodes connected to node 0 which are not visited already, but before that, we are asked to push them all into the stack, and the order in which you push doesn't matter at all. Therefore, we will push nodes 3, 2, and 1 into the stack.



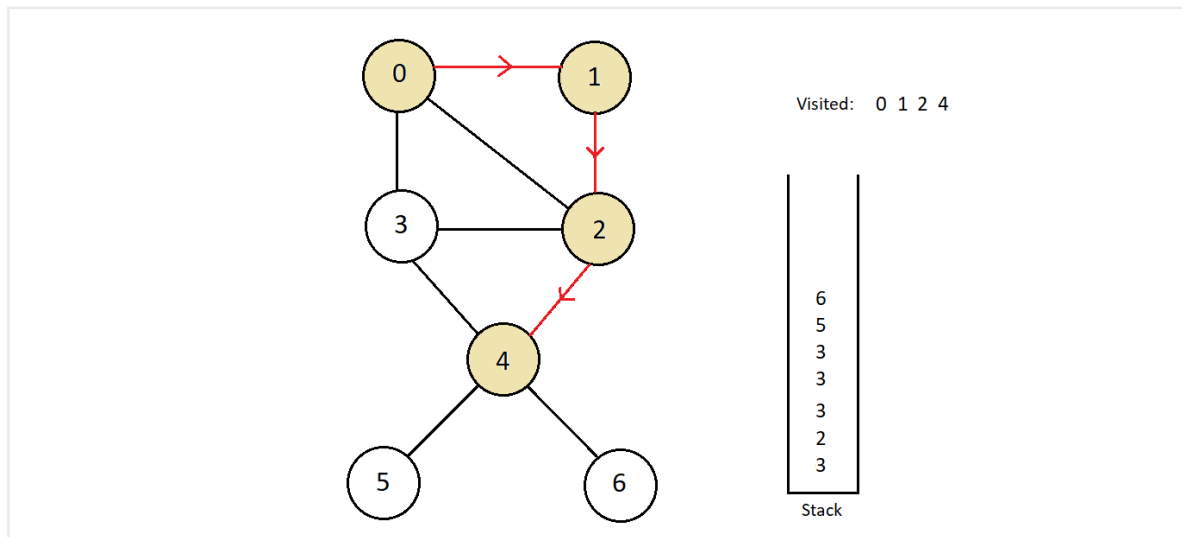
Repeating the steps above, we'll now pop the top element from the stack which is node 1, and mark it visited. Only nodes connected to node 1 were nodes 0 and 2, and since the only unvisited one is node 2. It's important to observe here, that although node 2 is in the stack, it is not visited. So, we'll push it into the stack again.



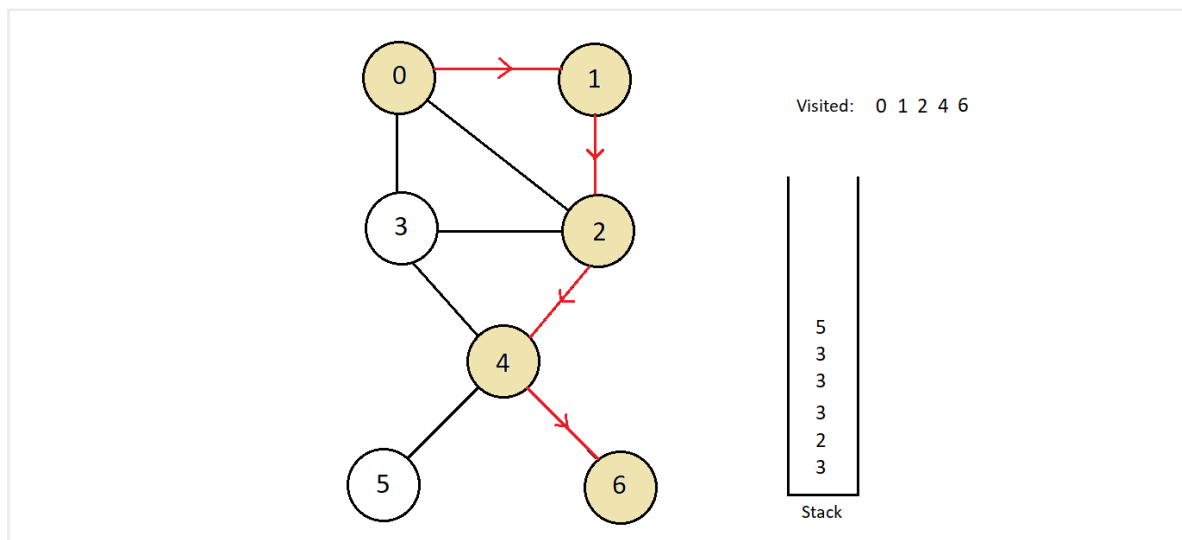
Next, we have node 2 at the top of the stack. We'll mark node 2 visited and unvisited nodes connected to node 2 are nodes 3 and 4, regardless of the fact that 3 is already there in the stack. So, we'll just push nodes 3 and 4 into the stack.



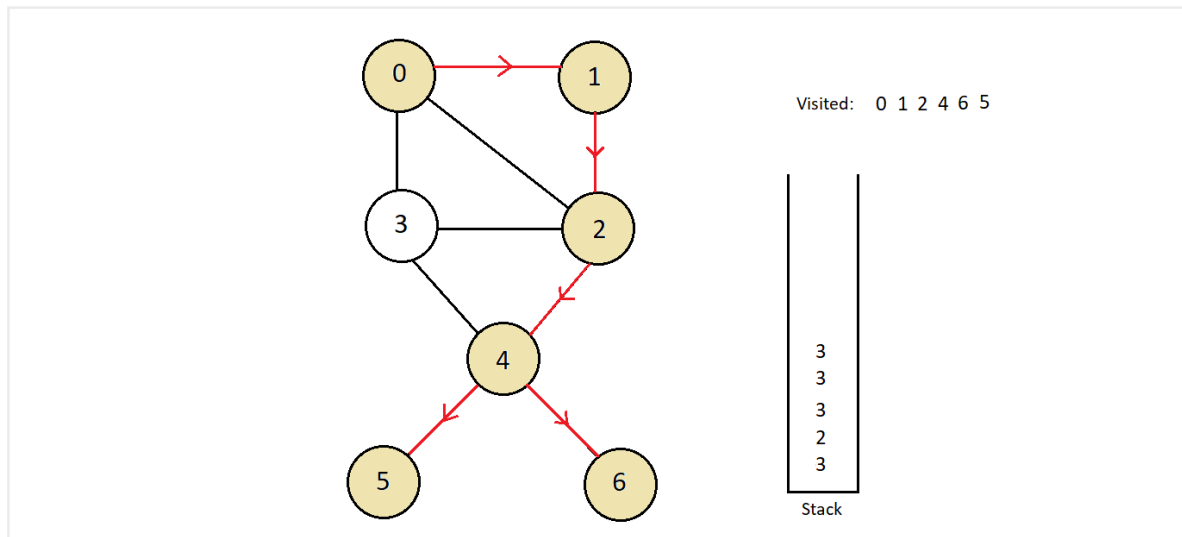
Node 4 is the next we have on the top. So, just mark it as visited. Since, all nodes 3, 5, and 6, except node 2, which are directly connected to it are not visited, we'll push them into the stack.



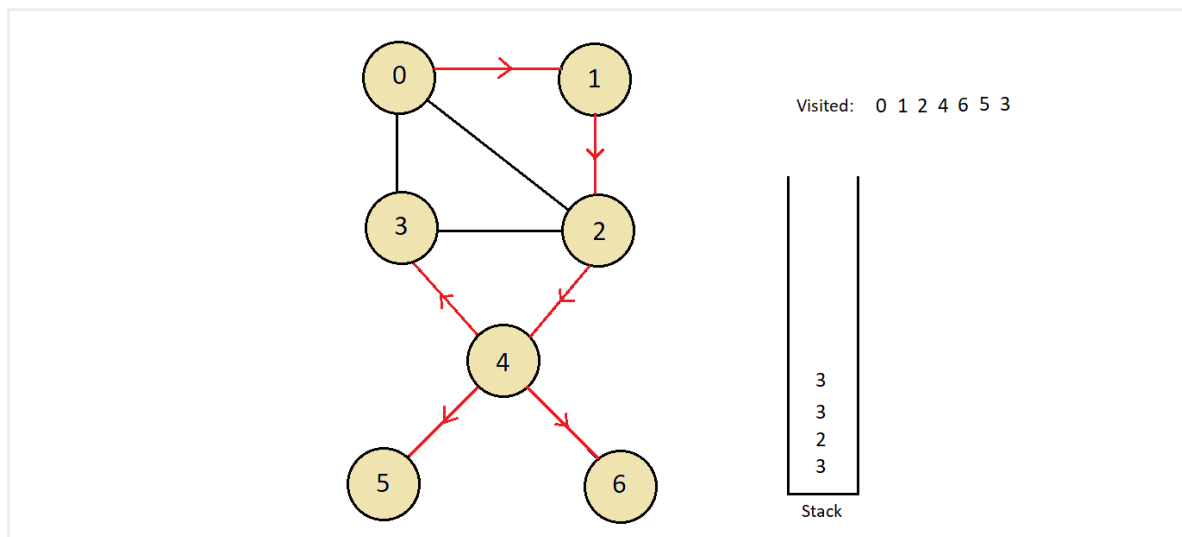
Next, we have node 6 on the top of the stack. Pop it and mark it visited. Since there are no nodes that are directed connected to node 6 and unvisited, we'll continue further without doing anything.



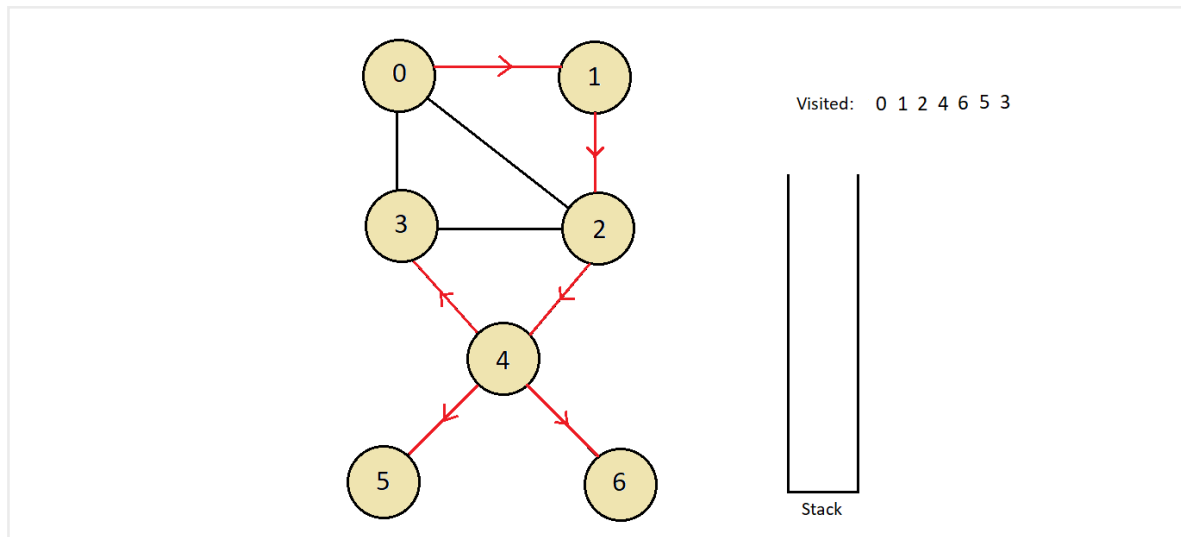
Next, we pop node 5 out of the stack and mark it visited. And since there is no unvisited node connected to it, we continue.



Node 3 comes next to be visited, being on the top in the stack. Mark node 3 visited and again there are no nodes left unvisited and connected to node 3. So, we just continue popping out elements from the stack.



Now, if you could observe, there are no nodes left to be visited. Although there are elements in the stack to be explored. So, we just pop them one by one and ignore finding them already visited. And this gets our stack emptied and every node traversed in Depth First Search manner, ultimately.



And the order in which we marked our nodes visited is the Depth First Search traversal order. Here, it is **0, 1, 2, 4, 6, 5, 3**. So basically, the visited array maintains whether the node itself is visited or not, and the stack maintains nodes whose exploration got suspended earlier. This was the difference.

One important reason why stack is used in the implementation of DFS and what makes it relatively easy is that it can directly be used in the form of functions since function calls use memory stacks. I will tell you how in the next lecture.

Let's now see how the process we did manually above can be automated in C although we will be using pseudocode for now. In the next lecture, we'll discuss it in more detail.

- **Input:** A graph $G = (V, E)$ and source code s in V
- **Algorithm:**

DFS(G, u)

```

u.visited = true
for each  $v \in G.adj(u)$ 
  if  $v.visited == false$ 
    DFS( $G, v$ )

```

```

init()
for each  $u \in G$ 
  u.visited = false
for each  $u \in G$ 
  DFS( $G, u$ )

```

We are now just left with the programming part, which will be covered in the next segment. Keep practicing the concepts until then. Try writing the algorithm in C yourselves using the pseudocode I have provided above. We would anyway do that in the next lecture.

DFS Implementation in C | C Code For Depth First Search

In the last lecture, we learned about the concepts of Depth First Search in graph traversal. We discussed the differences we observed in both the graph traversal methods. We discussed the algorithm to automate the DFS traversal of a graph. Today we will implement the same Depth First Search algorithm in C language.

Implementing Depth First Search, as I told you all in the last lecture, is one of the easiest jobs to do. DFS implementation asks you to use stacks, but if you know, a function call itself uses a memory stack, so calling a function recursively will do our work. And If you recall, we gave you all a pseudocode in the end and asked you all to read and try implementing the same in C. So, let me know if you could. The pseudocode was:

```
- Input: A graph  $G = (V,E)$  and source node  $s$  in  $V$ 
- Algorithm:

DFS( $G,u$ )

    u.visited = true
    for each  $v \in G.adj(u)$ 
        if v.visited == false
            DFS( $G,v$ )

init()
    for each  $u \in G$ 
        u.visited = false
    for each  $u \in G$ 
        DFS( $G,u$ )
```

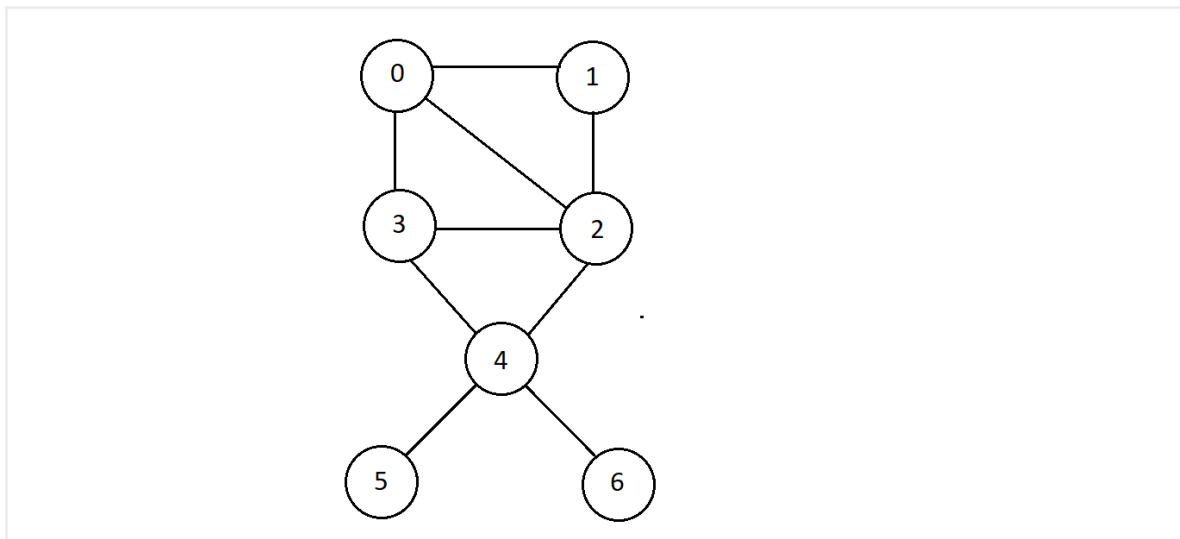
1. Now, the first thing we did was, we took the input. The input comprises the information concerning the graph, its nodes/vertices, and edges, and the source node we'll start the traversal with.
2. Then, we'll call the function DFS feeding a source node into it.
3. In the function, we'll mark the node visited, and then initiate a for loop which will access the connected nodes to this node, and see if they are visited or not.
4. And every time we find a node connected and not visited already, we call the DFS function recursively filling out our function stack. Eventually, the function stack becomes empty, and our traversal ends. And the visited array is the order of our Depth First Search traversal. Well, this was the theory part. We're now ready to get started on the programming part. Let's move directly to our editors. I have attached the source code below. Follow it as we proceed.

Understanding the source code below:

1. Since we will be using functions to be able to use stacks directly, we don't really have to bring the implementation part of the stack from our

previous lectures. Let's jump to our DFS implementation directly.

2. Before we start programming the DFS implementation of the graph, we should first define a graph. And out of all the methods we have studied to represent a graph, we will be using the adjacency matrix one to define our graph here. And since we have already done this in our last programming lecture where we implemented the Breadth-First Search, we'll bring the same graph here as well. This would save us a lot of time. So, basically, we got ourselves an adjacency matrix $A[7][7]$ corresponding to the graph I've illustrated below.



3. Now, define an integer array *visited* to store the status of a node (of size 7 here), and the values corresponding to a node is 0 if it is unvisited and 1 if it is visited. We will fill this array with all zeros since no node has already been visited when we start.

Creating function DFS:

4. Now, we wish to define a function that would carry the Depth First Search traversal of the graph for us. So, create a void function *DFS* which will have the node *i* we want to visit as the only parameter. Inside that function, since that node which the function received as the parameter is the one to be visited now, we would mark it visited and print it at the same time.

5. Next step requires you to see all nodes connected directly to this node. Run a for loop of size 7 with an iterator *j*, since that is the number of nodes we have. If any of them is not visited and there is an edge in between *i* and *j*, we would recursively call DFS passing this node *j*. And that's all. Trust your function and it will do things in its own way. Our job has finished.

```
void DFS(int i){  
    printf("%d ", i);  
    visited[i] = 1;  
    for (int j = 0; j < 7; j++)
```

```

    {
        if(A[i][j]==1 && !visited[j]){
            DFS(j);
        }
    }
}

```

Copy

Code Snippet 1: Creating the DFS function

Here is the whole source code:

```

#include<stdio.h>
#include<stdlib.h>

int visited[7] = {0,0,0,0,0,0,0};
int A [7][7] = {
    {0,1,1,1,0,0,0},
    {1,0,1,0,0,0,0},
    {1,1,0,1,1,0,0},
    {1,0,1,0,1,0,0},
    {0,0,1,1,0,1,1},
    {0,0,0,0,1,0,0},
    {0,0,0,0,1,0,0}
};

void DFS(int i){
    printf("%d ", i);
    visited[i] = 1;
    for (int j = 0; j < 7; j++)
    {
        if(A[i][j]==1 && !visited[j]){
            DFS(j);
        }
    }
}

```

```

int main(){
    // DFS Implementation
    DFS(0);
    return 0;
}

```

Copy

Code Snippet 2: Implementing DFS using recursive function in C

We'll see if our algorithm actually works, and if it reverts the Depth First Search traversal order of the graph we fed into the program. If you could observe, we have started our traversal considering node 0 as the root node. And when we ran the program, the output we received was:

0 1 2 3 4 5 6

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 1: Output when the root node is 0

But if we change our root node from 0 to 4, the output changes to:

4 2 0 1 3 5 6

PS D:\MyData\Business\code playground\Ds & Algo with Notes\Code>

Copy

Figure 2: Output when the root node is 4

And this is how we implemented Depth First Search traversal in C without even explicitly using stacks just via a recursive function. You should all give it a shot too. No matter what node you start with, and whatever graph you feed the program with, it will work. Hopefully, that was clear. You can go through the lectures again for a better understanding of things. Stay tuned, as we are about to dive deep into graph theory.

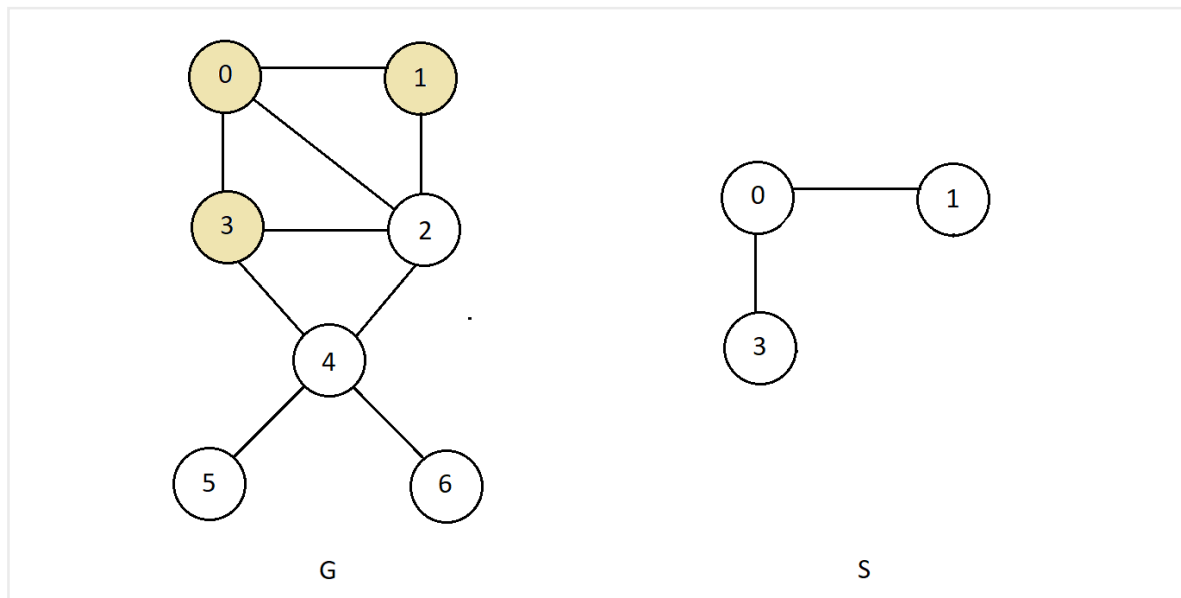
Spanning Trees & maximum no of possible spanning trees for complete graphs

In the last lecture, we finished learning both the traversal algorithms, the Breadth-First Search and the Depth First Search. We implemented both of these in C using queues and stacks respectively. Today, we'll learn what spanning trees are, and the concepts behind them, and other small topics such as connected graphs and others needed to understand spanning trees.

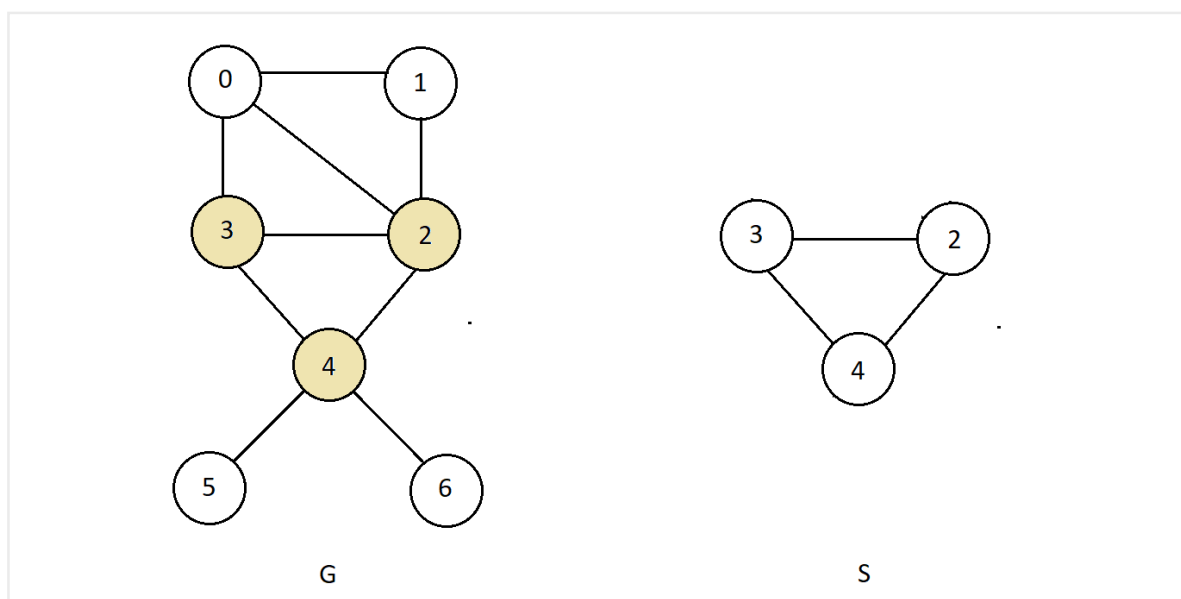
While we'll be learning more advanced topics, you will realize that applications of the algorithms we wrote BFS and DFS are not limited to just finding the traversal order of a graph, rather much more. Let's first start with what spanning trees are. But before we dig into spanning trees, we should talk about a few things you should know. First one being, a subgraph.

Subgraphs:

A subgraph of a graph G is a graph whose vertices and edges are subsets of the original graph G . It means that if we consider the graph G illustrated below, then its subgraphs could be the graph S .



S is a subgraph of graph G because nodes 0, 1, and 3 form the subset of the set of nodes in G, and the edges connecting 0 to 3 and 0 to 1 also form the subset of the set of edges in G. Another subgraph of Graph G could be the one mentioned below.

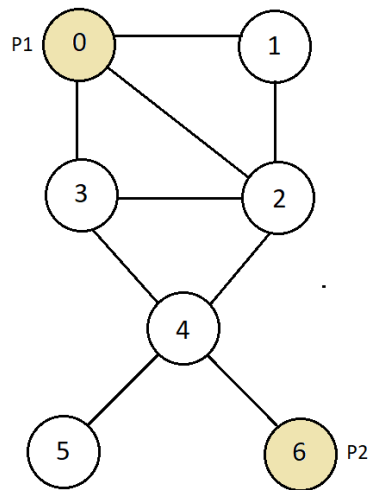


This is a subgraph of graph G too, because nodes 3, 2, and 4 form the subset of the set of nodes in G, and the edges connecting 3 to 4, 3 to 2, and 2 to 4 also form the subset of the set of edges in G.

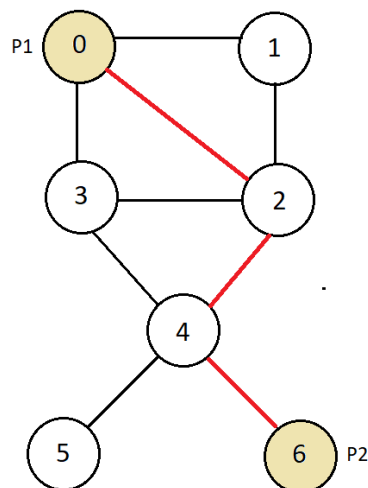
That should be enough for you to understand what subgraphs of a graph are. Moving ahead, we'll now talk about connected and complete graphs.

Connected Graphs:

A connected graph, as the word connected suggests, is a graph that is **connected** in the sense of a topological space, i.e., that there is a path from any point to any other point in the graph. And the graph which is not connected is said to be **disconnected**. Consider the graph below:



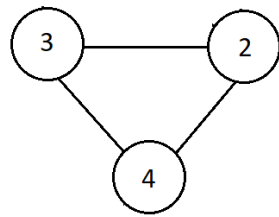
We have marked two random points P1 and P2, and we'll see if there is a path from P1 to P2. And if you could see, there is indeed a path from P1 to P2 via nodes 2 and 4.



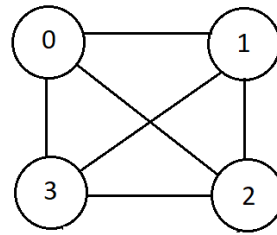
There could be a number of other ways to reach point P2 from P1, but there should exist at least one path from any point to another point for the graph to be called connected, otherwise disconnected. You should check for some other pair of vertices in the above graph.

Complete Graphs:

A complete graph is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge. Below, I have illustrated complete graphs with 3 and 4 nodes respectively.



Complete graph with 3 nodes

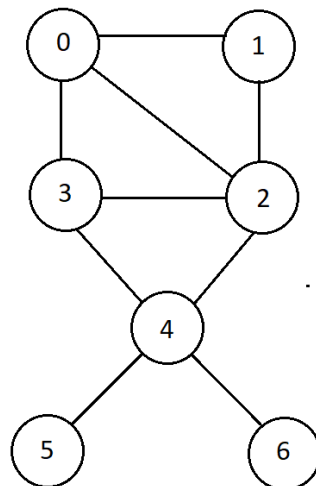


Complete graph with 4 nodes

As you can see, in any of the examples above, every pair of nodes is connected using a unique edge of their own. That is what makes a graph complete. You should consider making a complete graph with more vertices, say 5 or 7.

Note: Every complete graph is a connected graph, although every connected graph is not necessarily complete.

Quick Quiz:



1. Is the graph illustrated above connected?
2. Is the same graph complete?

Hint: Nodes 3 and 1 are not directly connected.

And now, having done subgraphs, connected and complete graphs, we are good to learn about spanning trees. Let's proceed with them.

What are spanning trees?

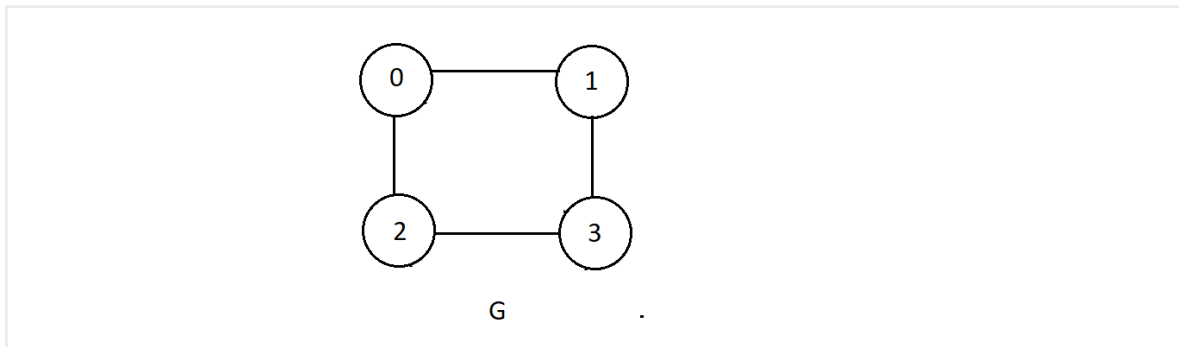
As we learnt, a subgraph of a graph G is a graph whose vertices and edges are subsets of the original graph G . Hence, a connected subgraph ' S ' of a graph $G (V, E)$ is said to be **a spanning tree** of the graph if and only if:

1. All the vertices of G are present in S ,
2. No. of edges in S should be $|V|-1$, where $|V|$ represents the number of

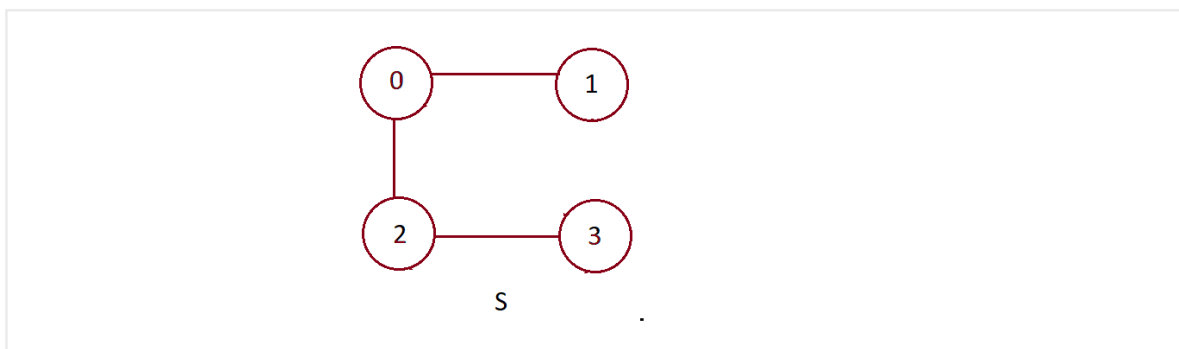
vertices.

That is, for a **subgraph** of a graph to be called a **spanning tree** of that graph, it should have **all vertices of the original graph** and must have exactly **$|V|-1$ edges**, where $|V|$ represents the number of vertices and the graph should be **connected**.

You might find this difficult to comprehend, but let me give you an example to help you understand. Consider a simple graph G I have illustrated below:



Now, consider another graph S.



Now, let's find out, step by step, if graph S is a spanning tree of graph G or not, considering nodes 1 and 3 don't have an edge in common.

- **Is graph S a subgraph of graph G? ✓**

Yes, graph S is a subgraph of graph G. All nodes/vertices present in S are also a part of graph G, and all vertices exist in graph G too.

- **Is graph S connected? ✓**

Yes, graph S is connected since we can go from any node to any other node via some edges in the graph.

- **Are all vertices of graph G present in graph S? ✓**

Yes, all vertices of graph G which are vertices 0, 1, 2, and 3, are present in graph S.

- **Does the number of edges in graph S equal the number of vertices in graph G - 1? ✓**

Yes, the number of edges in graph S equals the number of vertices in graph G - 1, since the number of vertices in graph G is 4, and the number of edges in graph S is 3.

Since graph S satisfies all the above conditions, it is a spanning tree of the graph G. Therefore, whenever you are given a graph and are asked whether this graph is a spanning tree of another graph or not, you should just simply check for these four conditions, and declare it a spanning tree only if it

satisfies all four conditions, and not a spanning tree even if it misses by any one of them.

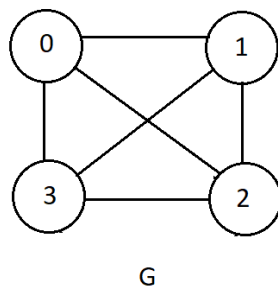
And if you are asked to create a spanning tree of a graph, you must first plot all the vertices of the original graph. And then create $V-1$ edges which were there in the original graph and what makes your graph connected.

This would be sufficient, and you can just ignore all other edges present in the original graph.

Now, you must be wondering about how many possible spanning trees are there for a graph. Well, there isn't a general formula for any random graph, but there is one for all complete graphs.

Number of Spanning trees for Complete graphs:

A complete graph has $n^{(n-2)}$ spanning trees where n represents the number of vertices in the graph. Consider the complete graph I have made below with 4 nodes.

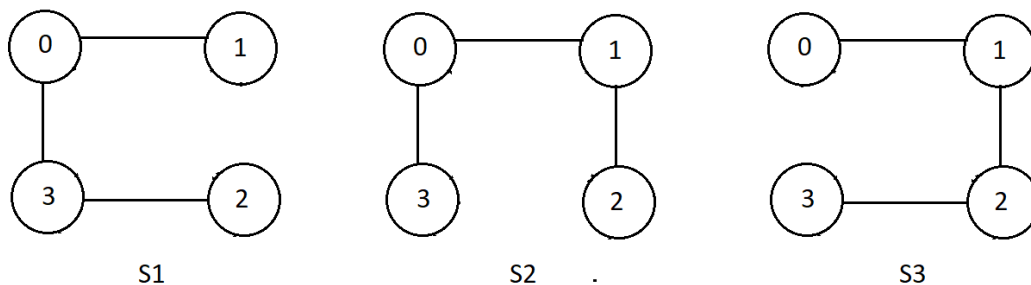


This complete graph has 4 vertices, hence the number of spanning trees it can have is, 4 raised to the power $(4-2)$, i.e., 4^2 which is 16.

Quick Quiz:

1. Draw any 3 spanning trees of the graph illustrated above.

Answer:



So, that was it for today. We learned what spanning trees are and other graph terminologies. Yet, so much more is left to the present. Next, we will learn about minimum spanning trees and spanning tree cost calculation, and more. Stay tuned.

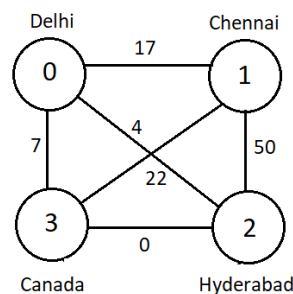
Calculating Spanning tree Cost & Minimum Spanning Tree

In the last lecture, we introduced to you the concept of a spanning tree. We discussed the conditions required for a graph to be called a spanning tree for another graph and several other terminologies related to graphs. Today, we'll delve deep into spanning trees and see their applications as well as learn what minimum cost spanning trees are.

For easy understanding of things, I'll walk you all through an instance where there is a subject named *Prem*, and he's desperate to meet his beloved who is residing currently at someplace whose location he is not aware of.

Although he has good options for places to visit and there are different routes to see all those places. *Prem* is not sort of a rich brat. He would definitely love to save money while he travels through places.

Consider the graph given below and the places are shown in the graph are the ones *Prem* has to visit. He wants to travel through all the places and at the same time for minimum possible expenditure. Each two places are connected through a route and every route would have some travel cost as well.



Prem, therefore, needs to come up with an algorithm that will help him find his beloved by wandering through all possible locations and at the least possible cost. And this is where finding a minimum cost spanning tree helps.

Let's give ourselves a quick revision of a few things:

- **Connected Graphs:**

A connected graph is a graph where there is a path from any point to any other point in the graph. And the graph which is not connected is said to be disconnected.

- **Complete Graphs:**

A complete graph is a simple undirected graph in which every pair of distinct vertices is connected by a unique edge.

- **Spanning trees:**

We learned that any graph *S* which satisfies below mentioned four conditions would be considered a spanning tree for another graph *G*. Those four conditions were:

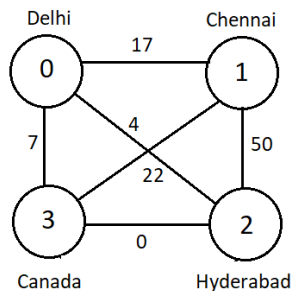
1. Graph *S* is a subgraph of graph *G*.
2. Graph *S* is a connected graph.
3. All vertices of graph *G* are present in graph *S*.

4. The number of edges in graph S equals the number of vertices in graph $G - 1$.

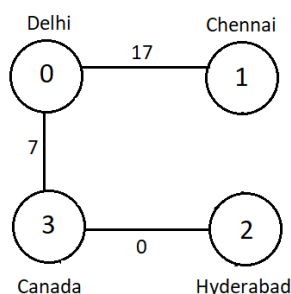
Having done the revision, we'll now look to proceed with what we mean by the cost of a spanning tree.

Cost of a Spanning Tree:

The cost of the spanning tree is the sum of the weights of all the edges in the tree. Now, consider the example of *Prem* we took at the beginning, and the graph he was to create a spanning tree of.

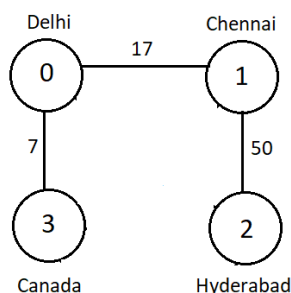


Now, suppose we created a spanning tree out of it which I've illustrated below.



S1

What would be the cost of this spanning tree? Yes, $17+7+0$ which is, 24. Let this cost for the spanning tree $S1$, be $Cost1$. Suppose another spanning tree of the same graph be:



S2

This spanning tree has a cost worth $17+7+50$, which is 74, and which is definitely greater than 24 what we calculated earlier for $S1$. And

therefore, *Prem* would definitely not want to incur some cost greater than 24. Let's move ahead now and see what a minimum spanning tree is.

Minimum spanning tree:

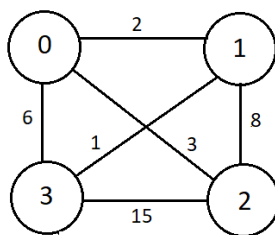
A Minimum Spanning tree, abbreviated as MST, is the spanning tree having the minimum cost. A graph can have a number of spanning trees all having some cost, say S1, S2, S3, S4, and S5 having cost 100, 104, 500, 400, and 10 respectively, but the one incurring the minimum cost is called the minimum spanning tree. Here, S5 having cost 10 would be the minimum spanning tree.

Applications of Minimum Spanning Tree:

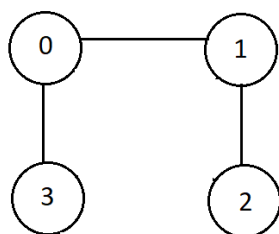
Applications of a minimum spanning tree must have gotten somewhat clear from the example of *Prem*, where we want to traverse all the nodes, which means the graph remains connected but it must have only bare minimum costing edges. So, this was a basic application of a minimum spanning tree. Things will gradually get clearer. Let's take some graphs and do some exercises on them ourselves.

Exercise:

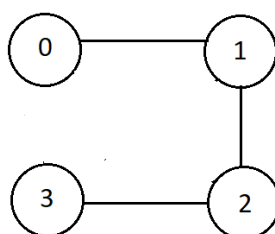
- Find the cost of any 3 spanning trees of the graph given below.



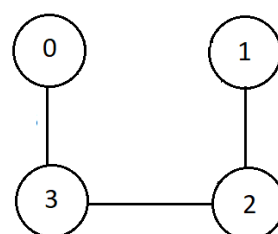
Consider the given graph be G, and our first step would be to create three spanning trees S1, S2, and S3 of the graph G which we have already learned in the last class.



S1

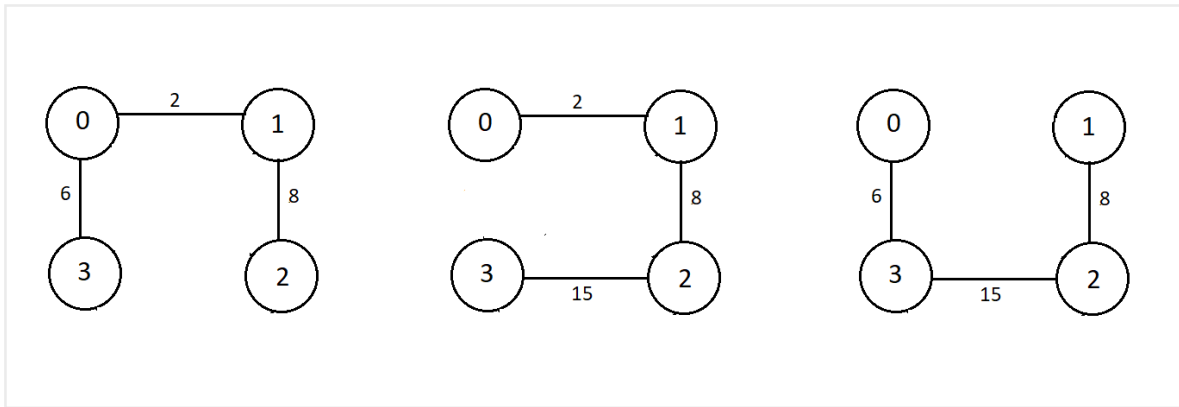


S2



S3

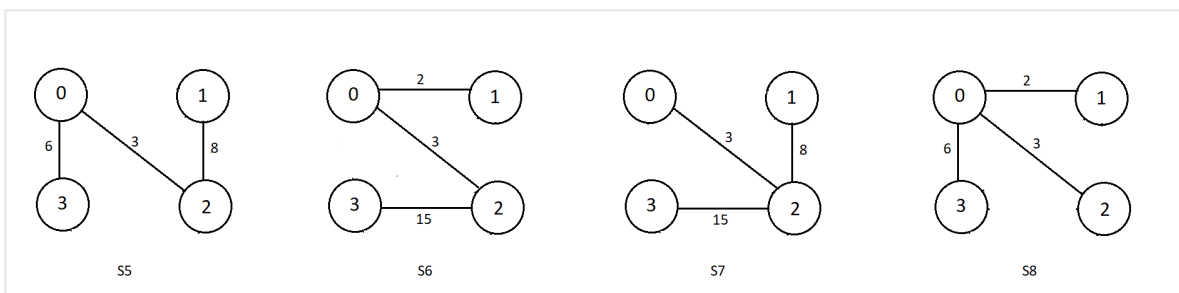
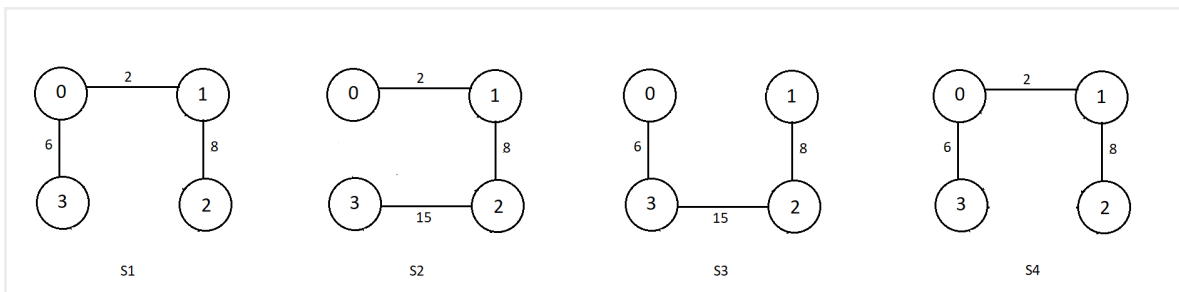
Then, mention weights to all the edges.

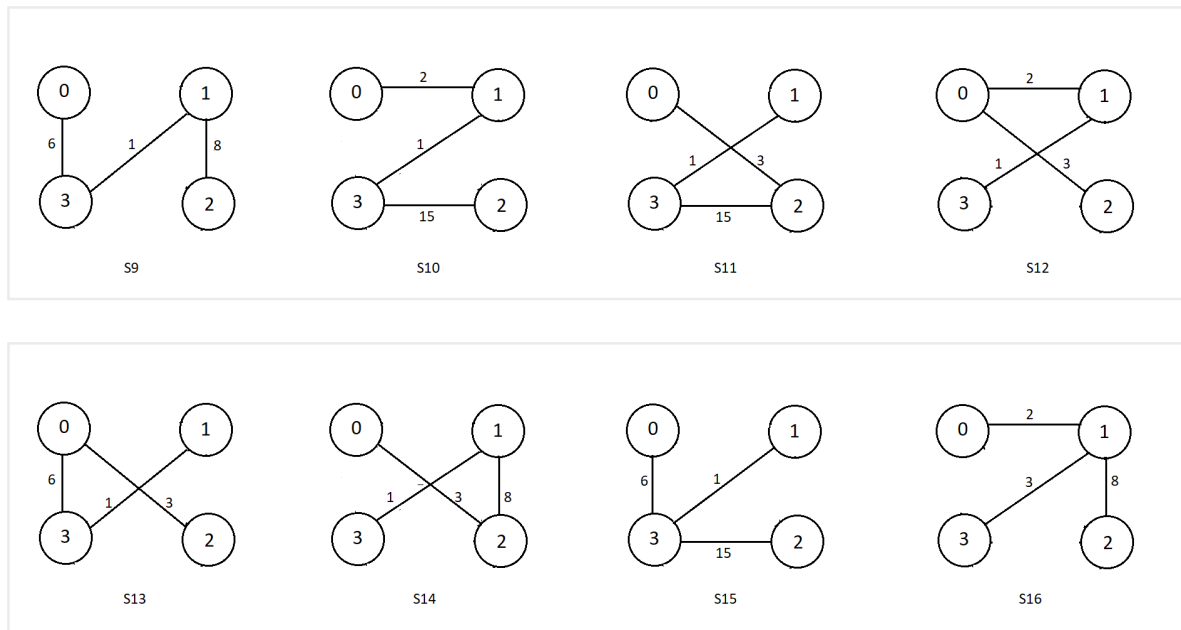


Now, let C_1 , C_2 , and C_3 be the cost of spanning trees S_1 , S_2 , and S_3 respectively. Therefore, C_1 is $(6+2+8) = 16$, C_2 is $(2+8+15) = 25$ and C_3 is $(6+15+8) = 29$. Hence the cost of these spanning trees is 16, 25, 29 respectively. One thing we can infer from this is, C_1 is less than C_2 , and C_2 is less than C_3 , i.e., $C_1 < C_2 < C_3$. As a result, C_1 is preferred to both C_2 and C_3 , and C_2 is preferred to C_3 .

- **Find the minimum spanning tree of the same graph we considered in the previous exercise.**

Since graph G is a complete graph and it has a total of four nodes, the total number of spanning trees possible for graph G would be 4 raised to $(4-2)$, which is 4^2 , 16. So, the first step would be to create all the 16 possible spanning trees. All of them are illustrated below along with the edge weights.





So, these were all the possible spanning trees we had. The rest I'll leave to you. You first have to count the cost of all these spanning trees, and then choose the minimum out of them. Tell me in the comments if you did all of them. I'll be more than happy to know if you could.

That was all for today. I hope things are coming together for you. We'll do more of these lectures. Next, we'll discuss Prim's algorithm, one of the reasons I chose *Prem* as the subject.:) This is to automate the process of finding the minimum spanning trees of graphs. Another algorithm is Kruskal's. Just stay, and you'll get to learn all of it.

Prims Minimum Spanning Tree Algorithm (Step by Step with examples)

In the last lecture, we learnt about the cost of a spanning tree, and we discussed what a minimum spanning tree is along with some of its applications. Today, we will see Prim's algorithm which is one of the algorithms to find the minimum spanning tree of a graph.

If you remember, we took the example of *Prem* and solved his dilemma of choosing the best possible route to traverse every city in search of his beloved while spending the least possible money. It is here that his big brother, *Prim*, steps in to help. His brother, *Prim*, has an algorithm named after him, the Prim's algorithm.

Rather than diving into the details of the algorithm right now, let's take a moment to revisit the concepts we've learned so far.

Connected graphs are graphs where every node in the graph has a path to every other node. A **complete graph** is one in which every pair of nodes has a unique edge. You should know those four conditions for a tree to be a **spanning tree** of some other graph. The **Cost of a spanning tree** is the sum of the weights of all the edges in the tree. And a **minimum spanning tree** has the minimum cost over all spanning trees of a graph.

Quick Quiz: Why is a spanning tree called '*spanning*' '*tree*'?

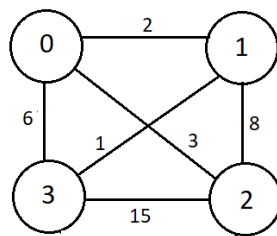
Hint: It spans over the whole graph, and it is a tree.

This gives us the green light to move forward with Prim's algorithm.

Prim's Algorithm:

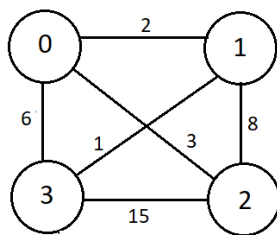
- Prim's algorithm uses the Greedy approach to find the minimum spanning tree.
- There are mainly two steps that this algorithm follows to find the minimum spanning tree of a graph:
 1. We start with any node and start creating the Minimum Spanning Tree.
 2. In Prim's Algorithm, we grow the spanning tree from a starting position until $n-1$ edges are or all nodes are covered.

Let me now take a simple example. Consider the complete graph with 4 vertices, K_4 , illustrated below:

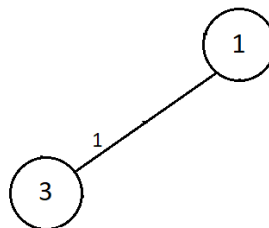


K_4

Now, as the first step says, we can start without any node in the graph. Arbitrarily, we will choose node 1. And since the edge between nodes 1 and 3 is the least weighted, we'll consider this edge in our minimum spanning tree.

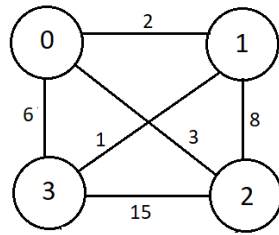


K_4

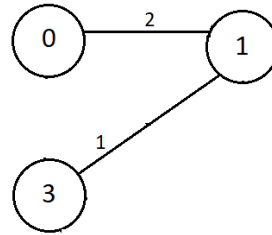


Spanning Tree

Next, we'll try involving node 0 in our spanning tree. Possible candidates for connecting node 0 from either node 3 or node 1 which compose the current spanning tree are edges having weights 6 and 2. Obviously, we'll choose the one with weight 2.

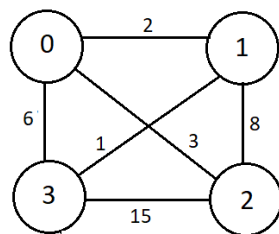


K_4

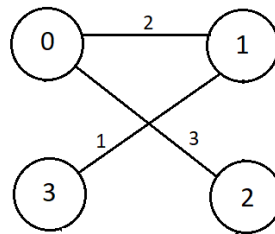


Spanning Tree

Now, the only node left is node 2. Possible candidates for connecting this node to the current spanning tree are edges with weights 15, 3, and 8. And there shouldn't be any doubt while considering the edge with weight 3.



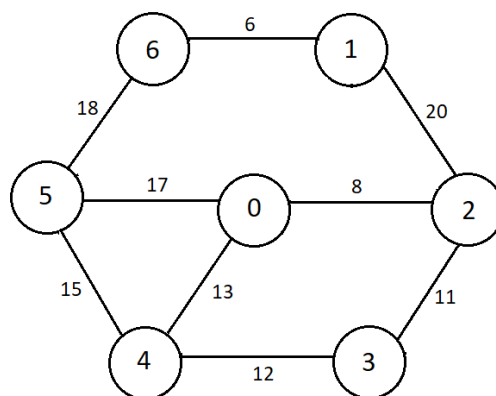
K_4



Spanning Tree

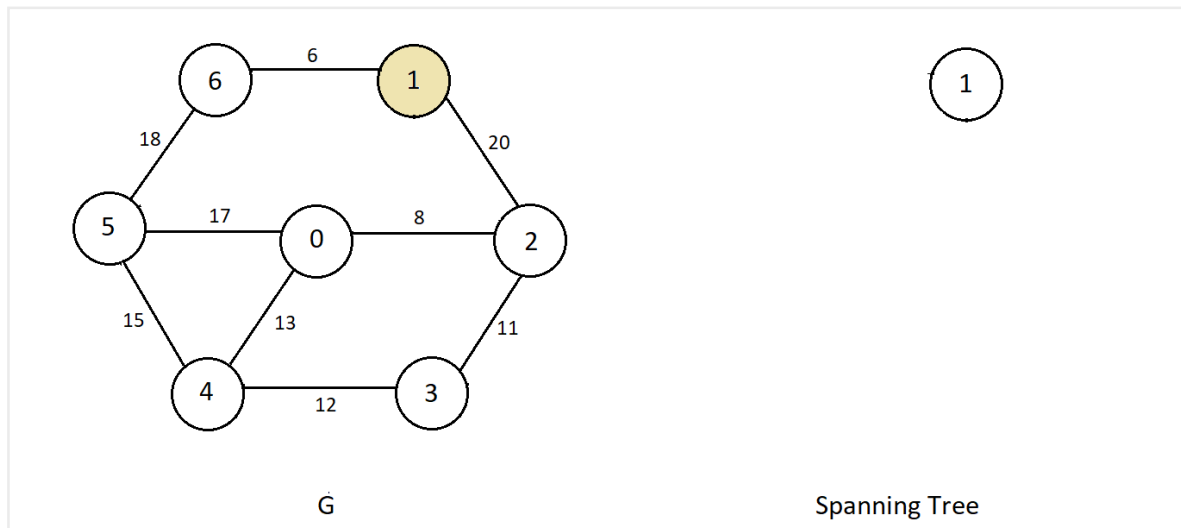
The cost of this spanning tree is $(1+2+3)$, i.e., 6 which is the minimum possible. And that finishes our construction of the spanning tree for graph K_4 . And this is exactly the way Prim's algorithm works.

Now, I'll give you one relatively tough example to get a better feel of the algorithm. Consider the graph I've illustrated below.

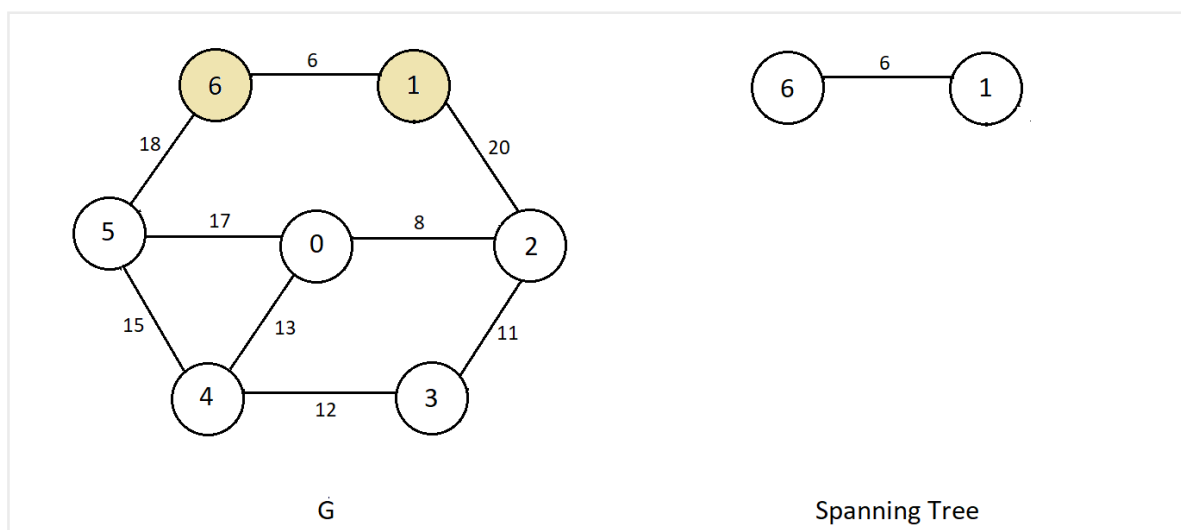


G

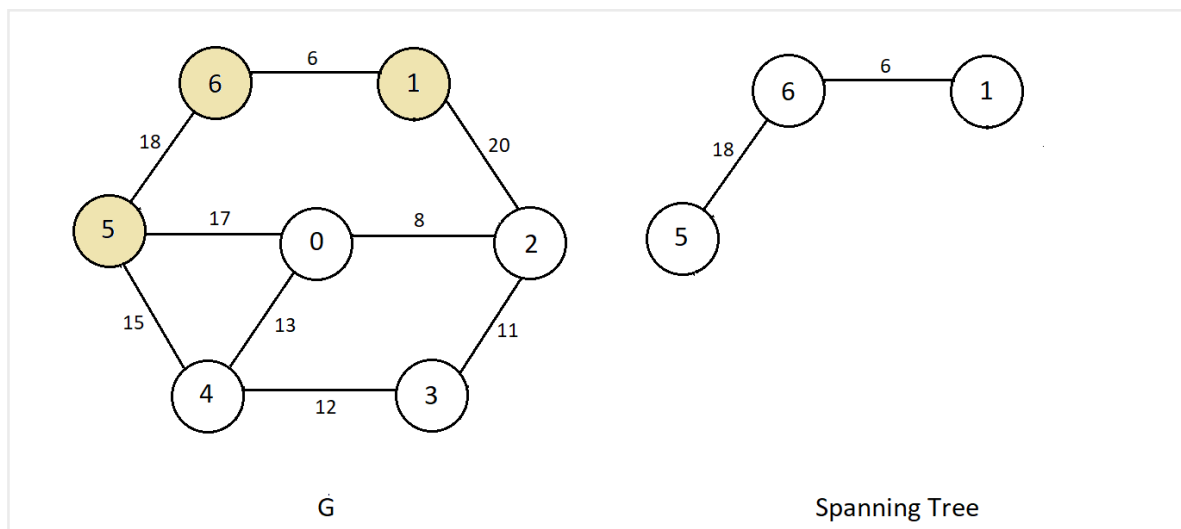
The first step, as the algorithm says, is to choose any arbitrary node and start creating the spanning tree. We have chosen node 1 for the same.



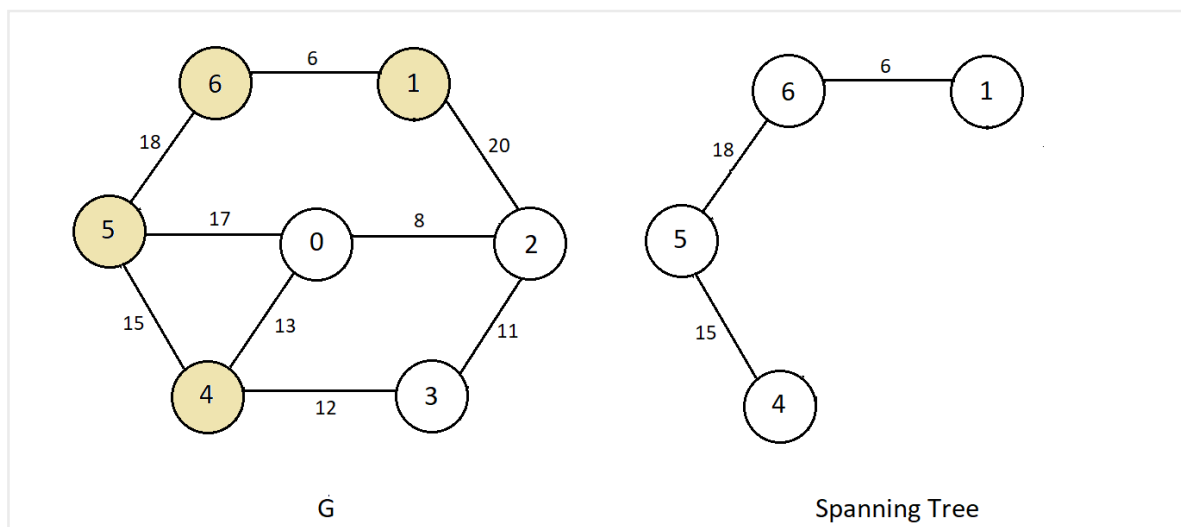
Colored nodes indicate that they have been considered for the spanning tree while the rest of the nodes are still left to cover. Here, in the Prim's algorithm, we maintain two sets of nodes, one for the nodes counted in the spanning tree, A , and another for the rest of them, V . Nodes connected to node 1 are 6 and 2, connected through edges of weights 6 and 20 respectively. Therefore, we will choose the one with a weight of 6.



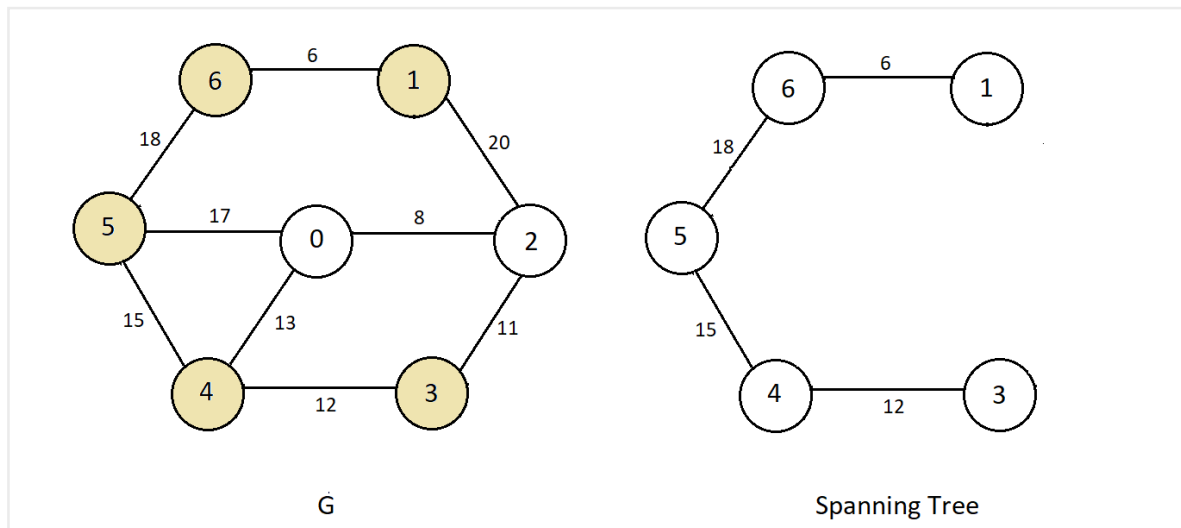
Now, 6 gets added to the set containing the covered elements. Nodes possible to get connected to the current spanning tree are 5 and 2. Node 5 and node 2 are connected to node 6 and node 1 respectively through edges of weights 18 and 20. We will obviously choose the one with less weight, i.e., weighted edge 18.



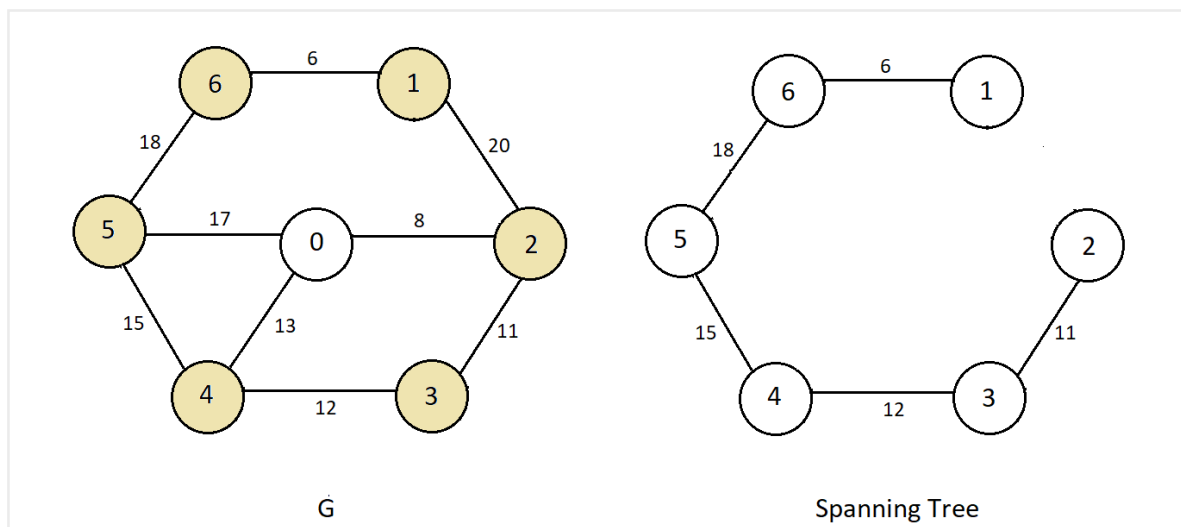
Now, 5 gets added to the set containing the covered elements. Possible nodes to get connected to the current spanning tree are 4, 0, and 2. Node 4 and node 0 are connected to node 5 through edges of weights 15 and 17 and node 2 is connected to node 1 through an edge of weight 20. We will, therefore, choose the one with weight $\min(15, 17, 20)$ i.e., 15. Hence, node 4 gets added to the current spanning tree.



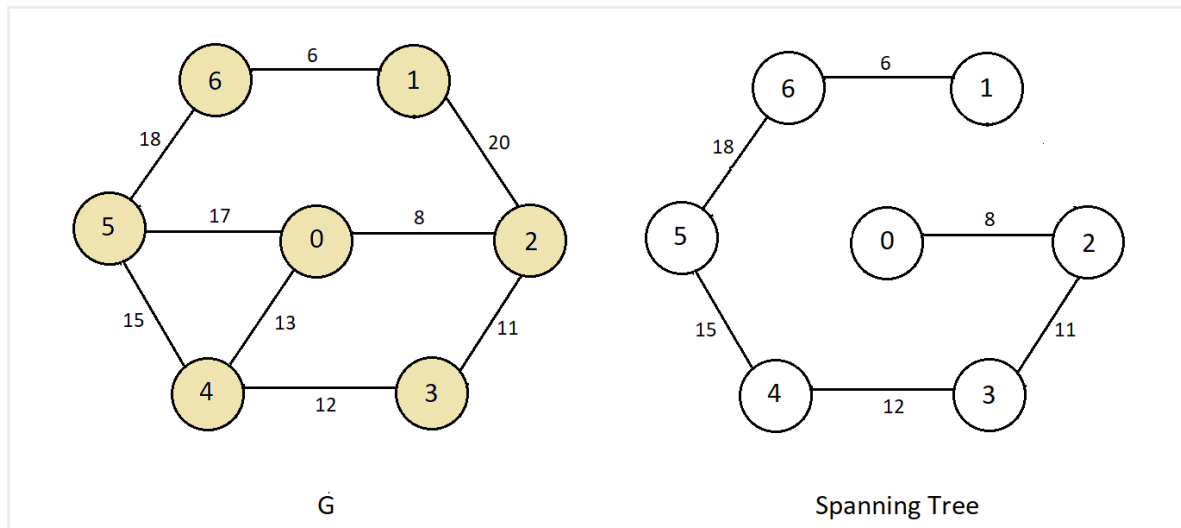
Now, we have nodes 0, 2, and 3 as possible candidates to get connected to the current spanning tree. Node 0 is connected to node 5 and node 4 through edges of weights 17 and 13 respectively. Node 3 is connected to node 4 through an edge of weight 12, and node 2 is connected to node 1 through an edge of weight 20. Therefore, we'll choose the one with weight $\min(17, 13, 12, 20)$ i.e., 12. Hence, node 3 gets added to the current spanning tree.



Next, we are left with only two nodes to cover. Node 0 is connected to node 5 and node 4 through edges of weights 17 and 13 respectively. Similarly, Node 2 is connected to node 1 and node 3 through edges of weights 20 and 11 respectively. Therefore, we'll choose the one with weight $\min(17, 13, 20, 11)$ i.e., 11. Hence, node 2 gets added to the current spanning tree through node 3.



The only node left now is node 0. We have 3 possible ways to connect this node to the rest of the spanning tree. There are edges of weights 17, 13, and 8 through nodes 5, 4, and 2 respectively. Therefore, we'll choose the edge with a weight of 8.



This completes our spanning tree with 7 nodes and 6 edges. The cost of this spanning tree sums to $(6+18+15+12+11+8)$, which is equal to 70. And you could verify if this is actually the minimum of all possible spanning trees. This is how Prim's algorithm works. It maintains the sets of elements covered and uncovered and sees the least possible edge to connect any one of the covered nodes to the uncovered ones. And the algorithm stops functioning once the set with uncovered nodes gets emptied.

And this is where we wrap up for today. I hope you could understand Prim's algorithm, and ultimately, we could solve Prem's dilemma as well. Another algorithm that is often used in finding the spanning tree of a graph is the Kruskal's. We'll see it too in the coming lectures. Stay tuned.